K Resilient Suite React

Source Code Submission

Generated: 2026-05-28 20:08:51

Included source files: 56

Table of Contents / Included Files

Table of Contents / Included Files

package.json

```
{
  "name": "k-resilient-suite-react",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "tauri": "tauri"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "@tauri-apps/api": "^2",
    "@tauri-apps/plugin-opener": "^2"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^6.0.3",
    "@tauri-apps/cli": "^2"
  }
}
```

src-tauri/Cargo.toml

```
[package]
name = "k-resilient-suite-react"
version = "1.0.0"
description = "A Tauri App"
authors = ["PWKHO"]
edition = "2021"
default-run = "k-resilient-suite-react"

# See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

[lib]
# The `_lib` suffix may seem redundant but it is necessary
# to make the lib name unique and wouldn't conflict with the bin name.
# This seems to be only an issue on Windows, see https://github.com/rust-lang/cargo/issues/8519
name = "k_resilient_suite_react_lib"
crate-type = ["staticlib", "cdylib", "rlib"]

[build-dependencies]
tauri-build = { version = "2", features = [] }

[dependencies]
tauri = { version = "2", features = [] }
tauri-plugin-opener = "2"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
aes-gcm = "0.10.3"
generic-array = "0.14.7"
mcore= {path="mcore", version="0.1.0"}
rand="0.8.4"
once_cell = "1.18"
mysql = "23.0"
log = "0.4"
rayon = "1.10"
serde_cbor = "0.11"
base64 = "0.21"
sha2 = "0.10"
hex = "0.4"
```

src-tauri/src/bin/enron_benchmark.rs

```
fn main() {
    let config = match k_resilient_suite_react_lib::system::benchmark::BenchmarkConfig::from_args(
        std::env::args().skip(1),
    ) {
        Ok(config) => config,
        Err(error) => {
            eprintln!("Benchmark argument error: {error}");
            eprintln!(
                "Usage: cargo run --release --bin enron_benchmark -- [--threads N] [--debug-raw]"
            );
            std::process::exit(2);
        }
    };

    if let Err(error) = k_resilient_suite_react_lib::system::benchmark::run_enron_benchmark(config)
    {
        eprintln!("Benchmark failed: {error}");
        std::process::exit(1);
    }
}
```

src-tauri/src/kr_ibe/api.rs

```
use super::ciphertext::Ciphertext;
use super::main as kribe_core;
use super::params::Params;
use super::plaintext::Plaintext;
use super::private_key::PrivateKey;

use once_cell::sync::Lazy;
use std::sync::Mutex;
use std::time::Instant;

static PARAMS: Lazy<Mutex<Option<Params>>> = Lazy::new(|| Mutex::new(None));
static SK: Lazy<Mutex<Option<PrivateKey>>> = Lazy::new(|| Mutex::new(None));
static CT: Lazy<Mutex<Option<Ciphertext>>> = Lazy::new(|| Mutex::new(None));
static PT: Lazy<Mutex<Option<Plaintext>>> = Lazy::new(|| Mutex::new(None));
static TOTAL_RUNTIME: Lazy<Mutex<f64>> = Lazy::new(|| Mutex::new(0.0)); // Total sum in seconds

#[tauri::command]
pub fn kr_ibe_setup(k: usize) -> String {
    let start_time = Instant::now();

    let mut params = Params::new();
    kribe_core::setup(&mut params, k);

    let duration = start_time.elapsed();
    *PARAMS.lock().unwrap() = Some(params);
    *TOTAL_RUNTIME.lock().unwrap() = 0.0; // Reset total runtime when setup new
    *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate setup time

    let mut output = String::new();
    output.push_str("  KR-IBE Setup Complete!\n\n");
    output.push_str(&PARAMS.lock().unwrap().as_ref().unwrap().format_full());
    output.push_str(&format!("\n  Setup Time: {:.2?}\n", duration));

    output
}

#[tauri::command]
pub fn kr_ibe_extract(id: String) -> String {
    let start_time = Instant::now();

    let id_bytes = id.into_bytes();
    let maybe_params = PARAMS.lock().unwrap();
    if maybe_params.is_none() {
        return "  Params not initialized. Please run setup first.".into();
    }

    let params = maybe_params.as_ref().unwrap();
    let mut sk = PrivateKey::new();

    kribe_core::extract(params, &mut sk, &id_bytes);

    let duration = start_time.elapsed();
    *SK.lock().unwrap() = Some(sk);
    *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate extract time

    let mut output = String::new();
    output.push_str("  Private Key Extracted.\n\n");
    output.push_str(&SK.lock().unwrap().as_ref().unwrap().format_full());
    output.push_str(&format!("\n  Extract Time: {:.2?}\n", duration));

    output
}

#[tauri::command]
pub fn kr_ibe_encrypt(id: String, plaintext: String) -> String {
    let start_time = Instant::now();

    let id_bytes = id.into_bytes();
    let msg_bytes = plaintext.into_bytes();

    let maybe_params = PARAMS.lock().unwrap();
    if maybe_params.is_none() {
        return "  Params not initialized. Please run setup first.".into();
    }

    let params = maybe_params.as_ref().unwrap();
    let mut ciphertext = Ciphertext::new();

    kribe_core::encryption(params, &mut ciphertext, &id_bytes, &msg_bytes);
```

src-tauri/src/kr_ibe/api.rs (continued)

```
    let duration = start_time.elapsed();
    *CT.lock().unwrap() = Some(ciphertext);
    *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate encryption time

    let mut output = String::new();
    output.push_str("  Encryption Completed.\n\n");
    output.push_str(&CT.lock().unwrap().as_ref().unwrap().format_full());
    output.push_str(&format!("\n  Encryption Time: {:.2?}\n", duration));

    output
}

#[tauri::command]
pub fn kr_ibe_decrypt() -> String {
    let start_time = Instant::now();

    let mut pt = Plaintext::new();
    {
        let mut pt_lock = PT.lock().unwrap();

        let params_guard = PARAMS.lock().unwrap();
        let sk_guard = SK.lock().unwrap();
        let mut ct_guard = CT.lock().unwrap();

        if params_guard.is_none() || sk_guard.is_none() || ct_guard.is_none() {
            return "  Missing data. Please run setup, extract, and encrypt first.".into();
        }

        let params = params_guard.as_ref().unwrap();
        let sk = sk_guard.as_ref().unwrap();
        let ct = ct_guard.as_mut().unwrap();

        kribe_core::decryption(params, sk, ct, &mut pt);

        *pt_lock = Some(pt);
    }

    let decryption_duration = start_time.elapsed();
    *TOTAL_RUNTIME.lock().unwrap() += decryption_duration.as_secs_f64(); // accumulate decryption time

    let pt_read = PT.lock().unwrap();
    let total_runtime = *TOTAL_RUNTIME.lock().unwrap(); // sum total

    let mut output = String::new();
    output.push_str("  Decryption Completed.\n\n");
    output.push_str(&pt_read.as_ref().unwrap().format_full());
    output.push_str(&format!(
        "\n  Decryption Time: {:.2?}\n",
        decryption_duration
    ));
    output.push_str(&format!(
        "  Total Computation Time: {}\n",
        format_runtime(total_runtime)
    ));

    output
}

fn format_runtime(seconds: f64) -> String {
    if seconds < 1.0 {
        // Less than 1 second → show milliseconds
        format!("{:.0} ms", seconds * 1000.0)
    } else {
        // 1 second or more → show seconds
        format!("{:.2} s", seconds)
    }
}
```

src-tauri/src/kr_ibe/ciphertext.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;

#[derive(Clone)]
pub struct Ciphertext {
    u1: ecp::ECP,
    u2: ecp::ECP,
    c: ecp::ECP,
    v_id: ecp::ECP,
    aes_cipher: Vec<u8>,
}

impl Ciphertext {
    pub fn new() -> Self {
        Ciphertext {
            u1: ecp::ECP::new(),
            u2: ecp::ECP::new(),
            c: ecp::ECP::new(),
            v_id: ecp::ECP::new(),
            aes_cipher: Vec::new(),
        }
    }

    pub fn new_ciphertext(
        u1: &ecp::ECP,
        u2: &ecp::ECP,
        c: &ecp::ECP,
        v_id: &ecp::ECP,
        aes_cipher: Vec<u8>,
    ) -> Self {
        Ciphertext {
            u1: u1.clone(),
            u2: u2.clone(),
            c: c.clone(),
            v_id: v_id.clone(),
            aes_cipher,
        }
    }

    pub fn set_ciphertext(
        &mut self,
        u1: ecp::ECP,
        u2: ecp::ECP,
        c: ecp::ECP,
        v_id: ecp::ECP,
        aes_cipher: Vec<u8>,
    ) {
        self.u1 = u1;
        self.u2 = u2;
        self.c = c;
        self.v_id = v_id;
        self.aes_cipher = aes_cipher;
    }

    pub fn get_u1(&self) -> &ecp::ECP {
        &self.u1
    }

    pub fn get_u2(&self) -> &ecp::ECP {
        &self.u2
    }

    pub fn get_c(&self) -> &ecp::ECP {
        &self.c
    }

    pub fn get_v_id(&self) -> &ecp::ECP {
        &self.v_id
    }

    pub fn get_aes_cipher(&self) -> &Vec<u8> {
        &self.aes_cipher
    }

    pub fn print(&self) {
        println!("========Begin Ciphertext=========");
        println!("u1: {}", ecp_to_hex(&self.u1));
        println!("u2: {}", ecp_to_hex(&self.u2));
```

src-tauri/src/kr_ibe/ciphertext.rs (continued)

```
        println!("c: {}", ecp_to_hex(&self.c));
        println!("v_id: {}", ecp_to_hex(&self.v_id));
        println!("aes_cipher: {}", bytes_to_hex(&self.aes_cipher));
        println!("========End of Ciphertext=========");
    }

    pub fn format_full(&self) -> String {
        let mut str = String::new();
        str.push_str("\n========Begin Ciphertext=========\n");
        str.push_str(&format!("u1: {}\n", ecp_to_hex(&self.u1)));
        str.push_str(&format!("u2: {}\n", ecp_to_hex(&self.u2)));
        str.push_str(&format!("c: {}\n", ecp_to_hex(&self.c)));
        str.push_str(&format!("v_id: {}\n", ecp_to_hex(&self.v_id)));
        str.push_str(&format!("aes_cipher: {}\n", bytes_to_hex(&self.aes_cipher)));
        str.push_str("========End of Ciphertext========\n");
        return str;
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}

fn bytes_to_hex(bytes: &[u8]) -> String {
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}
```

src-tauri/src/kr_ibe/main.rs

```
#![allow(non_snake_case)]

use super::ciphertext::Ciphertext;
use super::params::Params;
use super::plaintext::Plaintext;
use super::polynomial;
use super::private_key::PrivateKey;

extern crate mcore;

use aes_gcm::{
    aead::{Aead, KeyInit, OsRng},
    Aes256Gcm, Nonce,
};
use generic_array::typenum::U32;
use generic_array::GenericArray;
use mcore::ed25519::big;
use mcore::ed25519::big::MODBYTES;
use mcore::ed25519::ecdh;
use mcore::ed25519::ecp;
use mcore::ed25519::rom;
use mcore::rand::RAND;
use mcore::sha3::{SHA3, SHAKE256};
use rand::RngCore;

use std::time::Instant;

pub fn setup(params: &mut Params, k: usize) {
    let order = big::BIG::new_ints(&rom::CURVE_ORDER);

    let mut rng = gen_seed();
    let r = big::BIG::randomnum(&order, &mut rng);

    let g1 = ecp::ECP::generator();
    let g2 = g1.mul(&r);

    //validate g1 and g2
    if is_valid(&g1) != 0 {
        println!("g1 is invalid! Abort!");
        std::process::abort();
    }

    if is_valid(&g2) != 0 {
        println!("g2 is invalid! Abort!");
        std::process::abort();
    }

    //Choose the six random k-degree polynomials from Zq
    let f1 = polynomial::Polynomial::Gennew(k, &big::BIG::randomnum(&order, &mut rng), &order);
    let f2 = polynomial::Polynomial::Gennew(k, &big::BIG::randomnum(&order, &mut rng), &order);
    let h1 = polynomial::Polynomial::Gennew(k, &big::BIG::randomnum(&order, &mut rng), &order);
    let h2 = polynomial::Polynomial::Gennew(k, &big::BIG::randomnum(&order, &mut rng), &order);
    let p1 = polynomial::Polynomial::Gennew(k, &big::BIG::randomnum(&order, &mut rng), &order);
    let p2 = polynomial::Polynomial::Gennew(k, &big::BIG::randomnum(&order, &mut rng), &order);

    //Compute At, Bt, Dt
    let mut At = vec![ecp::ECP::new(); k];
    let mut Bt = vec![ecp::ECP::new(); k];
    let mut Dt = vec![ecp::ECP::new(); k];

    for i in 0..k {
        At[i] = g1.mul(&f1.get_coeff_at(i));
        At[i].add(&g2.mul(&f2.get_coeff_at(i)));

        Bt[i] = g1.mul(&h1.get_coeff_at(i));
        Bt[i].add(&g2.mul(&h2.get_coeff_at(i)));

        Dt[i] = g1.mul(&p1.get_coeff_at(i));
        Dt[i].add(&g2.mul(&p2.get_coeff_at(i)));
    }

    //Set the params
    params.set_params(k, order, g1, g2, At, Bt, Dt, f1, f2, h1, h2, p1, p2);
}

pub fn extract(params: &Params, sk: &mut PrivateKey, id: &Vec<u8>) {
    let order = params.get_order();

    //get the value of each polynomial
    let f1 = params.get_f1();
```

src-tauri/src/kr_ibe/main.rs (continued)

```
    let f2 = params.get_f2();
    let h1 = params.get_h1();
    let h2 = params.get_h2();
    let p1 = params.get_p1();
    let p2 = params.get_p2();

    //Hash the string ID into BIG
    let hash_id = hash_to_big(id, order);

    //Return the secret key
    let f1ID = f1.evaluate(&hash_id);
    let f2ID = f2.evaluate(&hash_id);
    let h1ID = h1.evaluate(&hash_id);
    let h2ID = h2.evaluate(&hash_id);
    let p1ID = p1.evaluate(&hash_id);
    let p2ID = p2.evaluate(&hash_id);

    sk.set_private_key(f1ID, f2ID, h1ID, h2ID, p1ID, p2ID);
}

pub fn encryption(params: &Params, ciphertext: &mut Ciphertext, id: &Vec<u8>, m: &Vec<u8>) {
    let order = params.get_order();

    //get the value of params
    let g1 = params.get_g1();
    let g2 = params.get_g2();
    let At = params.get_At();
    let Bt = params.get_Bt();
    let Dt = params.get_Dt();

    //validate g1 and g2
    if is_valid(&g1) != 0 {
        println!("g1 is invalid! Abort!");
        std::process::abort();
    }
    if is_valid(&g2) != 0 {
        println!("g2 is invalid! Abort!");
        std::process::abort();
    }

    let mut rng = gen_seed();

    //Hash the string ID into BIG
    let mut hash_id = hash_to_big(id, order);

    //E1
    let r1 = big::BIG::randomnum(&order, &mut rng);

    //E2
    let u1 = g1.mul(&r1);

    //E3
    let u2 = g2.mul(&r1);

    //E4
    let mut A_id = At[0].mul(&hash_id.powmod(&big::BIG::new_int(0), order));

    for i in 1..At.len() {
        let i_isize: isize = i.try_into().unwrap();
        let temp = At[i].mul(&hash_id.powmod(&big::BIG::new_int(i_isize), order));
        A_id.add(&temp);
    }

    let mut B_id = Bt[0].mul(&hash_id.powmod(&big::BIG::new_int(0), order));

    for i in 1..Bt.len() {
        let i_isize: isize = i.try_into().unwrap();
        let temp = Bt[i].mul(&hash_id.powmod(&big::BIG::new_int(i_isize), order));
        B_id.add(&temp);
    }

    let mut D_id = Dt[0].mul(&hash_id.powmod(&big::BIG::new_int(0), order));

    for i in 1..Dt.len() {
        let i_isize: isize = i.try_into().unwrap();
        let temp = Dt[i].mul(&hash_id.powmod(&big::BIG::new_int(i_isize), order));
        D_id.add(&temp);
    }

    //E5
    let s = D_id.mul(&r1);

    // AES-GCM Encryption Start
    // Generate a random AES key
    let key = Aes256Gcm::generate_key(OsRng); //32 bytes
```

```
    //convert key to ecp in order to be encrypted by IBE
    let ecp_key = ecp::ECP::mapit(&key);
    let mut bytes_ecp_key: Vec<u8> = vec![0; MODBYTES + 1];
    ecp_key.tobytes(&mut bytes_ecp_key, true);

    //split the AES key, first 32 bytes to be key, last 12 bytes to be the nonce (maybe overlap)
    let (aes_key, aes_nonce) = split_key_nonce(&bytes_ecp_key);

    // Convert Vec<u8> to GenericArray
    let generic_key = vec_to_generic_array(aes_key); //32 bytes
    let cipher = Aes256Gcm::new(&generic_key);

    // Nonce (must be unique for every encryption operation)
    let nonce = Nonce::from_slice(&aes_nonce); // 12 bytes

    // AES Encrypt message m
    let aes_ciphertext = cipher
        .encrypt(nonce, m.as_ref())
        .expect("encryption failure!"); // NOTE: handle this error appropriately!

    // AES-GCM Encryption End

    //E6
    let mut temp_m = ecp::ECP::frombytes(&bytes_ecp_key);
    temp_m.add(&s);
    let c = temp_m;

    //E7
    let mut temp_alpha = u1.clone();
    temp_alpha.add(&u2);
    temp_alpha.add(&c);
    let alpha = hash_ECP_to_big(temp_alpha);

    //E8
    let mut v_id = A_id.mul(&r1);
    let r1_alpha = big::BIG::modmul(&r1, &alpha, order);
    v_id.add(&B_id.mul(&r1_alpha));

    //E9
    ciphertext.set_ciphertext(u1, u2, c, v_id, aes_ciphertext);
}

pub fn decryption(
    params: &Params,
    sk: &PrivateKey,
    ciphertext: &mut Ciphertext,
    plaintext: &mut Plaintext,
) {
    let order = params.get_order();

    let u1 = ciphertext.get_u1();
    let u2 = ciphertext.get_u2();
    let c = ciphertext.get_c();
    let aes_cipher = ciphertext.get_aes_cipher();

    //D1
    let mut temp_alpha = u1.clone();
    temp_alpha.add(u2);
    temp_alpha.add(c);
    let alpha = hash_ECP_to_big(temp_alpha);

    //D2
    let v_id = ciphertext.get_v_id();

    let f1ID = sk.get_f1ID();
    let f2ID = sk.get_f2ID();
    let h1ID = sk.get_h1ID();
    let h2ID = sk.get_h2ID();

    let h1ID_alpha = big::BIG::modmul(h1ID, &alpha, order);
    let pow1 = big::BIG::modadd(f1ID, &h1ID_alpha, order);
    let mut temp_u1 = u1.clmul(&pow1, order);

    let h2ID_alpha = big::BIG::modmul(h2ID, &alpha, order);
    let pow2 = big::BIG::modadd(f2ID, &h2ID_alpha, order);
    let temp_u2 = u2.clmul(&pow2, order);

    temp_u1.add(&temp_u2);

    let temp_v_id = temp_u1;
    let is_equal_v_id = v_id.equals(&temp_v_id);

    if is_equal_v_id == true {
        //D3
```

src-tauri/src/kr_ibe/main.rs (continued)

```
        let p1ID = sk.get_p1ID();
        let p2ID = sk.get_p2ID();

        let temp_s1 = u1.clone();
        let temp_s2 = u2.clone();

        let mut temp_u1_p1ID = temp_s1.clmul(p1ID, order);
        let temp_u2_p2ID = temp_s2.clmul(p2ID, order);
        temp_u1_p1ID.add(&temp_u2_p2ID);
        let mut s = temp_u1_p1ID;

        //D4
        let mut temp_c = c.clone();
        temp_c.sub(&s);

        // AES-GCM Decryption Start
        // Convert ECP key back to AES key after IBE decryption
        let mut recovered_key = vec![0; MODBYTES + 1];
        temp_c.tobytes(&mut recovered_key, true);

        // Split the recovered AES key into aes_key and aes_nonce
        let (aes_key, aes_nonce) = split_key_nonce(&recovered_key);

        // Convert Vec<u8> to GenericArray
        let generic_key = vec_to_generic_array(aes_key); //32 bytes

        let cipher = Aes256Gcm::new(&generic_key);

        let nonce = Nonce::from_slice(&aes_nonce); // 12 bytes

        // Decrypt the AES ciphertext
        let decrypted_plaintext = cipher
            .decrypt(nonce, aes_cipher.as_ref())
            .expect("decryption failure!"); // NOTE: handle this error appropriately!

        // Convert the decrypted plaintext from Vec<u8> to String
        let decrypted_string = String::from_utf8(decrypted_plaintext)
            .expect("Failed to convert decrypted plaintext to string");

        // AES-GCM Decryption End

        plaintext.set_plaintext(decrypted_string);
    } else {
        println!("The v_id in E8 is not equal to v_id in D2 !");
    }
}

fn main() {
    let w = "Urgent";
    let id = "alice@mail.com";
    let k = 20;

    // Convert strings to byte arrays
    let w_bytes = string_to_bytes(w);
    let id_bytes = string_to_bytes(id);

    let mut params = Params::new();
    let mut sk = PrivateKey::new();
    let mut ciphertext = Ciphertext::new();
    let mut plaintext = Plaintext::new();

    let start = Instant::now();
    setup(&mut params, k);
    let duration = start.elapsed();
    params.print();
    println!("Setup took: {:?}", duration);

    let start = Instant::now();
    extract(&params, &mut sk, &id_bytes);
    let duration = start.elapsed();
    sk.print();
    println!("Extract took: {:?}", duration);

    let start = Instant::now();
    encryption(&params, &mut ciphertext, &id_bytes, &w_bytes);
    let duration = start.elapsed();
    ciphertext.print();
    println!("Encryption took: {:?}", duration);

    let start = Instant::now();
    decryption(&params, &mut sk, &mut ciphertext, &mut plaintext);
    let duration = start.elapsed();
    plaintext.print();
    println!("Decryption took: {:?}", duration);
}
```

src-tauri/src/kr_ibe/main.rs (continued)

```
fn gen_seed() -> RAND {
    let mut raw: [u8; 100] = [0; 100];
    let mut rng = RAND::new();
    rng.clean();
    OsRng.fill_bytes(&mut raw);
    rng.seed(100, &raw);
    rng
}

fn is_valid(p: &ecp::ECP) -> isize {
    let mut bytes = vec![0; big::MODBYTES + 1];
    p.tobytes(&mut bytes, true);

    let result = ecdh::public_key_validate(&bytes);
    if result != 0 {
        return result;
    }
    0
}

pub fn hash_to_big(input: &[u8], order: &big::BIG) -> big::BIG {
    let mut hasher = SHA3::new(SHAKE256);
    hasher.process_array(input);
    let mut output = [0u8; big::MODBYTES];
    hasher.shake(&mut output, big::MODBYTES);
    let mut v = big::BIG::frombytes(&output);
    v.rmod(order);
    v
}

fn hash_ECP_to_big(input: ecp::ECP) -> big::BIG {
    let mut hasher = SHA3::new(SHAKE256);
    let mut b: Vec<u8> = vec![0; MODBYTES + 1];
    input.tobytes(&mut b, true);
    hasher.process_array(&b);
    let mut output = [0u8; MODBYTES];
    hasher.shake(&mut output, MODBYTES);
    big::BIG::frombytes(&output)
}

fn split_key_nonce(key_nonce: &[u8]) -> (Vec<u8>, Vec<u8>) {
    let aes_key = key_nonce[0..32].to_vec(); // First 32 bytes
    let aes_nonce = key_nonce[key_nonce.len() - 12..key_nonce.len()].to_vec(); // Last 12 bytes

    (aes_key, aes_nonce)
}

fn vec_to_generic_array(vec: Vec<u8>) -> GenericArray<u8, U32> {
    generic_array::GenericArray::from_slice(&vec).clone()
}

fn string_to_bytes(input: &str) -> Vec<u8> {
    input.as_bytes().to_vec()
}
```

src-tauri/src/kr_ibe/mod.rs

```
pub mod api;
pub mod ciphertext;
pub mod main;
pub mod params;
pub mod plaintext;
pub mod polynomial;
pub mod private_key; // your crypto logic
```

src-tauri/src/kr_ibe/params.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;

use super::polynomial;
use super::polynomial::Polynomial;

#[derive(Clone)]
pub struct Params {
    k: usize,
    order: big::BIG,
    g1: ecp::ECP,
    g2: ecp::ECP,
    At: Vec<ecp::ECP>,
    Bt: Vec<ecp::ECP>,
    Dt: Vec<ecp::ECP>,
    f1: polynomial::Polynomial,
    f2: polynomial::Polynomial,
    h1: polynomial::Polynomial,
    h2: polynomial::Polynomial,
    p1: polynomial::Polynomial,
    p2: polynomial::Polynomial,
}

impl Params {
    pub fn new() -> Self {
        Params {
            k: 0,
            order: big::BIG::new(),
            g1: ecp::ECP::new(),
            g2: ecp::ECP::new(),
            At: Vec::new(),
            Bt: Vec::new(),
            Dt: Vec::new(),
            f1: Polynomial::new(),
            f2: Polynomial::new(),
            h1: Polynomial::new(),
            h2: Polynomial::new(),
            p1: Polynomial::new(),
            p2: Polynomial::new(),
        }
    }

    pub fn new_params(
        k: &usize,
        order: &big::BIG,
        g1: &ecp::ECP,
        g2: &ecp::ECP,
        At: &Vec<ecp::ECP>,
        Bt: &Vec<ecp::ECP>,
        Dt: &Vec<ecp::ECP>,
        f1: polynomial::Polynomial,
        f2: polynomial::Polynomial,
        h1: polynomial::Polynomial,
        h2: polynomial::Polynomial,
        p1: polynomial::Polynomial,
        p2: polynomial::Polynomial,
    ) -> Self {
        Params {
            k: *k,
            order: *order,
            g1: g1.clone(),
            g2: g2.clone(),
            At: At.clone(),
            Bt: Bt.clone(),
            Dt: Dt.clone(),
            f1: f1,
            f2: f2,
            h1: h1,
            h2: h2,
            p1: p1,
            p2: p2,
        }
    }

    pub fn set_params(
        &mut self,
        k: usize,
        order: big::BIG,
```

src-tauri/src/kr_ibe/params.rs (continued)

```
        g1: ecp::ECP,
        g2: ecp::ECP,
        At: Vec<ecp::ECP>,
        Bt: Vec<ecp::ECP>,
        Dt: Vec<ecp::ECP>,
        f1: polynomial::Polynomial,
        f2: polynomial::Polynomial,
        h1: polynomial::Polynomial,
        h2: polynomial::Polynomial,
        p1: polynomial::Polynomial,
        p2: polynomial::Polynomial,
    ) {
        self.k = k;
        self.order = order;
        self.g1 = g1;
        self.g2 = g2;
        self.At = At;
        self.Bt = Bt;
        self.Dt = Dt;
        self.f1 = f1;
        self.f2 = f2;
        self.h1 = h1;
        self.h2 = h2;
        self.p1 = p1;
        self.p2 = p2;
    }

    pub fn get_k(&self) -> usize {
        self.k
    }

    pub fn get_order(&self) -> &big::BIG {
        &self.order
    }

    pub fn get_g1(&self) -> &ecp::ECP {
        &self.g1
    }

    pub fn get_g2(&self) -> &ecp::ECP {
        &self.g2
    }

    pub fn get_At(&self) -> &Vec<ecp::ECP> {
        &self.At
    }

    pub fn get_Bt(&self) -> &Vec<ecp::ECP> {
        &self.Bt
    }

    pub fn get_Dt(&self) -> &Vec<ecp::ECP> {
        &self.Dt
    }

    pub fn get_f1(&self) -> &Polynomial {
        &self.f1
    }

    pub fn get_f2(&self) -> &Polynomial {
        &self.f2
    }

    pub fn get_h1(&self) -> &Polynomial {
        &self.h1
    }

    pub fn get_h2(&self) -> &Polynomial {
        &self.h2
    }

    pub fn get_p1(&self) -> &Polynomial {
        &self.p1
    }

    pub fn get_p2(&self) -> &Polynomial {
        &self.p2
    }

    pub fn print(&self) {
        println!("========Begin Params=========");
        println!("k: {}", self.k);
        println!("order: {}", big_to_hex(&self.order));
        println!("g1: {}", ecp_to_hex(&self.g1));
        println!("g2: {}", ecp_to_hex(&self.g2));
```

src-tauri/src/kr_ibe/params.rs (continued)

```
        for i in 0..self.At.len() {
            println!("At[{}]: {}", i, ecp_to_hex(&self.At[i]));
        }
        for i in 0..self.Bt.len() {
            println!("Bt[{}]: {}", i, ecp_to_hex(&self.Bt[i]));
        }
        for i in 0..self.Dt.len() {
            println!("Dt[{}]: {}", i, ecp_to_hex(&self.Dt[i]));
        }
        for i in 0..self.f1.get_degree() {
            println!("f1[{}]: {}", i, big_to_hex(&self.f1.get_coeff_at(i)));
        }
        for i in 0..self.f2.get_degree() {
            println!("f2[{}]: {}", i, big_to_hex(&self.f2.get_coeff_at(i)));
        }
        for i in 0..self.h1.get_degree() {
            println!("h1[{}]: {}", i, big_to_hex(&self.h1.get_coeff_at(i)));
        }
        for i in 0..self.h2.get_degree() {
            println!("h2[{}]: {}", i, big_to_hex(&self.h2.get_coeff_at(i)));
        }
        for i in 0..self.p1.get_degree() {
            println!("p1[{}]: {}", i, big_to_hex(&self.p1.get_coeff_at(i)));
        }
        for i in 0..self.p2.get_degree() {
            println!("p2[{}]: {}", i, big_to_hex(&self.p2.get_coeff_at(i)));
        }
        println!("========End of Params=========");
    }

    pub fn format_full(&self) -> String {
        let mut output = String::new();

        output.push_str("========Begin Params=========\n");
        output.push_str(&format!("k: {}\n", self.k));
        output.push_str(&format!("order: {}\n", big_to_hex(&self.order)));
        output.push_str(&format!("g1: {}\n", ecp_to_hex(&self.g1)));
        output.push_str(&format!("g2: {}\n", ecp_to_hex(&self.g2)));

        for i in 0..self.At.len() {
            output.push_str(&format!("At[{}]: {}\n", i, ecp_to_hex(&self.At[i])));
        }
        for i in 0..self.Bt.len() {
            output.push_str(&format!("Bt[{}]: {}\n", i, ecp_to_hex(&self.Bt[i])));
        }
        for i in 0..self.Dt.len() {
            output.push_str(&format!("Dt[{}]: {}\n", i, ecp_to_hex(&self.Dt[i])));
        }
        for i in 0..self.f1.get_degree() {
            output.push_str(&format!(
                "f1[{}]: {}\n",
                i,
                big_to_hex(&self.f1.get_coeff_at(i))
            ));
        }
        for i in 0..self.f2.get_degree() {
            output.push_str(&format!(
                "f2[{}]: {}\n",
                i,
                big_to_hex(&self.f2.get_coeff_at(i))
            ));
        }
        for i in 0..self.h1.get_degree() {
            output.push_str(&format!(
                "h1[{}]: {}\n",
                i,
                big_to_hex(&self.h1.get_coeff_at(i))
            ));
        }
        for i in 0..self.h2.get_degree() {
            output.push_str(&format!(
                "h2[{}]: {}\n",
                i,
                big_to_hex(&self.h2.get_coeff_at(i))
            ));
        }
        for i in 0..self.p1.get_degree() {
            output.push_str(&format!(
                "p1[{}]: {}\n",
                i,
                big_to_hex(&self.p1.get_coeff_at(i))
            ));
        }
        for i in 0..self.p2.get_degree() {
```

src-tauri/src/kr_ibe/params.rs (continued)

```
            output.push_str(&format!(
                "p2[{}]: {}\n",
                i,
                big_to_hex(&self.p2.get_coeff_at(i))
            ));
        }
        output.push_str("========End of Params=========\n");

        output
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}
```

src-tauri/src/kr_ibe/plaintext.rs

```
extern crate mcore;

pub struct Plaintext {
    plaintext: String,
}

impl Plaintext {
    pub fn new() -> Self {
        Plaintext {
            plaintext: String::new(),
        }
    }

    pub fn new_plaintext(plaintext: String) -> Self {
        Plaintext { plaintext }
    }

    pub fn set_plaintext(&mut self, plaintext: String) {
        self.plaintext = plaintext;
    }

    pub fn print(&self) {
        println!("========Begin Plaintext=========");
        println!("plaintext: {}", self.plaintext);
        println!("========End of Plaintext=========");
    }

    pub fn to_string(&self) -> String {
        self.plaintext.clone()
    }

    pub fn format_full(&self) -> String {
        format!("Plaintext: {}", self.plaintext)
    }
}
```

src-tauri/src/kr_ibe/polynomial.rs

```
extern crate mcore;
use mcore::ed25519::big;
use mcore::rand::RAND;
use std::fmt;

#[derive(Clone)]
/// Polynomial operations.
pub struct Polynomial {
    degree: usize,
    coeff: Vec<big::BIG>,
    order: big::BIG,
}

impl Polynomial {
    pub fn new() -> Self {
        Polynomial {
            degree: 0,
            coeff: Vec::new(),
            order: big::BIG::new(),
        }
    }

    pub fn Gennew(degree: usize, sk: &big::BIG, order: &big::BIG) -> Self {
        let mut rng = RAND::new();
        let mut rand = rand::thread_rng();
        let mut seed = [0u8; 100];
        //rand.fill(&mut seed);
        rng.clean();
        rng.seed(100, &seed);

        let mut coeff = vec![big::BIG::new(); degree];
        coeff[0] = sk.clone();

        for i in 1..degree {
            coeff[i] = big::BIG::randomnum(order, &mut rng);
        }

        Polynomial {
            degree,
            coeff,
            order: order.clone(),
        }
    }

    pub fn evaluate(&self, x: &big::BIG) -> big::BIG {
        let mut accum = big::BIG::new();
        for j in 0..self.degree {
            let exp = big::BIG::new_int(j as isize);
            let mut x_clone = x.clone();
            let t2 = big::BIG::modmul(
                &self.coeff[j],
                &x_clone.powmod(&exp, &self.order),
                &self.order,
            );
            accum.add(&t2);
            accum.rmod(&self.order);
        }
        accum
    }

    pub fn get_degree(&self) -> usize {
        self.degree
    }

    pub fn get_coeff(&self) -> &Vec<big::BIG> {
        &self.coeff
    }

    pub fn get_coeff_at(&self, i: usize) -> &big::BIG {
        &self.coeff[i]
    }

    pub fn get_order(&self) -> &big::BIG {
        &self.order
    }

    pub fn fmt(&self) -> String {
        let mut str = String::new();
        str.push_str("\n========Begin Polynomial=========\n");
        str.push_str(&format!("degree: {}\n", self.degree));
```

src-tauri/src/kr_ibe/polynomial.rs (continued)

```
        str.push_str(&format!("order: {}\n", big_to_hex(&self.order)));

        for (i, coeff) in self.coeff.iter().enumerate() {
            str.push_str(&format!("coeff[{}]: {}\n", i, big_to_hex(&coeff)));
        }
        str.push_str("========End of Polynomial========\n");
        return str;
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}
```

src-tauri/src/kr_ibe/private_key.rs

```
extern crate mcore;

use mcore::ed25519::big;

#[derive(Clone)]
pub struct PrivateKey {
    f1ID: big::BIG,
    f2ID: big::BIG,
    h1ID: big::BIG,
    h2ID: big::BIG,
    p1ID: big::BIG,
    p2ID: big::BIG,
}

impl PrivateKey {
    pub fn new() -> Self {
        PrivateKey {
            f1ID: big::BIG::new(),
            f2ID: big::BIG::new(),
            h1ID: big::BIG::new(),
            h2ID: big::BIG::new(),
            p1ID: big::BIG::new(),
            p2ID: big::BIG::new(),
        }
    }

    pub fn set_private_key(
        &mut self,
        f1ID: big::BIG,
        f2ID: big::BIG,
        h1ID: big::BIG,
        h2ID: big::BIG,
        p1ID: big::BIG,
        p2ID: big::BIG,
    ) {
        self.f1ID = f1ID;
        self.f2ID = f2ID;
        self.h1ID = h1ID;
        self.h2ID = h2ID;
        self.p1ID = p1ID;
        self.p2ID = p2ID;
    }

    pub fn get_f1ID(&self) -> &big::BIG {
        &self.f1ID
    }

    pub fn get_f2ID(&self) -> &big::BIG {
        &self.f2ID
    }

    pub fn get_h1ID(&self) -> &big::BIG {
        &self.h1ID
    }

    pub fn get_h2ID(&self) -> &big::BIG {
        &self.h2ID
    }

    pub fn get_p1ID(&self) -> &big::BIG {
        &self.p1ID
    }

    pub fn get_p2ID(&self) -> &big::BIG {
        &self.p2ID
    }

    pub fn print(&self) {
        println!("========Begin Private Key=========");
        println!("f1ID: {}", big_to_hex(&self.f1ID));
        println!("f2ID: {}", big_to_hex(&self.f2ID));
        println!("h1ID: {}", big_to_hex(&self.h1ID));
        println!("h2ID: {}", big_to_hex(&self.h2ID));
        println!("p1ID: {}", big_to_hex(&self.p1ID));
        println!("p2ID: {}", big_to_hex(&self.p2ID));
        println!("========End of Private Key=========");
    }

    pub fn format_full(&self) -> String {
        let mut output = String::new();
```

src-tauri/src/kr_ibe/private_key.rs (continued)

```
        output.push_str("========Begin Private Key=========\n");

        output.push_str(&format!("f1ID: {}\n", big_to_hex(&self.f1ID)));
        output.push_str(&format!("f2ID: {}\n", big_to_hex(&self.f2ID)));
        output.push_str(&format!("h1ID: {}\n", big_to_hex(&self.h1ID)));
        output.push_str(&format!("h2ID: {}\n", big_to_hex(&self.h2ID)));
        output.push_str(&format!("p1ID: {}\n", big_to_hex(&self.p1ID)));
        output.push_str(&format!("p2ID: {}\n", big_to_hex(&self.p2ID)));

        output.push_str("========End of Private Key=========\n");
        output
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}
```

src-tauri/src/kr_ibi/api.rs

```
use tauri::command;

use super::main as kribi_core;
use super::params::Params;
use once_cell::sync::Lazy;
use std::sync::Mutex;
use std::time::Instant;

static PARAMS: Lazy<Mutex<Option<Params>>> = Lazy::new(|| Mutex::new(None));
static TOTAL_RUNTIME: Lazy<Mutex<f64>> = Lazy::new(|| Mutex::new(0.0));

//   Add this function for smart unit formatting
fn format_runtime(seconds: f64) -> String {
    if seconds < 1.0 {
        format!("{:.0} ms", seconds * 1000.0)
    } else {
        format!("{:.2} s", seconds)
    }
}

#[command]
pub fn kr_ibi_setup(k: usize) -> String {
    let start_time = Instant::now();

    let mut params = Params::new();
    kribi_core::setup(&mut params, k);

    let duration = start_time.elapsed();
    *PARAMS.lock().unwrap() = Some(params);
    *TOTAL_RUNTIME.lock().unwrap() = 0.0; // Reset total runtime
    *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate setup time

    let mut output = String::new();
    output.push_str("  KR-IBI Setup Complete!\n\n");
    output.push_str(&PARAMS.lock().unwrap().as_ref().unwrap().format_full());
    output.push_str(&format!(
        "\n  Setup Time: {}\n",
        format_runtime(duration.as_secs_f64())
    ));

    output
}

#[command]
pub fn kr_ibi_extract(id: String) -> String {
    let start_time = Instant::now();

    let params_guard = PARAMS.lock().unwrap();
    if let Some(ref params) = *params_guard {
        let id_bytes = kribi_core::string_to_bytes(&id);

        let (f1, f2) = kribi_core::extract(params, &id_bytes);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate extract time

        format!(
            "  KR-IBI Private Key Extracted:\n\nf(ID1): {}\nf(ID2): {}\n\n  Extract Time: {}",
            kribi_core::big_to_hex(&f1),
            kribi_core::big_to_hex(&f2),
            format_runtime(duration.as_secs_f64())
        )
    } else {
        "  Error: KR-IBI setup not done yet!".into()
    }
}

#[command]
pub fn kr_ibi_sign(id: String) -> String {
    let start_time = Instant::now();

    let params_guard = PARAMS.lock().unwrap();
    if let Some(ref params) = *params_guard {
        let id_bytes = kribi_core::string_to_bytes(&id);

        let (fID1, fID2) = kribi_core::extract(params, &id_bytes);

        let mut rng = kribi_core::gen_seed();
        let (g_r, r) = kribi_core::commit(params, &mut rng);
        let (c1, c2) = kribi_core::challenge(params, &mut rng);
```

src-tauri/src/kr_ibi/api.rs (continued)

```
        let (s1, s2) = kribi_core::respond(&r, &(c1, c2), &(fID1, fID2), params.get_order());

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate sign time

        format!(
            "  KR-IBI Signature Generated:\n\nCommit: ({}, {})\nChallenge: ({}, {})\nResponse: ({}, {})\n\n  Sign Time: {}",
            kribi_core::ecp_to_hex(&g_r.0),
            kribi_core::ecp_to_hex(&g_r.1),
            kribi_core::big_to_hex(&c1),
            kribi_core::big_to_hex(&c2),
            kribi_core::big_to_hex(&s1),
            kribi_core::big_to_hex(&s2),
            format_runtime(duration.as_secs_f64())
        )
    } else {
        "  Error: KR-IBI setup not done yet!".into()
    }
}

#[command]
pub fn kr_ibi_verify(id: String) -> String {
    let start_time = Instant::now();

    let params_guard = PARAMS.lock().unwrap();
    if let Some(ref params) = *params_guard {
        let id_bytes = kribi_core::string_to_bytes(&id);

        let (fID1, fID2) = kribi_core::extract(params, &id_bytes);

        let mut rng = kribi_core::gen_seed();
        let (g_r, r) = kribi_core::commit(params, &mut rng);
        let (c1, c2) = kribi_core::challenge(params, &mut rng);

        let (s1, s2) = kribi_core::respond(&r, &(c1, c2), &(fID1, fID2), params.get_order());

        let valid = kribi_core::verify(params, &g_r, &(s1, s2), &(c1, c2), &id_bytes);

        let verify_duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += verify_duration.as_secs_f64(); // accumulate verify time

        let total_runtime = *TOTAL_RUNTIME.lock().unwrap(); // get full total runtime

        if valid {
            format!(
                "  KR-IBI Verification Successful!\n\n  Verify Time: {}\n  Total Computation Time: {}",
                format_runtime(verify_duration.as_secs_f64()),
                format_runtime(total_runtime)
            )
        } else {
            format!(
                "  KR-IBI Verification Failed!\n\n  Verify Time: {}\n  Total Computation Time: {}",
                format_runtime(verify_duration.as_secs_f64()),
                format_runtime(total_runtime)
            )
        }
    } else {
        "  Error: KR-IBI setup not done yet!".into()
    }
}
```

src-tauri/src/kr_ibi/main.rs

```
#![allow(non_snake_case)]

use super::params::Params;
use std::time::Instant;

extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;
use mcore::ed25519::rom;
use mcore::rand::RAND;
use mcore::sha3::{SHA3, SHAKE256};
use rand::rngs::OsRng;
use rand::RngCore;

use mcore::ed25519::big::BIG;
use mcore::ed25519::ecp::ECP;

pub fn setup(params: &mut Params, k: usize) {
    let start = Instant::now();

    let order = big::BIG::new_ints(&rom::CURVE_ORDER);

    let g = ecp::ECP::generator();

    let mut rng = gen_seed();
    let t = k + 1;

    let mut fx1 = vec![big::BIG::new(); t];
    let mut fx2 = vec![big::BIG::new(); t];

    for i in 0..t {
        fx1[i] = big::BIG::randomnum(&order, &mut rng);
        fx2[i] = big::BIG::randomnum(&order, &mut rng);
    }

    let mut Dt1 = vec![ecp::ECP::new(); t];
    let mut Dt2 = vec![ecp::ECP::new(); t];

    for i in 0..t {
        Dt1[i] = g.mul(&fx1[i]);
        Dt2[i] = g.mul(&fx2[i]);
    }

    params.set_params(k, order, g, Dt1, Dt2, fx1, fx2);
    println!("Setup time: {:?}", start.elapsed());
}

pub fn extract(params: &Params, id: &[u8]) -> (big::BIG, big::BIG) {
    let start = Instant::now();
    let k = params.get_k() + 1;
    let order = params.get_order();
    let d1 = params.get_msk1();
    let d2 = params.get_msk2();

    let mut x = hash_to_big(id);

    let mut f1 = big::BIG::new_int(0);
    let mut f2 = big::BIG::new_int(0);

    for i in 0..k {
        let i_isize: isize = i.try_into().unwrap();
        let xpowi = big::BIG::powmod(&mut x, &big::BIG::new_int(i_isize), &order);

        let temp1 = big::BIG::modmul(&d1[i], &xpowi, &order);
        f1.add(&temp1);
        f1.rmod(&order);

        let temp2 = big::BIG::modmul(&d2[i], &xpowi, &order);
        f2.add(&temp2);
        f2.rmod(&order);
    }

    println!("Extract time: {:?}", start.elapsed());
    (f1, f2)
}

// Prover Commitment
pub fn commit(params: &Params, rng: &mut RAND) -> ((ecp::ECP, ecp::ECP), (big::BIG, big::BIG)) {
    let r1 = big::BIG::randomnum(params.get_order(), rng);
```

src-tauri/src/kr_ibi/main.rs (continued)

```
    let r2 = big::BIG::randomnum(params.get_order(), rng);

    let g_r1 = params.get_g().mul(&r1);
    let g_r2 = params.get_g().mul(&r2);

    ((g_r1, g_r2), (r1, r2))
}

// Verifier's Challenge
pub fn challenge(params: &Params, rng: &mut RAND) -> (big::BIG, big::BIG) {
    let c1 = big::BIG::randomnum(params.get_order(), rng);
    let c2 = big::BIG::randomnum(params.get_order(), rng);

    (c1, c2)
}

// Prover's Response
pub fn respond(
    r: &(big::BIG, big::BIG),
    challenge: &(big::BIG, big::BIG),
    f_id: &(big::BIG, big::BIG),
    order: &big::BIG,
) -> (big::BIG, big::BIG) {
    let mut response_1 = big::BIG::modmul(&f_id.0, &challenge.0, order);
    response_1 = big::BIG::modadd(&response_1, &r.0, order);

    let mut response_2 = big::BIG::modmul(&f_id.1, &challenge.1, order);
    response_2 = big::BIG::modadd(&response_2, &r.1, order);

    (response_1, response_2)
}

pub fn verify(
    params: &Params,
    g_r: &(ecp::ECP, ecp::ECP),
    response: &(big::BIG, big::BIG),
    challenge: &(big::BIG, big::BIG),
    id: &[u8],
) -> bool {
    let start = Instant::now();
    let mut x = hash_to_big(id);

    let f_id_point1 = {
        let mut sum = ecp::ECP::new();
        for i in 0..params.get_k() + 1 {
            let i_isize = big::BIG::new_int(i as isize);
            let xpowi = big::BIG::powmod(&mut x, &i_isize, params.get_order());
            let temp = params.get_Dt1()[i].mul(&xpowi);
            sum.add(&temp);
        }
        sum
    };

    let f_id_point2 = {
        let mut sum = ecp::ECP::new();
        for i in 0..params.get_k() + 1 {
            let i_isize = big::BIG::new_int(i as isize);
            let xpowi = big::BIG::powmod(&mut x, &i_isize, params.get_order());
            let temp = params.get_Dt2()[i].mul(&xpowi);
            sum.add(&temp);
        }
        sum
    };

    for i in 0..params.get_k() + 1 {
        let public_key_1: &ecp::ECP = &params.get_Dt1()[i];
        let public_key_2: &ecp::ECP = &params.get_Dt2()[i];

        if public_key_1.is_infinity() || public_key_2.is_infinity() {
            println!("Invalid public key at index {}", i);
            return false;
        }
    }

    let mut g_r_f_id_c1 = f_id_point1.mul(&challenge.0);
    g_r_f_id_c1.add(&g_r.0);

    let mut g_r_f_id_c2 = f_id_point2.mul(&challenge.1);
    g_r_f_id_c2.add(&g_r.1);

    let valid_1 = g_r_f_id_c1.equals(&ecp::ECP::generator().mul(&response.0));
    let valid_2 = g_r_f_id_c2.equals(&ecp::ECP::generator().mul(&response.1));

    if !valid_1 || !valid_2 {
        println!("Commitment or response verification failed.");
```

src-tauri/src/kr_ibi/main.rs (continued)

```
    }
    let duration = start.elapsed(); // Calculate the elapsed time
    println!("Verification time: {:?}", duration); // Output the timing information
    valid_1 && valid_2
}

fn main() {
    let start_total = Instant::now();
    let id: &str = "aniksen360@mail.com";
    let k = 20;
    let id = string_to_bytes(id);

    let mut params: Params = Params::new();

    setup(&mut params, k);
    params.print();

    let (fID1, fID2): (BIG, BIG) = extract(&params, &id);
    println!("f(ID1): {}", big_to_hex(&fID1));
    println!("f(ID2): {}", big_to_hex(&fID2));
    //println!("Total operation time: {:?}", start_total.elapsed());

    let mut rng = gen_seed();

    // Prover's commitment
    let (g_r, r) = commit(&params, &mut rng);
    println!(
        "Prover Commitment (g^r): {:?}",
        (ecp_to_hex(&g_r.0), ecp_to_hex(&g_r.1))
    );

    // Verifier's challenges
    let (c1, c2) = challenge(&params, &mut rng);
    println!(
        "Verifier Challenges: {:?}",
        (big_to_hex(&c1), big_to_hex(&c2))
    );

    // Prover's responses
    let (s1, s2) = respond(&r, &(c1, c2), &(fID1, fID2), params.get_order());
    println!("Prover Responses: {:?}", (big_to_hex(&s1), big_to_hex(&s2)));

    // Verifier's check
    let is_valid = verify(&params, &g_r, &(s1, s2), &(c1, c2), &id);
    println!("Verification result: {}", is_valid);
}

pub fn gen_seed() -> RAND {
    let mut raw: [u8; 100] = [0; 100];
    let mut rng = RAND::new();
    rng.clean();
    OsRng.fill_bytes(&mut raw);
    rng.seed(100, &raw);
    rng
}

pub fn hash_to_big(data: &[u8]) -> BIG {
    let mut sha = SHA3::new(SHAKE256);
    for &byte in data {
        sha.process(byte);
    }
    let mut output = [0u8; big::MODBYTES];
    sha.shake(&mut output, big::MODBYTES);
    BIG::frombytes(&output)
}

pub fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

pub fn hex_to_big(hex: &str) -> big::BIG {
    let mut bytes = Vec::new();
    for i in (0..hex.len()).step_by(2) {
        let byte_str = &hex[i..i + 2];
        let byte = u8::from_str_radix(byte_str, 16).unwrap_or(0);
        bytes.push(byte);
    }
    while bytes.len() < big::MODBYTES {
        bytes.insert(0, 0);
    }
    let mut byte_array = [0u8; big::MODBYTES];
    byte_array.copy_from_slice(&bytes[..big::MODBYTES]);
    big::BIG::frombytes(&byte_array)
```

src-tauri/src/kr_ibi/main.rs (continued)

```
}

pub fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}
pub fn string_to_bytes(input: &str) -> Vec<u8> {
    input.as_bytes().to_vec()
}
```

src-tauri/src/kr_ibi/mod.rs

```
pub mod api;
pub mod main;
pub mod params;
```

src-tauri/src/kr_ibi/params.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;

pub struct Params {
    k: usize,
    order: big::BIG,
    g: ecp::ECP,
    Dt1: Vec<ecp::ECP>,
    Dt2: Vec<ecp::ECP>,
    msk1: Vec<big::BIG>,
    msk2: Vec<big::BIG>,
}

impl Params {
    pub fn new() -> Self {
        Params {
            k: 0,
            order: big::BIG::new(),
            g: ecp::ECP::new(),
            Dt1: Vec::new(),
            Dt2: Vec::new(),
            msk1: Vec::new(),
            msk2: Vec::new(),
        }
    }

    pub fn set_params(
        &mut self,
        k: usize,
        order: big::BIG,
        g: ecp::ECP,
        Dt1: Vec<ecp::ECP>,
        Dt2: Vec<ecp::ECP>,
        msk1: Vec<big::BIG>,
        msk2: Vec<big::BIG>,
    ) {
        self.k = k;
        self.order = order;
        self.g = g;
        self.Dt1 = Dt1;
        self.Dt2 = Dt2;
        self.msk1 = msk1;
        self.msk2 = msk2;
    }

    pub fn get_k(&self) -> usize {
        self.k
    }

    pub fn get_order(&self) -> &big::BIG {
        &self.order
    }

    pub fn get_g(&self) -> &ecp::ECP {
        &self.g
    }

    pub fn get_msk1(&self) -> &Vec<big::BIG> {
        &self.msk1
    }

    pub fn get_msk2(&self) -> &Vec<big::BIG> {
        &self.msk2
    }

    pub fn get_Dt1(&self) -> &Vec<ecp::ECP> {
        &self.Dt1
    }

    pub fn get_Dt2(&self) -> &Vec<ecp::ECP> {
        &self.Dt2
    }

    pub fn print(&self) {
        println!("k: {}", self.k);
        println!("Order: {}", big_to_hex(&self.order));
        println!("g: {}", ecp_to_hex(&self.g));
        for i in 0..self.Dt1.len() {
```

src-tauri/src/kr_ibi/params.rs (continued)

```
            println!("Dt1[{}]: {}", i, ecp_to_hex(&self.Dt1[i]));
            println!("Dt2[{}]: {}", i, ecp_to_hex(&self.Dt2[i]));
        }
        for i in 0..self.msk1.len() {
            println!("msk1[{}]: {}", i, big_to_hex(&self.msk1[i]));
            println!("msk2[{}]: {}", i, big_to_hex(&self.msk2[i]));
        }
    }

    pub fn format_full(&self) -> String {
        let mut output = String::new();
        output.push_str("========Begin Params=========\n");
        output.push_str(&format!("k: {}\n", self.k));
        output.push_str(&format!("order: {}\n", big_to_hex(&self.order)));
        output.push_str(&format!("g: {}\n", ecp_to_hex(&self.g)));
        for i in 0..self.Dt1.len() {
            output.push_str(&format!("Dt1[{}]: {}\n", i, ecp_to_hex(&self.Dt1[i])));
            output.push_str(&format!("Dt2[{}]: {}\n", i, ecp_to_hex(&self.Dt2[i])));
        }
        for i in 0..self.msk1.len() {
            output.push_str(&format!("msk1[{}]: {}\n", i, big_to_hex(&self.msk1[i])));
            output.push_str(&format!("msk2[{}]: {}\n", i, big_to_hex(&self.msk2[i])));
        }
        output.push_str("========End Params=========");
        output
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}
```

src-tauri/src/kr_paeks/api.rs

```
use tauri::command;

use super::main as krpaeks_core;
use crate::kr_paeks::ciphertext::Ciphertext;
use crate::kr_paeks::params::Params;
use crate::kr_paeks::private_key::PrivateKey;
use crate::kr_paeks::public_key::PublicKey;
use crate::kr_paeks::trapdoor::Trapdoor;

use once_cell::sync::Lazy;
use std::sync::Mutex;
use std::time::Instant;

static PARAMS: Lazy<Mutex<Option<Params>>> = Lazy::new(|| Mutex::new(None));
static SENDER_SK: Lazy<Mutex<Option<PrivateKey>>> = Lazy::new(|| Mutex::new(None));
static SENDER_PK: Lazy<Mutex<Option<PublicKey>>> = Lazy::new(|| Mutex::new(None));
static RECEIVER_SK: Lazy<Mutex<Option<PrivateKey>>> = Lazy::new(|| Mutex::new(None));
static RECEIVER_PK: Lazy<Mutex<Option<PublicKey>>> = Lazy::new(|| Mutex::new(None));
static CIPHERTEXT: Lazy<Mutex<Option<Ciphertext>>> = Lazy::new(|| Mutex::new(None));
static TRAPDOOR: Lazy<Mutex<Option<Trapdoor>>> = Lazy::new(|| Mutex::new(None));
static TOTAL_RUNTIME: Lazy<Mutex<f64>> = Lazy::new(|| Mutex::new(0.0)); // total computation time

#[command]
pub fn kr_paeks_setup(k: usize) -> String {
    let start_time = Instant::now();

    let mut params = Params::new();
    krpaeks_core::setup(&mut params, k);

    let duration = start_time.elapsed();
    *PARAMS.lock().unwrap() = Some(params);
    *TOTAL_RUNTIME.lock().unwrap() = 0.0; // Reset runtime
    *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate setup

    let mut output = String::new();
    output.push_str("  KR-PAEKS Setup Complete!\n\n");
    output.push_str(&PARAMS.lock().unwrap().as_ref().unwrap().format_full());
    output.push_str(&format!("\n  Setup Time: {:.2?}\n", duration));

    output
}

#[command]
pub fn kr_paeks_keygen() -> String {
    let start_time = Instant::now();

    let params_lock = PARAMS.lock().unwrap();
    if let Some(ref params) = *params_lock {
        let mut sender_pk = PublicKey::new();
        let mut sender_sk = PrivateKey::new();
        let mut receiver_pk = PublicKey::new();
        let mut receiver_sk = PrivateKey::new();

        krpaeks_core::keygen(params, &mut sender_pk, &mut sender_sk);
        krpaeks_core::keygen(params, &mut receiver_pk, &mut receiver_sk);

        *SENDER_PK.lock().unwrap() = Some(sender_pk);
        *SENDER_SK.lock().unwrap() = Some(sender_sk);
        *RECEIVER_PK.lock().unwrap() = Some(receiver_pk);
        *RECEIVER_SK.lock().unwrap() = Some(receiver_sk);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate keygen

        let mut output = String::new();
        output.push_str("  KR-PAEKS Keygen Complete!\n\n");
        output.push_str("  Sender Private Key:\n");
        output.push_str(&SENDER_SK.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str("\n\n  Sender Public Key:\n");
        output.push_str(&SENDER_PK.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str("\n\n  Receiver Private Key:\n");
        output.push_str(&RECEIVER_SK.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str("\n\n  Receiver Public Key:\n");
        output.push_str(&RECEIVER_PK.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str(&format!(
            "\n  Keygen Time: {}\n",
            format_runtime(duration.as_secs_f64())
        ));

        output
```

src-tauri/src/kr_paeks/api.rs (continued)

```
    } else {
        "  Error: KR-PAEKS setup not done yet!".into()
    }
}

#[command]
pub fn kr_paeks_encrypt(keyword: String) -> String {
    let start_time = Instant::now();

    let params_lock = PARAMS.lock().unwrap();
    let receiver_pk_lock = RECEIVER_PK.lock().unwrap();
    let sender_sk_lock = SENDER_SK.lock().unwrap();

    if let (Some(ref params), Some(ref receiver_pk), Some(ref sender_sk)) = (
        params_lock.as_ref(),
        receiver_pk_lock.as_ref(),
        sender_sk_lock.as_ref(),
    ) {
        let keyword_big = krpaeks_core::hash_to_big(&keyword);
        let ct = krpaeks_core::encrypt(params, receiver_pk, sender_sk, &keyword_big);

        *CIPHERTEXT.lock().unwrap() = Some(ct);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate encryption

        let mut output = String::new();
        output.push_str("  KR-PAEKS Encryption Complete!\n\n");
        output.push_str("  Ciphertext:\n");
        output.push_str(&CIPHERTEXT.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str(&format!(
            "\n  Encryption Time: {}\n",
            format_runtime(duration.as_secs_f64())
        ));

        output
    } else {
        "  Error: Keygen not done yet!".into()
    }
}

#[command]
pub fn kr_paeks_trapdoor(keyword: String) -> String {
    let start_time = Instant::now();

    let params_lock = PARAMS.lock().unwrap();
    let sender_pk_lock = SENDER_PK.lock().unwrap();
    let receiver_sk_lock = RECEIVER_SK.lock().unwrap();

    if let (Some(ref params), Some(ref sender_pk), Some(ref receiver_sk)) = (
        params_lock.as_ref(),
        sender_pk_lock.as_ref(),
        receiver_sk_lock.as_ref(),
    ) {
        let keyword_big = krpaeks_core::hash_to_big(&keyword);
        let td = krpaeks_core::trapdoor(params, sender_pk, receiver_sk, &keyword_big);

        *TRAPDOOR.lock().unwrap() = Some(td);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate trapdoor

        let mut output = String::new();
        output.push_str("  KR-PAEKS Trapdoor Generation Complete!\n\n");
        output.push_str("  Trapdoor:\n");
        output.push_str(&TRAPDOOR.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str(&format!(
            "\n  Trapdoor Time: {}\n",
            format_runtime(duration.as_secs_f64())
        ));

        output
    } else {
        "  Error: Keygen not done yet!".into()
    }
}

#[command]
pub fn kr_paeks_test() -> String {
    let start_time = Instant::now();

    let ct_lock = CIPHERTEXT.lock().unwrap();
    let td_lock = TRAPDOOR.lock().unwrap();

    if let (Some(ref ct), Some(ref td)) = (ct_lock.as_ref(), td_lock.as_ref()) {
```

src-tauri/src/kr_paeks/api.rs (continued)

```
        let result = krpaeks_core::test(ct, td);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate test

        let total_runtime = *TOTAL_RUNTIME.lock().unwrap();

        let mut output = String::new();
        if result {
            output.push_str("  KR-PAEKS Test Successful!\n\n");
        } else {
            output.push_str("  KR-PAEKS Test Failed!\n\n");
        }

        output.push_str(&format!("  Test Time: {:.2?}\n", duration));
        output.push_str(&format!(
            "  Total Computation Time: {}\n",
            format_runtime(total_runtime)
        ));

        output
    } else {
        "  Error: Need to run encryption and trapdoor first!".into()
    }
}

fn format_runtime(seconds: f64) -> String {
    if seconds < 1.0 {
        // Less than 1 second → show milliseconds
        format!("{:.0} ms", seconds * 1000.0)
    } else {
        // 1 second or more → show seconds
        format!("{:.2} s", seconds)
    }
}
```

src-tauri/src/kr_paeks/ciphertext.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;
use mysql::serde;
use serde::Deserialize;
use serde::Serialize;

// #[derive(Serialize, Deserialize)]
#[derive(Clone)]
pub struct Ciphertext {
    pub c1: ecp::ECP,
    pub c2: ecp::ECP,
    pub u: big::BIG,
}

impl Ciphertext {
    // Constructor (equivalent to Java's constructors)
    pub fn new() -> Self {
        Ciphertext {
            c1: ecp::ECP::new(),
            c2: ecp::ECP::new(),
            u: big::BIG::new(),
        }
    }

    pub fn new_ciphertext(c1: &ecp::ECP, c2: &ecp::ECP, u: &big::BIG) -> Self {
        Ciphertext {
            c1: c1.clone(),
            c2: c2.clone(),
            u: u.clone(),
        }
    }

    // Setters (not strictly necessary in Rust, but provided for completeness)
    pub fn set_ciphertext(&mut self, c1: ecp::ECP, c2: ecp::ECP, u: big::BIG) {
        self.c1 = c1;
        self.c2 = c2;
        self.u = u;
    }

    pub fn set_c1(&mut self, c1: ecp::ECP) {
        self.c1 = c1;
    }

    pub fn set_c2(&mut self, c2: ecp::ECP) {
        self.c2 = c2;
    }

    pub fn set_u(&mut self, u: big::BIG) {
        self.u = u;
    }

    pub fn print(&self) {
        println!("========Begin Ciphertext=========");
        println!("C1: {}", ecp_to_hex(&self.c1));
        println!("C2: {}", ecp_to_hex(&self.c2));
        println!("U: {}", big_to_hex(&self.u));
        println!("========End of Ciphertext=========");
    }
    pub fn format_full(&self) -> String {
        format!(
            "C1: {}\nC2: {}\nU: {}",
            ecp_to_hex(&self.c1),
            ecp_to_hex(&self.c2),
            big_to_hex(&self.u)
        )
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
```

src-tauri/src/kr_paeks/ciphertext.rs (continued)

```
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}

fn bytes_to_hex(bytes: &[u8]) -> String {
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}
```

src-tauri/src/kr_paeks/lagrange.rs

```
extern crate mcore;

use mcore::ed25519::big;
use std::vec::Vec;

pub struct Lagrange;

impl Lagrange {
    /// Computes the Lagrange interpolating polynomial at x = 0
    /// using the given x_values, y_values, and modulo order.
    pub fn interpolate_x(
        x_values: &Vec<big::BIG>,
        y_values: &Vec<big::BIG>,
        order: &big::BIG,
    ) -> big::BIG {
        let k = x_values.len();
        assert_eq!(k, y_values.len(), "Mismatched input lengths!");

        let mut result = big::BIG::new();

        for i in 0..k {
            let mut numerator = big::BIG::new_int(1);
            let mut denominator = big::BIG::new_int(1);

            for j in 0..k {
                if i != j {
                    let mut xj = x_values[j].clone();
                    let mut xi = x_values[i].clone();
                    let mut diff = big::BIG::modadd(&xj, &big::BIG::modneg(&xi, order), order); // (x_j - x_i) mod q
                    diff.invmodp(order); // Compute modular inverse

                    numerator = big::BIG::modmul(&numerator, &xj, order); // Multiply x_j terms
                    denominator = big::BIG::modmul(&denominator, &diff, order); // Multiply (x_j - x_i)^(-1)
                }
            }

            let mut term = big::BIG::modmul(&y_values[i], &numerator, order);
            term = big::BIG::modmul(&term, &denominator, order);
            result = big::BIG::modadd(&result, &term, order); // Accumulate result
        }

        result
    }
}
```

src-tauri/src/kr_paeks/main.rs

```
#![allow(non_snake_case)]

use super::ciphertext::Ciphertext;
use super::lagrange::Lagrange;
use super::params::Params;
use super::polynomial;
use super::private_key::PrivateKey;
use super::public_key::PublicKey;
use super::trapdoor::Trapdoor;

extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecdh;
use mcore::ed25519::ecp;
use mcore::rand::RAND;
use polynomial::Polynomial;
use rand::RngCore;
use std::time::Instant;

use log::error;
use mysql::prelude::*;
use mysql::*;
use rayon::prelude::*;
use serde_cbor;
use serde_json::json;
use std::error::Error;
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::{Arc, Mutex};

/// Generates a random seed for RNG
pub fn gen_seed() -> RAND {
    let mut rng = RAND::new();
    let mut seed = [0u8; 100];
    rand::thread_rng().fill_bytes(&mut seed);
    rng.clean();
    rng.seed(100, &seed);
    rng
}

/// KR-PAEKS Setup: Generates system parameters and the master secret key.
pub fn setup(params: &mut Params, k: usize) {
    let mut rng = gen_seed();
    let order = big::BIG::new_ints(&mcore::ed25519::rom::CURVE_ORDER);
    let msk = big::BIG::randomnum(&order, &mut rng);
    let g1 = ecp::ECP::generator();
    let g2 = g1.mul(&msk);

    //validate g1 and g2
    if is_valid(&g1) != 0 {
        println!("g1 is invalid! Abort!");
        std::process::abort();
    }

    if is_valid(&g2) != 0 {
        println!("g2 is invalid! Abort!");
        std::process::abort();
    }

    //Set the params
    params.set_params(g1, g2, msk, order, k);
}

/// Key Generation: Generates a public/private key pair.
pub fn keygen(params: &Params, pk: &mut PublicKey, sk: &mut PrivateKey) {
    let k = params.get_k();
    let order = params.get_order();
    let g1 = params.get_g1();
    let g2 = params.get_g2();

    let p1 = Polynomial::new_polynomial(k, *order);
    let p2 = Polynomial::new_polynomial(k, *order);

    let mut Dt = vec![ecp::ECP::new(); k];

    for i in 0..k {
        Dt[i] = g1.mul(&p1.get_coeff_at(i));
        Dt[i].add(&g2.mul(&p2.get_coeff_at(i)));
    }
```

src-tauri/src/kr_paeks/main.rs (continued)

```
    sk.set_private_key(p1, p2);
    pk.set_public_key(Dt);
}

/// Encrypts a keyword under the given public key.
pub fn encrypt(params: &Params, pk: &PublicKey, sk: &PrivateKey, keyword: &big::BIG) -> Ciphertext {
    let order = &params.order;
    let k = params.k;
    let mut rng = gen_seed();

    let mut hash_kw = keyword.clone();

    // Step 2: Generate random `r`
    let r = big::BIG::randomnum(order, &mut rng);

    // Step 3: Compute `temp = sk.p1.evaluate(hash_kw)`
    let temp = sk.p1.evaluate(&hash_kw);
    let newtemp = big::BIG::modmul(&temp, &r, order);

    // Step 4: Compute `C1`
    let mut c1 = pk.dt[0].mul(&hash_kw.powmod(&big::BIG::new_int(0), order));
    for i in 1..k {
        let exp = hash_kw.powmod(&big::BIG::new_int(i as isize), order);
        c1.add(&pk.dt[i].mul(&exp));
    }
    let tc1 = c1.mul(&newtemp); // Final `C1`

    // Step 5: Compute `temp2 = sk.p2.evaluate(hash_kw)`
    let temp2 = sk.p2.evaluate(&hash_kw);
    let newtemp2 = big::BIG::modmul(&temp2, &r, order);

    // Step 6: Compute `C2`
    let mut c2 = pk.dt[0].mul(&hash_kw.powmod(&big::BIG::new_int(0), order));
    for i in 1..k {
        let exp = hash_kw.powmod(&big::BIG::new_int(i as isize), order);
        c2.add(&pk.dt[i].mul(&exp));
    }
    let tc2 = c2.mul(&newtemp2); // Final `C2`

    // Step 7: Compute `u = p2(w) / p1(w) mod q`
    let mut u = sk.p2.evaluate(&hash_kw);
    let mut inv = sk.p1.evaluate(&hash_kw);
    inv.invmodp(order); // Compute modular inverse
    let new_u = big::BIG::modmul(&u, &inv, order);

    // Return the ciphertext
    Ciphertext::new_ciphertext(&tc1, &tc2, &new_u)
}

pub fn encrypt_multi_keyword(
    params: &Params,
    pk: &PublicKey,
    sk: &PrivateKey,
    keywords: &[String],
) -> Ciphertext {
    let order = &params.order;
    let k = params.k;
    let mut rng = gen_seed();

    // Generate a random r
    let r = big::BIG::randomnum(order, &mut rng);

    // Hash the keywords to a single BIG value
    let mut hash_kw = hash_to_big_array(keywords);

    // Compute temp = sk.p1.evaluate(hash_kw)
    let temp = sk.p1.evaluate(&hash_kw);
    let newtemp = big::BIG::modmul(&temp, &r, order);

    // Compute C1 using multi-keywords
    let mut c1 = pk.dt[0].mul(&hash_kw.powmod(&big::BIG::new_int(0), order));
    for i in 1..k {
        let exp = hash_kw.powmod(&big::BIG::new_int(i as isize), order);
        c1.add(&pk.dt[i].mul(&exp));
    }
    let tc1 = c1.mul(&newtemp); // Final C1

    // Compute temp2 = sk.p2.evaluate(hash_kw)
    let temp2 = sk.p2.evaluate(&hash_kw);
    let newtemp2 = big::BIG::modmul(&temp2, &r, order);

    // Compute C2 using multi-keywords
    let mut c2 = pk.dt[0].mul(&hash_kw.powmod(&big::BIG::new_int(0), order));
    for i in 1..k {
        let exp = hash_kw.powmod(&big::BIG::new_int(i as isize), order);
```

src-tauri/src/kr_paeks/main.rs (continued)

```
        c2.add(&pk.dt[i].mul(&exp));
    }
    let tc2 = c2.mul(&newtemp2); // Final C2

    // Compute u = p2(w) / p1(w) mod q
    let mut u = sk.p2.evaluate(&hash_kw);
    let mut inv = sk.p1.evaluate(&hash_kw);
    inv.invmodp(order); // Compute modular inverse
    let new_u = big::BIG::modmul(&u, &inv, order);

    Ciphertext::new_ciphertext(&tc1, &tc2, &new_u)
}

/// Generates a trapdoor for keyword search.
pub fn trapdoor(params: &Params, pk: &PublicKey, sk: &PrivateKey, keyword: &big::BIG) -> Trapdoor {
    let order = &params.order;
    let k = params.k;
    let mut rng = gen_seed();

    // Step 1: Compute `r`
    let r = big::BIG::randomnum(order, &mut rng);

    // Step 2: Compute `temp = sk.p1.evaluate(keyword)`
    let temp = sk.p1.evaluate(keyword);
    let newtemp = big::BIG::modmul(&temp, &r, order);

    // Step 3: Compute `T1`
    let mut t1 = ecp::ECP::new();
    for i in 0..k {
        let mut keyword_clone = keyword.clone(); // Clone keyword to allow mutation
        let exp = keyword_clone.powmod(&big::BIG::new_int(i as isize), order);
        let term = pk.dt[i].mul(&exp);
        t1.add(&term);
    }
    let tt1 = t1.mul(&newtemp); // TT1 = T1 * newtemp

    // Step 4: Compute `temp2 = sk.p2.evaluate(keyword)`
    let temp2 = sk.p2.evaluate(keyword);
    let newtemp2 = big::BIG::modmul(&temp2, &r, order);

    // Step 5: Compute `T2`
    let mut t2 = ecp::ECP::new();
    for i in 0..k {
        let mut keyword_clone = keyword.clone(); // Clone keyword to allow mutation
        let exp = keyword_clone.powmod(&big::BIG::new_int(i as isize), order);
        let term = pk.dt[i].mul(&exp);
        t2.add(&term);
    }
    let tt2 = t2.mul(&newtemp2); // TT2 = T2 * newtemp2

    // Step 6: Compute `u_cap`
    let mut u_cap = sk.p2.evaluate(keyword);
    let mut inv = sk.p1.evaluate(keyword).clone(); // Clone before inverting
    inv.invmodp(order); // Now modify `inv` safely
    let new_u_cap = big::BIG::modmul(&u_cap, &inv, order);

    Trapdoor::new_trapdoor(&tt1, &tt2, &new_u_cap)
}

pub fn trapdoor_multi_keyword(
    params: &Params,
    pk: &PublicKey,
    sk: &PrivateKey,
    keywords: &[String],
) -> Trapdoor {
    let order = &params.order;
    let k = params.k;
    let mut rng = gen_seed();

    // Generate random r
    let r = big::BIG::randomnum(order, &mut rng);

    // Hash the keywords to a single BIG value
    let mut hash_kw = hash_to_big_array(keywords);

    // Compute temp = sk.p1.evaluate(hash_kw)
    let temp = sk.p1.evaluate(&hash_kw);
    let newtemp = big::BIG::modmul(&temp, &r, order);

    // Compute T1 using multi-keywords
    let mut t1 = ecp::ECP::new();
    for i in 0..k {
        let mut keyword_clone = hash_kw.clone();
        let exp = keyword_clone.powmod(&big::BIG::new_int(i as isize), order);
        let term = pk.dt[i].mul(&exp);
```

src-tauri/src/kr_paeks/main.rs (continued)

```
        t1.add(&term);
    }
    let tt1 = t1.mul(&newtemp); // TT1 = T1 * newtemp

    // Compute temp2 = sk.p2.evaluate(hash_kw)
    let temp2 = sk.p2.evaluate(&hash_kw);
    let newtemp2 = big::BIG::modmul(&temp2, &r, order);

    // Compute T2 using multi-keywords
    let mut t2 = ecp::ECP::new();
    for i in 0..k {
        let mut keyword_clone = hash_kw.clone();
        let exp = keyword_clone.powmod(&big::BIG::new_int(i as isize), order);
        let term = pk.dt[i].mul(&exp);
        t2.add(&term);
    }
    let tt2 = t2.mul(&newtemp2); // TT2 = T2 * newtemp2

    // Compute u_cap
    let mut u_cap = sk.p2.evaluate(&hash_kw);
    let mut inv = sk.p1.evaluate(&hash_kw).clone(); // Clone before inverting
    inv.invmodp(order); // Now modify inv safely
    let new_u_cap = big::BIG::modmul(&u_cap, &inv, order);

    Trapdoor::new_trapdoor(&tt1, &tt2, &new_u_cap)
}

/// Tests if a ciphertext contains the keyword using the trapdoor.

pub fn test(ciphertext: &Ciphertext, trapdoor: &Trapdoor) -> bool {
    // Compute LHS: u * C1 + T2
    let mut lhs = ciphertext.c1.mul(&ciphertext.u);
    lhs.add(&trapdoor.t2);

    // Compute RHS: u_cap * T1 + C2
    let mut rhs = trapdoor.t1.mul(&trapdoor.u_cap);
    rhs.add(&ciphertext.c2);

    // Check if LHS equals RHS
    lhs.equals(&rhs)
}

/// Main function: Runs the full KR-PAEKS scheme
fn main() {
    let k = 20;

    let keyword_str = "secure";
    let keyword = hash_to_big(keyword_str);

    let mut params = Params::new();

    let mut sender_pk = PublicKey::new();
    let mut sender_sk = PrivateKey::new();

    let mut receiver_pk = PublicKey::new();
    let mut receiver_sk = PrivateKey::new();

    println!("======== Running KR-PAEKS ========");

    // Setup
    let setup_start = Instant::now();
    setup(&mut params, k);
    let setup_time = setup_start.elapsed();
    params.print();

    // Sender Key Generation
    let sender_keygen_start = Instant::now();
    keygen(&params, &mut sender_pk, &mut sender_sk);
    let sender_keygen_time = sender_keygen_start.elapsed();

    println!("\n======== Begin Sender ========\n");
    sender_sk.print();
    sender_pk.print();
    println!("\n======== End Sender ========\n");

    // Receiver Key Generation
    let receiver_keygen_start = Instant::now();
    keygen(&params, &mut receiver_pk, &mut receiver_sk);
    let receiver_keygen_time = receiver_keygen_start.elapsed();

    println!("\n======== Begin Receiver ========\n");
    receiver_sk.print();
    receiver_pk.print();
    println!("\n======== End Receiver ========\n");
```

```
    // Encrypt
    let ciphertext_start = Instant::now();
    let ciphertext = encrypt(&params, &receiver_pk, &sender_sk, &keyword);
    let ciphertext_time = ciphertext_start.elapsed();
    ciphertext.print();

    // Generate Trapdoor
    let trapdoor_start = Instant::now();
    let trapdoor1 = trapdoor(&params, &sender_pk, &receiver_sk, &keyword);
    let trapdoor_time = trapdoor_start.elapsed();
    trapdoor1.print();

    // Test
    let test_start = Instant::now();
    let test_result = test(&ciphertext, &trapdoor1);
    let test_time = test_start.elapsed();

    // if test_result {
    //     println!("Test Successful!\n");
    //     println!("====== Runtime ======\nSetup Time = {:?}\nSender KeyGen Time = {:?}\nReceiver KeyGen Time = {:?}\nCiphertext
 Time = {:?}\nTrapdoor Time = {:?}\nTest Time = {:?}",
    //                 setup_time, sender_keygen_time, receiver_keygen_time, ciphertext_time, trapdoor_time, test_time);
    //     println!("====================");
    // } else {
    //     println!("Test Unsuccessful!");
    // }

    // let url = "mysql://root:root@192.168.68.110/EnronMailDS";
    // let pool = Pool::new(url)?;

    // // test_sql_enc(&params, &receiver_pk, &sender_sk, "Forwarded", &pool).unwrap();

    // // let keyword_test = "Forwarded";
    // // let kw = hash_to_big(keyword_test);
    // // let trapdoor_start = Instant::now();
    // // let testt = trapdoor(&params, &sender_pk, &receiver_sk, &kw);
    // // let trapdoor_time = trapdoor_start.elapsed().as_millis();
    // // println!("\nTime for Generating Trapdoor with k={}: {} ms.\n", params.get_k(), trapdoor_time);

    // let kw1: &[String] = &vec!["forwarded".to_string(), "please".to_string(), "hey".to_string()];
    // test_sql_enc_multi(&params, &receiver_pk, &sender_sk, kw1, &pool).unwrap();

    // // let kw = hash_to_big_array(kw1);
    // let trapdoor_start = Instant::now();
    // let testt = trapdoor_multi_keyword(&params, &sender_pk, &receiver_sk, &kw1);
    // let trapdoor_time = trapdoor_start.elapsed().as_millis();
    // println!("\nTime for Generating Trapdoor with k={}: {} ms.\n", params.get_k(), trapdoor_time);

    // test_sql_test(&params, &testt, &pool).unwrap();

    // Ok(())
}

/// Converts a string to a `BIG` using SHAKE256.
pub fn hash_to_big(input: &str) -> big::BIG {
    use mcore::sha3::{SHA3, SHAKE256};

    let mut hasher = SHA3::new(SHAKE256);
    hasher.process_array(input.as_bytes());

    let mut output = [0u8; big::MODBYTES];
    hasher.shake(&mut output, big::MODBYTES);

    big::BIG::frombytes(&output)
}

fn hash_to_big_array(keywords: &[String]) -> big::BIG {
    let mut combined_str = String::new();
    for keyword in keywords {
        combined_str.push_str(keyword);
    }

    use mcore::sha3::{SHA3, SHAKE256};

    let mut hasher = SHA3::new(SHAKE256);
    hasher.process_array(combined_str.as_bytes());

    let mut output = [0u8; big::MODBYTES];
    hasher.shake(&mut output, big::MODBYTES);

    big::BIG::frombytes(&output)
}

fn is_valid(p: &ecp::ECP) -> isize {
    let mut bytes = vec![0; big::MODBYTES + 1];
```

```
    p.tobytes(&mut bytes, true);

    let result = ecdh::public_key_validate(&bytes);
    if result != 0 {
        return result;
    }
    0
}

// pub fn test_sql_enc(
//     params: &Params,
//     rpk: &PublicKey,
//     ssk: &PrivateKey,
//     kw: &str,
//     pool: &Pool,  // Pass a pre-initialized connection pool
// ) -> std::result::Result<(), Box<dyn Error>> {
//     let mut conn = pool.get_conn()?;
//     let query = "SELECT IdEmail, body, Date, `X-To` FROM JohnArnoldMail";
//     let emails: Vec<(i32, Option<String>, String, String)> = conn.query(query)?;

//     let total_enc_time = AtomicU64::new(0);
//     let cnt = AtomicU64::new(0);
//     let keyword = hash_to_big(kw);

//     // Use par_iter() instead of into_par_iter()
//     let encrypted_results: Vec<(String, String, String, i32)> = emails
//         .par_iter()
//         .filter_map(|(id, body, date, to)| {
//             if let Some(body) = body {
//                 if body.contains(kw) {
//                     let start = Instant::now();
//                     let ciphertext = encrypt(params, rpk, ssk, &keyword);
//                     let elapsed = start.elapsed().as_millis() as u64;

//                     total_enc_time.fetch_add(elapsed, Ordering::SeqCst);
//                     cnt.fetch_add(1, Ordering::SeqCst);

//                     let string_cipher = serde_cbor::to_vec(&ciphertext).ok()?; // Use CBOR instead of JSON
//                     let encoded_cipher = base64::encode(&string_cipher);
//                     return Some((encoded_cipher, date.clone(), to.clone(), *id));
//                 }
//             }
//             None
//         })
//         .collect();

//     // Batch update with prepared statements
//     if !encrypted_results.is_empty() {
//         let mut stmt = conn.prep("UPDATE JohnArnoldMail SET ciphertext = ? WHERE `Date` = ? AND `X-To` = ? AND `IdEmail` = ?")
?;
//         conn.exec_batch(&stmt, encrypted_results)?;
//     }

//     let total_time = total_enc_time.load(Ordering::SeqCst);
//     let count = cnt.load(Ordering::SeqCst);

//     println!("\nTotal Number of Emails Encrypted: {}\n", count);
//     println!("Total Time Encrypting {} Emails: {} ms.\n", count, total_time);
//     if count > 0 {
//         println!("Average Time Encrypting 1 Email: {} ms.\n", total_time / count);
//     }

//     Ok(())
// }

// pub fn test_sql_enc_multi(
//     params: &Params,
//     rpk: &PublicKey,
//     ssk: &PrivateKey,
//     kw: &[String],
//     pool: &Pool,  // Pass a pre-initialized connection pool
// ) -> std::result::Result<(), Box<dyn Error>> {
//     let mut conn = pool.get_conn()?;
//     let query = "SELECT IdEmail, body, Date, `X-To` FROM JohnArnoldMail";
//     let emails: Vec<(i32, Option<String>, String, String)> = conn.query(query)?;

//     let total_enc_time = AtomicU64::new(0);
//     let cnt = AtomicU64::new(0);
//     let keyword = hash_to_big_array(kw);

//     // Use par_iter() instead of into_par_iter()
//     let encrypted_results: Vec<(String, String, String, i32)> = emails
//     .par_iter()
//     .filter_map(|(id, body, date, to)| {
//         if let Some(body) = body {
```

```
//             let mut ccontains_all_keywords = true;
//             let body_lower = body.to_lowercase();

//             // Check if body contains all keywords
//             for keyword in kw {
//                 if !body_lower.contains(&keyword.to_lowercase()) {
//                     ccontains_all_keywords = false;
//                     break;
//                 }
//             }

//             if ccontains_all_keywords {
//                 let start = Instant::now();
//                 let ciphertext = encrypt_multi_keyword(params, rpk, ssk, &kw);  // Assuming kw is the list of keywords
//                 let elapsed = start.elapsed().as_millis() as u64;

//                 // Accumulate encryption time and count
//                 total_enc_time.fetch_add(elapsed, Ordering::SeqCst);
//                 cnt.fetch_add(1, Ordering::SeqCst);

//                 // Serialize ciphertext using CBOR and encode it in base64
//                 let string_cipher = serde_cbor::to_vec(&ciphertext).ok()?;
//                 let encoded_cipher = base64::encode(&string_cipher);

//                 // Return the result as a tuple
//                 return Some((encoded_cipher, date.clone(), to.clone(), *id));
//             }
//         }
//         None
//     })
//     .collect();

//     // Batch update with prepared statements
//     if !encrypted_results.is_empty() {
//         let mut stmt = conn.prep("UPDATE JohnArnoldMail SET ciphertext = ? WHERE `Date` = ? AND `X-To` = ? AND `IdEmail` = ?")
?;
//         conn.exec_batch(&stmt, encrypted_results)?;
//     }

//     let total_time = total_enc_time.load(Ordering::SeqCst);
//     let count = cnt.load(Ordering::SeqCst);

//     println!("\nTotal Number of Emails Encrypted: {}\n", count);
//     println!("Total Time Encrypting {} Emails: {} ms.\n", count, total_time);
//     if count > 0 {
//         println!("Average Time Encrypting 1 Email: {} ms.\n", total_time / count);
//     }

//     Ok(())
// }

// pub fn test_sql_test(params: &Params, t: &Trapdoor, pool: &Pool) -> std::result::Result<(), Box<dyn Error>> {
//     let mut conn = pool.get_conn()?;
//     let query = "SELECT ciphertext, Date, `X-To` FROM JohnArnoldMail WHERE ciphertext IS NOT NULL";
//     let emails: Vec<(Option<String>, String, String)> = conn.query(query)?;

//     let total_test_time = AtomicU64::new(0);
//     let cnt = AtomicU64::new(0);
//     // let keyword = hash_to_big(kw);

//     let matched_results: Vec<(String, String)> = emails
//         .par_iter()
//         .filter_map(|(ciphertext_opt, date, to)| {
//             if let Some(ciphertext_base64) = ciphertext_opt {
//                 // Decode base64 & CBOR
//                 let decoded_ciphertext = base64::decode(ciphertext_base64).ok()?;
//                 let ciphertext: Ciphertext = serde_cbor::from_slice(&decoded_ciphertext).ok()?;

//                 let start = Instant::now();
//                 let result =  test(&ciphertext, t);
//                 let elapsed = start.elapsed().as_millis() as u64;
//                 total_test_time.fetch_add(elapsed, Ordering::SeqCst);
//                 cnt.fetch_add(1, Ordering::SeqCst);

//                 if result {
//                     return Some((date.clone(), to.clone()));
//                 }
//             }
//             None
//         })
//         .collect();

//     println!("\nTotal Number of Emails Matched: {}\n", cnt.load(Ordering::SeqCst));
//     println!("Total Time Testing {} Emails: {} ms.\n", cnt.load(Ordering::SeqCst), total_test_time.load(Ordering::SeqCst));
```

src-tauri/src/kr_paeks/main.rs (continued)

```
//     Ok(())
// }
```

src-tauri/src/kr_paeks/mod.rs

```
pub mod api;
pub mod ciphertext;
pub mod lagrange;
pub mod main;
pub mod params;
pub mod polynomial;
pub mod private_key;
pub mod public_key;
pub mod trapdoor;
```

src-tauri/src/kr_paeks/params.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;
use std::fmt;

pub struct Params {
    pub g1: ecp::ECP,
    pub g2: ecp::ECP,
    pub order: big::BIG,
    pub msk: big::BIG,
    pub k: usize,
}

impl Params {
    // Constructor (equivalent to Java's constructor)
    pub fn new() -> Self {
        Params {
            g1: ecp::ECP::new(),
            g2: ecp::ECP::new(),
            msk: big::BIG::new(),
            order: big::BIG::new(),
            k: 0,
        }
    }

    // Setters (explicit setters only if needed)
    pub fn set_params(
        &mut self,
        g1: ecp::ECP,
        g2: ecp::ECP,
        msk: big::BIG,
        order: big::BIG,
        k: usize,
    ) {
        self.g1 = g1;
        self.g2 = g2;
        self.msk = msk;
        self.order = order;
        self.k = k;
    }

    pub fn get_k(&self) -> usize {
        self.k
    }

    pub fn get_order(&self) -> &big::BIG {
        &self.order
    }

    pub fn get_g1(&self) -> &ecp::ECP {
        &self.g1
    }

    pub fn get_g2(&self) -> &ecp::ECP {
        &self.g2
    }

    // Custom to_string function
    pub fn print(&self) {
        println!("========Begin Params=========");
        println!("k: {}", self.k);
        println!("order: {}", big_to_hex(&self.order));
        println!("msk: {}", big_to_hex(&self.msk));
        println!("g1: {}", ecp_to_hex(&self.g1));
        println!("g2: {}", ecp_to_hex(&self.g2));
        println!("========End of Params=========");
    }

    pub fn format_full(&self) -> String {
        format!(
            "k: {}\norder: {}\nmsk: {}\ng1: {}\ng2: {}",
            self.k,
            big_to_hex(&self.order),
            big_to_hex(&self.msk),
            ecp_to_hex(&self.g1),
            ecp_to_hex(&self.g2)
        )
    }
}
```

src-tauri/src/kr_paeks/params.rs (continued)

```
fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}
```

src-tauri/src/kr_paeks/polynomial.rs

```
extern crate mcore;
use mcore::ed25519::big;
use mcore::rand::RAND;
use rand::RngCore;
use std::fmt;

#[derive(Clone)]
/// Polynomial operations.
pub struct Polynomial {
    degree: usize,
    coeff: Vec<big::BIG>,
    order: big::BIG,
}

impl Polynomial {
    pub fn new() -> Self {
        Polynomial {
            degree: 0,
            coeff: Vec::new(),
            order: big::BIG::new(),
        }
    }

    pub fn new_polynomial(degree: usize, order: big::BIG) -> Self {
        let mut rng = gen_seed();

        let mut coeff = vec![big::BIG::new(); degree];

        for i in 0..degree {
            coeff[i] = big::BIG::randomnum(&order, &mut rng);
        }

        Polynomial {
            degree,
            coeff,
            order,
        }
    }

    pub fn evaluate(&self, x: &big::BIG) -> big::BIG {
        let mut accum = big::BIG::new();
        for j in 0..self.degree {
            let exp = big::BIG::new_int(j as isize);
            let mut x_clone = x.clone();
            let t2 = big::BIG::modmul(
                &self.coeff[j],
                &x_clone.powmod(&exp, &self.order),
                &self.order,
            );
            accum.add(&t2);
            accum.rmod(&self.order);
        }
        accum
    }

    pub fn get_degree(&self) -> usize {
        self.degree
    }

    pub fn get_coeff(&self) -> &Vec<big::BIG> {
        &self.coeff
    }

    pub fn get_coeff_at(&self, i: usize) -> &big::BIG {
        &self.coeff[i]
    }

    pub fn get_order(&self) -> &big::BIG {
        &self.order
    }

    pub fn fmt(&self) -> String {
        let mut str = String::new();
        str.push_str("\n========Begin Polynomial=========\n");
        str.push_str(&format!("degree: {}\n", self.degree));
        str.push_str(&format!("order: {}\n", big_to_hex(&self.order)));

        for (i, coeff) in self.coeff.iter().enumerate() {
            str.push_str(&format!("coeff[{}]: {}\n", i, big_to_hex(&coeff)));
        }
```

src-tauri/src/kr_paeks/polynomial.rs (continued)

```
        str.push_str("========End of Polynomial========\n");
        return str;
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn gen_seed() -> RAND {
    let mut rng = RAND::new();
    let mut seed = [0u8; 100];
    rand::thread_rng().fill_bytes(&mut seed);
    rng.clean();
    rng.seed(100, &seed);
    rng
}
```

src-tauri/src/kr_paeks/private_key.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;

use super::polynomial;
use super::polynomial::Polynomial;
use std::fmt;

#[derive(Clone)]
pub struct PrivateKey {
    pub p1: polynomial::Polynomial,
    pub p2: polynomial::Polynomial,
}

impl PrivateKey {
    // Constructor (equivalent to Java's constructor)
    pub fn new() -> Self {
        PrivateKey {
            p1: Polynomial::new(),
            p2: Polynomial::new(),
        }
    }

    // Setters (explicit setters only if needed)
    pub fn set_private_key(&mut self, p1: Polynomial, p2: Polynomial) {
        self.p1 = p1;
        self.p2 = p2;
    }

    pub fn get_p1(&self) -> &Polynomial {
        &self.p1
    }

    pub fn get_p2(&self) -> &Polynomial {
        &self.p2
    }

    pub fn print(&self) {
        // println!("========Begin Private Key=========");
        for i in 0..self.p1.get_degree() {
            println!("P1[{}]: {}", i, big_to_hex(&self.p1.get_coeff_at(i)));
        }
        for i in 0..self.p2.get_degree() {
            println!("P2[{}]: {}", i, big_to_hex(&self.p2.get_coeff_at(i)));
        }
        // println!("========End of Private Key=========");
    }

    pub fn format_full(&self) -> String {
        let mut output = String::new();
        for i in 0..self.p1.get_degree() {
            output.push_str(&format!(
                "P1[{}]: {}\n",
                i,
                big_to_hex(&self.p1.get_coeff_at(i))
            ));
        }
        for i in 0..self.p2.get_degree() {
            output.push_str(&format!(
                "P2[{}]: {}\n",
                i,
                big_to_hex(&self.p2.get_coeff_at(i))
            ));
        }
        output
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
```

src-tauri/src/kr_paeks/private_key.rs (continued)

```
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}
```

src-tauri/src/kr_paeks/public_key.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;
use std::fmt;

#[derive(Clone)]
pub struct PublicKey {
    pub dt: Vec<ecp::ECP>,
}

impl PublicKey {
    // Constructor
    pub fn new() -> Self {
        PublicKey { dt: Vec::new() }
    }

    // Setter
    pub fn set_public_key(&mut self, dt: Vec<ecp::ECP>) {
        self.dt = dt;
    }

    // Getter (returns a reference to avoid unnecessary cloning)
    pub fn get_public_key(&self) -> &Vec<ecp::ECP> {
        &self.dt
    }

    // Get specific index
    pub fn get_public_key_at(&self, i: usize) -> Option<&ecp::ECP> {
        self.dt.get(i)
    }

    // Custom to_string function

    pub fn print(&self) {
        // println!("========Begin Public Key=========");
        for i in 0..self.dt.len() {
            println!("dt[{}]: {}", i, ecp_to_hex(&self.dt[i]));
        }
        // println!("========End of Public Key=========");
    }

    pub fn format_full(&self) -> String {
        let mut output = String::new();
        for i in 0..self.dt.len() {
            output.push_str(&format!("dt[{}]: {}\n", i, ecp_to_hex(&self.dt[i])));
        }
        output
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}
```

src-tauri/src/kr_paeks/trapdoor.rs

```
extern crate mcore;

use mcore::ed25519::big;
use mcore::ed25519::ecp;
use std::fmt;

pub struct Trapdoor {
    pub t1: ecp::ECP,
    pub t2: ecp::ECP,
    pub u_cap: big::BIG,
}

impl Trapdoor {
    // Constructor
    pub fn new() -> Self {
        Trapdoor {
            t1: ecp::ECP::new(),
            t2: ecp::ECP::new(),
            u_cap: big::BIG::new(),
        }
    }

    pub fn new_trapdoor(t1: &ecp::ECP, t2: &ecp::ECP, u_cap: &big::BIG) -> Self {
        Trapdoor {
            t1: t1.clone(),
            t2: t2.clone(),
            u_cap: u_cap.clone(),
        }
    }

    // Setter (if necessary)
    pub fn set_trapdoor(&mut self, t1: ecp::ECP, t2: ecp::ECP, u_cap: big::BIG) {
        self.t1 = t1;
        self.t2 = t2;
        self.u_cap = u_cap;
    }

    // Custom to_string function
    pub fn print(&self) {
        println!("========Begin Trapdoor=========");
        println!("t1: {}", ecp_to_hex(&self.t1));
        println!("t2: {}", ecp_to_hex(&self.t2));
        println!("u_cap: {}", big_to_hex(&self.u_cap));
        println!("========End of Trapdoor=========");
    }

    pub fn format_full(&self) -> String {
        let mut output = String::new();
        output.push_str(&format!("t1: {}\n", ecp_to_hex(&self.t1)));
        output.push_str(&format!("t2: {}\n", ecp_to_hex(&self.t2)));
        output.push_str(&format!("u_cap: {}\n", big_to_hex(&self.u_cap)));
        output
    }
}

fn big_to_hex(b: &big::BIG) -> String {
    let mut bytes = [0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes.iter().map(|byte| format!("{:02x}", byte)).collect()
}

fn ecp_to_hex(p: &ecp::ECP) -> String {
    if p.is_infinity() {
        return String::from("infinity");
    }
    let wx = p.getx();
    let wy = p.gety();
    format!("({}, {})", big_to_hex(&wx), big_to_hex(&wy))
}
```

src-tauri/src/kr_peks/api.rs

```
use super::main as krpeks_core;
use crate::kr_peks::ciphertext::Ciphertext;
use crate::kr_peks::params::Params;
use crate::kr_peks::private_key::PrivateKey;
use crate::kr_peks::public_key::PublicKey;
use crate::kr_peks::trapdoor::Trapdoor;
use crate::kr_peks::utils::*;
use tauri::command;

use once_cell::sync::Lazy;
use std::sync::Mutex;
use std::time::Instant;

static PARAMS: Lazy<Mutex<Option<Params>>> = Lazy::new(|| Mutex::new(None));
static PK: Lazy<Mutex<Option<PublicKey>>> = Lazy::new(|| Mutex::new(None));
static SK: Lazy<Mutex<Option<PrivateKey>>> = Lazy::new(|| Mutex::new(None));
static CIPHERTEXT: Lazy<Mutex<Option<Ciphertext>>> = Lazy::new(|| Mutex::new(None));
static TRAPDOOR: Lazy<Mutex<Option<Trapdoor>>> = Lazy::new(|| Mutex::new(None));
static TOTAL_RUNTIME: Lazy<Mutex<f64>> = Lazy::new(|| Mutex::new(0.0)); // Total computation time

#[command]
pub fn kr_peks_setup(k: usize) -> String {
    let start_time = Instant::now();

    let mut params = Params::new();
    krpeks_core::setup(&mut params, k);

    let duration = start_time.elapsed();
    *PARAMS.lock().unwrap() = Some(params);
    *TOTAL_RUNTIME.lock().unwrap() = 0.0; // Reset runtime
    *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate

    let mut output = String::new();
    output.push_str("  KR-PEKS Setup Complete!\n\n");
    output.push_str(&PARAMS.lock().unwrap().as_ref().unwrap().format_full());
    output.push_str(&format!("\n  Setup Time: {:.2?}\n", duration));

    output
}

#[command]
pub fn kr_peks_keygen() -> String {
    let start_time = Instant::now();

    let params_lock = PARAMS.lock().unwrap();
    if let Some(ref params) = *params_lock {
        let mut pk = PublicKey::new();
        let mut sk = PrivateKey::new();

        krpeks_core::keygen(params, &mut pk, &mut sk);

        *PK.lock().unwrap() = Some(pk);
        *SK.lock().unwrap() = Some(sk);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate

        let mut output = String::new();
        output.push_str("  KR-PEKS Keygen Complete!\n\n");
        output.push_str("  Private Key:\n");
        output.push_str(&SK.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str("\n\n  Public Key:\n");
        output.push_str(&PK.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str(&format!("\n  Keygen Time: {:.2?}\n", duration));

        output
    } else {
        "  Error: KR-PEKS setup not done yet!".into()
    }
}

#[command]
pub fn kr_peks_encrypt(keyword: String) -> String {
    let start_time = Instant::now();

    let params_lock = PARAMS.lock().unwrap();
    let pk_lock = PK.lock().unwrap();

    if let (Some(ref params), Some(ref pk)) = (params_lock.as_ref(), pk_lock.as_ref()) {
        let keyword_bytes = string_to_bytes(&keyword);
```

src-tauri/src/kr_peks/api.rs (continued)

```
        let ct = krpeks_core::peks(params, pk, &keyword_bytes);

        match ct {
            Some(ciphertext) => {
                *CIPHERTEXT.lock().unwrap() = Some(ciphertext);

                let duration = start_time.elapsed();
                *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate

                let mut output = String::new();
                output.push_str("  KR-PEKS Encryption Complete!\n\n");
                output.push_str("  Ciphertext:\n");
                output.push_str(&CIPHERTEXT.lock().unwrap().as_ref().unwrap().format_full());
                output.push_str(&format!("\n  Encryption Time: {:.2?}\n", duration));

                output
            }
            None => "  Error: Encryption failed. Invalid public key.".into(),
        }
    } else {
        "  Error: Keygen not done yet!".into()
    }
}

#[command]
pub fn kr_peks_trapdoor(keyword: String) -> String {
    let start_time = Instant::now();

    let params_lock = PARAMS.lock().unwrap();
    let pk_lock = PK.lock().unwrap();
    let sk_lock = SK.lock().unwrap();

    if let (Some(ref params), Some(ref _pk), Some(ref sk)) =
        (params_lock.as_ref(), pk_lock.as_ref(), sk_lock.as_ref())
    {
        let keyword_bytes = string_to_bytes(&keyword);

        let td = krpeks_core::trapdoor(params, sk, &keyword_bytes);

        *TRAPDOOR.lock().unwrap() = Some(td);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate

        let mut output = String::new();
        output.push_str("  KR-PEKS Trapdoor Generation Complete!\n\n");
        output.push_str("  Trapdoor:\n");
        output.push_str(&TRAPDOOR.lock().unwrap().as_ref().unwrap().format_full());
        output.push_str(&format!("\n  Trapdoor Time: {:.2?}\n", duration));

        output
    } else {
        "  Error: Keygen not done yet!".into()
    }
}

#[command]
pub fn kr_peks_test() -> String {
    let start_time = Instant::now();

    let ct_lock = CIPHERTEXT.lock().unwrap();
    let td_lock = TRAPDOOR.lock().unwrap();

    if let (Some(ref ct), Some(ref td)) = (ct_lock.as_ref(), td_lock.as_ref()) {
        let result = krpeks_core::test(ct, td);

        let duration = start_time.elapsed();
        *TOTAL_RUNTIME.lock().unwrap() += duration.as_secs_f64(); // accumulate

        let total_runtime = *TOTAL_RUNTIME.lock().unwrap(); // fetch final total

        let mut output = String::new();
        if result {
            output.push_str("  KR-PEKS Test Successful!\n\n");
        } else {
            output.push_str("  KR-PEKS Test Failed!\n\n");
        }

        output.push_str(&format!("  Test Time: {:.2?}\n", duration));
        output.push_str(&format!(
            "  Total Computation Time: {}\n",
            format_runtime(total_runtime)
        ));
```

src-tauri/src/kr_peks/api.rs (continued)

```
        output
    } else {
        "  Error: Need to run encryption and trapdoor first!".into()
    }
}

fn format_runtime(seconds: f64) -> String {
    if seconds < 1.0 {
        // Less than 1 second → show milliseconds
        format!("{:.0} ms", seconds * 1000.0)
    } else {
        // 1 second or more → show seconds
        format!("{:.2} s", seconds)
    }
}
```

src-tauri/src/kr_peks/ciphertext.rs

```
use super::utils::ecp_to_hex;
use mcore::ed25519::ecp;

#[derive(Clone)]
pub struct Ciphertext {
    u1: ecp::ECP,
    u2: ecp::ECP,
    sw: ecp::ECP,
}

impl Ciphertext {
    pub fn new() -> Self {
        Ciphertext {
            u1: ecp::ECP::new(),
            u2: ecp::ECP::new(),
            sw: ecp::ECP::new(),
        }
    }

    pub fn new_ciphertext(u1: &ecp::ECP, u2: &ecp::ECP, sw: &ecp::ECP) -> Self {
        Ciphertext {
            u1: u1.clone(),
            u2: u2.clone(),
            sw: sw.clone(),
        }
    }

    pub fn set_ciphertext(&mut self, u1: ecp::ECP, u2: ecp::ECP, sw: ecp::ECP) {
        self.u1 = u1;
        self.u2 = u2;
        self.sw = sw;
    }

    pub fn get_u1(&self) -> &ecp::ECP {
        &self.u1
    }

    pub fn get_u2(&self) -> &ecp::ECP {
        &self.u2
    }

    pub fn get_sw(&self) -> &ecp::ECP {
        &self.sw
    }

    pub fn print(&self) {
        println!();
        println!("========Begin Ciphertext=========");
        println!("u1: {}", ecp_to_hex(&self.u1));
        println!("u2: {}", ecp_to_hex(&self.u2));
        println!("sw: {}", ecp_to_hex(&self.sw));
        println!("========End of Ciphertext=========");
    }

    pub fn is_valid(&self) -> bool {
        if self.u1.is_infinity() || self.u2.is_infinity() || self.sw.is_infinity() {
            return false;
        }
        true
    }

    pub fn format_full(&self) -> String {
        format!(
            "u1: {}\nu2: {}\nsw: {}",
            ecp_to_hex(&self.u1),
            ecp_to_hex(&self.u2),
            ecp_to_hex(&self.sw)
        )
    }
}
```

src-tauri/src/kr_peks/main.rs

```
#![allow(non_snake_case)]

use super::ciphertext::Ciphertext;
use super::params::Params;
use super::polynomial::Polynomial;
use super::private_key::PrivateKey;
use super::public_key::PublicKey;
use super::trapdoor::Trapdoor;
use super::utils::*;

use base64::*;
use mcore::ed25519::big;
use mcore::ed25519::ecp;
use mcore::ed25519::rom;
use mysql::prelude::*;
use mysql::*;
use serde_json::{json, Value};
use std::env;
use std::time::Instant;

pub fn setup(params: &mut Params, k: usize) {
    let order = big::BIG::new_ints(&rom::CURVE_ORDER);

    let mut rng = gen_seed();
    let r = big::BIG::randomnum(&order, &mut rng);

    let g1 = ecp::ECP::generator();
    let g2 = g1.mul(&r);

    params.set_params(k, order, g1, g2);
}

pub fn keygen(params: &Params, pk: &mut PublicKey, sk: &mut PrivateKey) {
    let k = params.get_k() + 1;
    let order = params.get_order();
    let g1 = params.get_g1();
    let g2 = params.get_g2();

    let p1 = Polynomial::new_polynomial(k, *order);
    let p2 = Polynomial::new_polynomial(k, *order);

    let mut Dt = vec![ecp::ECP::new(); k];

    for i in 0..k {
        Dt[i] = g1.mul(&p1.get_coeff_i(i));
        Dt[i].add(&g2.mul(&p2.get_coeff_i(i)));
    }

    sk.set_private_key(p1, p2);
    pk.set_public_key(Dt);
}

pub fn peks(params: &Params, pk: &PublicKey, w: &Vec<u8>) -> Option<Ciphertext> {
    if pk.is_valid() != 0 {
        println!("Public Key is invalid!");
        return None;
    }

    let k = params.get_k() + 1;
    let order = params.get_order();
    let g1 = params.get_g1();
    let g2 = params.get_g2();

    let mut hashw = hash_to_big(w, order);
    // println!("Hash of '{}': {}", w, big_to_hex(&hashw));

    let mut rng = gen_seed();
    let r = big::BIG::randomnum(&order, &mut rng);

    let u1 = g1.mul(&r);
    let u2 = g2.mul(&r);

    let mut sw = pk
        .get_Dt_i(0)
        .mul(&big::BIG::powmod(&mut hashw, &big::BIG::new_int(0), &order));
    for i in 1..k {
        let i_isize: isize = i.try_into().unwrap();
        sw.add(&pk.get_Dt_i(i).mul(&big::BIG::powmod(
            &mut hashw,
            &big::BIG::new_int(i_isize),
```

src-tauri/src/kr_peks/main.rs (continued)

```
            &order,
        )));
    }
    sw = sw.mul(&r);

    Some(Ciphertext::new_ciphertext(&u1, &u2, &sw))
}

pub fn trapdoor(params: &Params, sk: &PrivateKey, w: &Vec<u8>) -> Trapdoor {
    let order = params.get_order();
    let hashw = hash_to_big(w, order);

    let p1 = sk.get_p1().evaluate(hashw);
    let p2 = sk.get_p2().evaluate(hashw);

    Trapdoor::new_trapdoor(&p1, &p2)
}

pub fn test(c: &Ciphertext, t: &Trapdoor) -> bool {
    let mut right = c.get_u1().mul(&t.get_p1());
    right.add(&c.get_u2().mul(&t.get_p2()));

    if c.get_sw().equals(&right) {
        true
    } else {
        false
    }
}

fn main() {
    let w = "urgent";
    let k = 20;
    // Convert strings to byte arrays
    let w_bytes = string_to_bytes(w);

    let mut params = Params::new();
    let mut pk = PublicKey::new();
    let mut sk = PrivateKey::new();
    let mut c = Ciphertext::new();

    // User import params, sk, pk, ciphertext from files: cargo run parameter.params privatekey.sk publickey.pk ciphertext.cipher
    let args: Vec<String> = env::args().collect();
    for i in &args {
        if i.contains(".params") {
            let result = import_params(&mut params, &i);
            match result {
                Ok(_) => {
                    if params.is_valid() {
                        println!("System parameters imported successfully!");
                    } else {
                        eprintln!(
                            "Failed to import system parameters: System parameters are invalid."
                        );
                        return;
                    }
                }
                Err(e) => {
                    eprintln!("Failed to import system parameters: {}", e);
                    return;
                }
            }
        } else if i.contains(".sk") {
            let result = import_sk(&mut sk, &i);
            match result {
                Ok(_) => {
                    if sk.is_valid() {
                        println!("Private key imported successfully!");
                    } else {
                        eprintln!("Failed to import private key: Private key is invalid.");
                        return;
                    }
                }
                Err(e) => {
                    eprintln!("Failed to import private key: {}", e);
                    return;
                }
            }
        } else if i.contains(".pk") {
            let result = import_pk(&mut pk, &i);
            match result {
                Ok(_) => {
                    if pk.is_valid() == 0 {
                        println!("Public key imported successfully!");
                    } else {
                        eprintln!("Failed to import public key: Public key is invalid.");
```

src-tauri/src/kr_peks/main.rs (continued)

```
                    return;
                }
            }
            Err(e) => {
                eprintln!("Failed to import public key: {}", e);
                return;
            }
        }
    } else if i.contains(".cipher") {
        let result = import_c(&mut c, &i);
        match result {
            Ok(_) => {
                if c.is_valid() {
                    println!("Ciphertext imported successfully!");
                } else {
                    eprintln!("Failed to import ciphertext: Ciphertext is invalid.");
                    return;
                }
            }
            Err(e) => {
                eprintln!("Failed to import ciphertext: {}", e);
                return;
            }
        }
    }
}

// No import from file
if *&args.len() <= 1 {
    let start = Instant::now();
    setup(&mut params, k);
    let duration = start.elapsed();
    println!("Setup took: {:?}", duration);

    let start = Instant::now();
    keygen(&params, &mut pk, &mut sk);
    let duration = start.elapsed();
    println!("Keygen took: {:?}", duration);

    let start = Instant::now();
    c = match peks(&params, &pk, &w_bytes) {
        Some(ciphertext) => ciphertext,
        None => return,
    };
    let duration = start.elapsed();
    println!("PEKS took: {:?}", duration);

    export_params(&params, "parameter.params");
    export_sk(&sk, "privatekey.sk");
    export_c(&c, "ciphertext.cipher");

    match export_pk(&pk, "publickey.pk") {
        Ok(_) => (),
        Err(e) => println!("{}", e),
    }
    let duration = start.elapsed();
}

params.print();

sk.print();
pk.print();

c.print();

let start = Instant::now();
let t = trapdoor(&params, &sk, &w_bytes);
t.print();
let duration = start.elapsed();
println!("Trapdoor took: {:?}", duration);

let start = Instant::now();
let result = test(&c, &t);
let duration = start.elapsed();

if result {
    println!("\nTest Successful!");
    println!("Test took: {:?}", duration);
} else {
    println!("\nTest Unsuccessful!");
}

//***************Performance Analysis***************
// let TestCiphertextTime = Instant::now();
// TestSQLEnc(&params, &pk, "Forwarded");
```

src-tauri/src/kr_peks/main.rs (continued)

```
    // let FinalTestCiphertextTime = TestCiphertextTime.elapsed();

    // let TestTrapdoorTime = Instant::now();
    // let trapdoor_test = trapdoor(&params, &sk, "Forwarded");
    // let FinalTestTrapdoorTime = TestTrapdoorTime.elapsed();

    // let SqlTestTime = Instant::now();
    // let SqlTestCnt = TestSQLTest(&trapdoor_test);
    // let FinalSqlTestTime = SqlTestTime.elapsed();

    // println!("\nTotal Number of Test: {}", SqlTestCnt);
    // println!("===========Runtime==========");
    // println!("Time for Encrypt All Emails Contain Keyword \"Forwarded\": {} ms", FinalTestCiphertextTime.as_millis());
    // println!("Time for Gen Trapdoor: {} ms", FinalTestTrapdoorTime.as_millis());
    // println!("Time for Test All Ciphertext: {} ms", FinalSqlTestTime.as_millis());
    // println!("===========End Runtime==========");

    // Calculate average time
    // let mut times = Vec::new();

    // for i in 0..100 {
    //     let overall_start = Instant::now();

    //     let w = "Urgent";

    //     let mut params = Params::new();
    //     let mut pk = PublicKey::new();
    //     let mut sk = PrivateKey::new();

    //     let setup_start = Instant::now();
    //     setup(&mut params);
    //     let setup_time = setup_start.elapsed();
    //     // params.print();

    //     let keygen_start = Instant::now();
    //     keygen(&params, &mut pk, &mut sk);
    //     let keygen_time = keygen_start.elapsed();
    //     // sk.print();
    //     // pk.print();

    //     let peks_start = Instant::now();
    //     let c = peks(&params, &pk, &w);
    //     let peks_time = peks_start.elapsed();
    //     // c.print();

    //     let trapdoor_start = Instant::now();
    //     let t = trapdoor(&params, &sk, &w);
    //     let trapdoor_time = trapdoor_start.elapsed();
    //     // t.print();

    //     let test_start = Instant::now();
    //     let result = test(&c, &t);
    //     let test_time = test_start.elapsed();

    //     let total_time = overall_start.elapsed();

    //     times.push(vec![
    //         setup_time.as_secs_f64() * 1000.0,
    //         keygen_time.as_secs_f64() * 1000.0,
    //         peks_time.as_secs_f64() * 1000.0,
    //         trapdoor_time.as_secs_f64() * 1000.0,
    //         test_time.as_secs_f64() * 1000.0,
    //         total_time.as_secs_f64() * 1000.0,
    //     ]);

    //     if result {
    //         println!("Test Successful!");
    //     } else {
    //         println!("Test Unsuccessful!");
    //     }

    //     // println!("{} - Setup Time: {:.2?}, Keygen Time: {:.2?}, PEKS Time: {:.2?}, Trapdoor Time: {:.2?}, Test Time: {:.2?}
, Total Time: {:.2?}", i, setup_time, keygen_time, peks_time, trapdoor_time, test_time, total_time);
    // }

    // let avg_times: Vec<f64> = (0..6).map(|i| {
    //     times.iter().map(|t| t[i]).sum::<f64>() / times.len() as f64
    // }).collect();

    // println!("\nAverage Times:");
    // println!("Setup Time: {:.2} ms", avg_times[0]);
    // println!("Keygen Time: {:.2} ms", avg_times[1]);
    // println!("PEKS Time: {:.2} ms", avg_times[2]);
    // println!("Trapdoor Time: {:.2} ms", avg_times[3]);
    // println!("Test Time: {:.2} ms", avg_times[4]);
```

src-tauri/src/kr_peks/main.rs (continued)

```
    // println!("Total Time: {:.2} ms", avg_times[5]);

    //***************End Performance Analysis***************
}

fn TestSQLEnc(params: &Params, pk: &PublicKey, w: &Vec<u8>) {
    let url = "mysql://root@127.0.0.1/EnronMailDS";
    let pool = Pool::new(url).expect("Failed to create pool.");
    let mut conn = pool.get_conn().expect("Failed to get connection.");

    let select_stmt = "SELECT `body`, `Date`, `X-To` FROM JohnArnoldMail";
    let result: Vec<Row> = conn.query(select_stmt).expect("Query failed.");

    for row in result {
        let body: Option<String> = row.get("body");
        let date: String = row.get("Date").unwrap();
        let to: String = row.get("X-To").unwrap();

        if let Some(body) = body {
            // Convert the body string to a byte vector
            let body_bytes = body.as_bytes();

            // Check if the byte vector `w` is contained in `body_bytes`
            if body_bytes.windows(w.len()).any(|window| window == w) {
                let ciphertext = match peks(&params, &pk, &w) {
                    Some(ciphertext) => ciphertext,
                    None => return,
                };

                let StringCipher = json!({
                    "u1": encode(ecp_to_bytes(ciphertext.get_u1())),
                    "u2": encode(ecp_to_bytes(ciphertext.get_u2())),
                    "sw": encode(ecp_to_bytes(ciphertext.get_sw()))
                });

                let update_stmt =
                    "UPDATE JohnArnoldMail SET ciphertext = ? WHERE `Date` = ? AND `X-To` = ?";
                conn.exec_drop(update_stmt, (StringCipher, date, to))
                    .expect("Update failed.");
            }
        }
    }
}

fn TestSQLTest(t: &Trapdoor) -> i32 {
    let url = "mysql://root@127.0.0.1/EnronMailDS";
    let pool = Pool::new(url).expect("Failed to create pool.");
    let mut conn = pool.get_conn().expect("Failed to get connection.");

    let select_stmt = "SELECT ciphertext FROM JohnArnoldMail";
    let result: Vec<Row> = conn.query(select_stmt).expect("Query failed.");

    let mut cnt = 1;

    for row in result {
        let ciphertext_opt: Option<Option<String>> = row
            .get_opt("ciphertext")
            .expect("Failed to retrieve ciphertext_opt.")
            .ok();

        if let Some(Some(ciphertext_string)) = ciphertext_opt {
            let data: Value = serde_json::from_str(&ciphertext_string)
                .expect("Failed to deserialize ciphertext.");

            let u1 = bytes_to_ecp(
                &decode(
                    data["u1"]
                        .as_str()
                        .expect("Failed to get a valid string from JSON"),
                )
                .expect("Failed to decode base64"),
            );
            let u2 = bytes_to_ecp(
                &decode(
                    data["u2"]
                        .as_str()
                        .expect("Failed to get a valid string from JSON"),
                )
                .expect("Failed to decode base64"),
            );
            let sw = bytes_to_ecp(
                &decode(
                    data["sw"]
                        .as_str()
                        .expect("Failed to get a valid string from JSON"),
```

src-tauri/src/kr_peks/main.rs (continued)

```
                )
                .expect("Failed to decode base64"),
            );

            let c = Ciphertext::new_ciphertext(&u1, &u2, &sw);

            let CntTest = test(&c, &t);
            if CntTest {
                cnt += 1;
            }
        }
    }
    cnt
}
```

src-tauri/src/kr_peks/mod.rs

```
pub mod api;
pub mod ciphertext;
pub mod main;
pub mod params;
pub mod polynomial;
pub mod private_key;
pub mod public_key;
pub mod trapdoor;
pub mod utils;
```

src-tauri/src/kr_peks/params.rs

```
use super::utils::*;
use mcore::ed25519::big;
use mcore::ed25519::ecp;

pub struct Params {
    k: usize,
    order: big::BIG,
    g1: ecp::ECP,
    g2: ecp::ECP,
}

impl Params {
    pub fn new() -> Self {
        Params {
            k: 0,
            order: big::BIG::new(),
            g1: ecp::ECP::new(),
            g2: ecp::ECP::new(),
        }
    }

    pub fn new_params(k: &usize, order: &big::BIG, g1: &ecp::ECP, g2: &ecp::ECP) -> Self {
        Params {
            k: *k,
            order: *order,
            g1: g1.clone(),
            g2: g2.clone(),
        }
    }

    pub fn set_params(&mut self, k: usize, order: big::BIG, g1: ecp::ECP, g2: ecp::ECP) {
        self.k = k;
        self.order = order;
        self.g1 = g1;
        self.g2 = g2;
    }

    pub fn get_k(&self) -> usize {
        self.k
    }

    pub fn get_order(&self) -> &big::BIG {
        &self.order
    }

    pub fn get_g1(&self) -> &ecp::ECP {
        &self.g1
    }

    pub fn get_g2(&self) -> &ecp::ECP {
        &self.g2
    }

    pub fn print(&self) {
        println!();
        println!("========Begin Params=========");
        println!("g1: {}", ecp_to_hex(&self.g1));
        println!("g2: {}", ecp_to_hex(&self.g2));
        println!("order: {}", big_to_hex(&self.order));
        println!("k: {}", self.k);
        println!("========End of Params=========");
    }

    pub fn is_valid(&self) -> bool {
        if self.k == 0
            || big::BIG::comp(&self.order, &big::BIG::new()) == 0
            || self.g1.is_infinity()
            || self.g2.is_infinity()
        {
            return false;
        }
        true
    }

    pub fn format_full(&self) -> String {
        format!(
            "g1: {}\ng2: {}\norder: {}\nk: {}",
            ecp_to_hex(&self.g1),
            ecp_to_hex(&self.g2),
            big_to_hex(&self.order),
```

src-tauri/src/kr_peks/params.rs (continued)

```
            self.k
        )
    }
}
```

src-tauri/src/kr_peks/polynomial.rs

```
use super::utils::*;
use mcore::ed25519::big;

pub struct Polynomial {
    degree: usize,
    coeff: Vec<big::BIG>,
    order: big::BIG,
}

impl Polynomial {
    pub fn new() -> Self {
        Polynomial {
            degree: 0,
            coeff: Vec::new(),
            order: big::BIG::new(),
        }
    }

    pub fn new_polynomial(degree: usize, order: big::BIG) -> Self {
        let mut rng = gen_seed();

        let mut coeff = vec![big::BIG::new(); degree];

        for i in 0..degree {
            coeff[i] = big::BIG::randomnum(&order, &mut rng);
        }

        Polynomial {
            degree,
            coeff,
            order,
        }
    }

    pub fn set_polynomial(&mut self, degree: usize, coeff: Vec<big::BIG>, order: big::BIG) {
        self.degree = degree;
        self.coeff = coeff;
        self.order = order;
    }

    pub fn evaluate(&self, mut x: big::BIG) -> big::BIG {
        let mut accum = big::BIG::new_int(0);

        for i in 0..self.degree {
            let i_isize: isize = i.try_into().unwrap();

            let temp = big::BIG::modmul(
                &self.coeff[i],
                &big::BIG::powmod(&mut x, &big::BIG::new_int(i_isize), &self.order),
                &self.order,
            );

            accum.add(&temp);
            accum.rmod(&self.order);
        }

        accum
    }

    pub fn get_degree(&self) -> usize {
        self.degree
    }

    pub fn get_coeff(&self) -> &Vec<big::BIG> {
        &self.coeff
    }

    pub fn get_coeff_i(&self, i: usize) -> &big::BIG {
        &self.coeff[i]
    }

    pub fn get_order(&self) -> &big::BIG {
        &self.order
    }

    pub fn print(&self) {
        println!("========Begin Polynomial=========");
        println!("degree: {}", self.degree);
        println!("order: {}", big_to_hex(&self.order));
        for i in 0..self.coeff.len() {
```

src-tauri/src/kr_peks/polynomial.rs (continued)

```
            println!("coeff[{}]: {}", i, big_to_hex(&self.coeff[i]));
        }
        println!("========End of Polynomial=========");
    }

    pub fn format_full(&self) -> String {
        let mut result = String::new();
        result.push_str(&format!(
            "degree: {}\norder: {}\n",
            self.degree,
            big_to_hex(&self.order)
        ));
        for i in 0..self.coeff.len() {
            result.push_str(&format!("coeff[{}]: {}\n", i, big_to_hex(&self.coeff[i])));
        }
        result
    }

    pub fn is_valid(&self) -> bool {
        if self.degree == 0
            || big::BIG::comp(&self.order, &big::BIG::new()) == 0
            || self.coeff.is_empty()
            || self.coeff.len() != self.degree
        {
            return false;
        }
        true
    }
}
```

src-tauri/src/kr_peks/private_key.rs

```
use super::polynomial::Polynomial;

pub struct PrivateKey {
    p1: Polynomial,
    p2: Polynomial,
}

impl PrivateKey {
    pub fn new() -> Self {
        PrivateKey {
            p1: Polynomial::new(),
            p2: Polynomial::new(),
        }
    }

    pub fn new_private_key(p1: Polynomial, p2: Polynomial) -> Self {
        PrivateKey { p1, p2 }
    }

    pub fn set_private_key(&mut self, p1: Polynomial, p2: Polynomial) {
        self.p1 = p1;
        self.p2 = p2;
    }

    pub fn get_p1(&self) -> &Polynomial {
        &self.p1
    }

    pub fn get_p2(&self) -> &Polynomial {
        &self.p2
    }

    pub fn print(&self) {
        println!();
        println!("========Begin Private Key=========");
        println!("P1:");
        self.p1.print();
        println!("P2:");
        self.p2.print();
        println!("========End of Private Key=========");
    }

    pub fn is_valid(&self) -> bool {
        if !self.p1.is_valid() || !self.p2.is_valid() {
            return false;
        }
        true
    }

    pub fn format_full(&self) -> String {
        format!(
            "P1:\n{}\nP2:\n{}",
            self.p1.format_full(),
            self.p2.format_full()
        )
    }
}
```

src-tauri/src/kr_peks/public_key.rs

```
use super::utils::*;
use mcore::ed25519::big;
use mcore::ed25519::ecdh;
use mcore::ed25519::ecp;

pub struct PublicKey {
    Dt: Vec<ecp::ECP>,
}

impl PublicKey {
    pub fn new() -> Self {
        PublicKey { Dt: Vec::new() }
    }

    pub fn new_public_key(Dt: Vec<ecp::ECP>) -> Self {
        PublicKey { Dt: Dt }
    }

    pub fn set_public_key(&mut self, Dt: Vec<ecp::ECP>) {
        self.Dt = Dt;
    }

    pub fn get_Dt(&self) -> &Vec<ecp::ECP> {
        &self.Dt
    }

    pub fn get_Dt_i(&self, i: usize) -> &ecp::ECP {
        &self.Dt[i]
    }

    pub fn print(&self) {
        println!();
        println!("========Begin Public Key=========");
        for i in 0..self.Dt.len() {
            println!("Dt[{}]: {}", i, ecp_to_hex(&self.Dt[i]));
        }
        println!("========End of Public Key=========");
    }

    pub fn is_valid(&self) -> isize {
        for p in &self.Dt {
            let mut bytes = [0u8; 2 * big::MODBYTES + 1];
            p.tobytes(&mut bytes, false);

            let result = ecdh::public_key_validate(&bytes);
            if result != 0 {
                return result;
            }
        }
        0
    }

    pub fn format_full(&self) -> String {
        let mut result = String::new();
        for i in 0..self.Dt.len() {
            result.push_str(&format!("Dt[{}]: {}\n", i, ecp_to_hex(&self.Dt[i])));
        }
        result
    }
}
```

src-tauri/src/kr_peks/trapdoor.rs

```
use super::utils::*;
use mcore::ed25519::big;

pub struct Trapdoor {
    p1: big::BIG,
    p2: big::BIG,
}

impl Trapdoor {
    pub fn new() -> Self {
        Trapdoor {
            p1: big::BIG::new(),
            p2: big::BIG::new(),
        }
    }

    pub fn new_trapdoor(p1: &big::BIG, p2: &big::BIG) -> Self {
        Trapdoor {
            p1: p1.clone(),
            p2: p2.clone(),
        }
    }

    pub fn set_trapdoor(&mut self, p1: big::BIG, p2: big::BIG) {
        self.p1 = p1;
        self.p2 = p2;
    }

    pub fn get_p1(&self) -> &big::BIG {
        &self.p1
    }

    pub fn get_p2(&self) -> &big::BIG {
        &self.p2
    }

    pub fn print(&self) {
        println!();
        println!("========Begin Trapdoor=========");
        println!("p1: {}", big_to_hex(&self.p1));
        println!("p2: {}", big_to_hex(&self.p2));
        println!("========End of Trapdoor=========");
    }

    pub fn format_full(&self) -> String {
        format!("p1: {}\np2: {}", big_to_hex(&self.p1), big_to_hex(&self.p2))
    }
}
```

src-tauri/src/kr_peks/utils.rs

```
use base64::*;
use mcore::ed25519::big;
use mcore::ed25519::ecp;
use mcore::rand::RAND;
use mcore::sha3::{SHA3, SHAKE256};
use rand::rngs::OsRng;
use rand::RngCore;
use serde_json::{json, Value};
use std::fs::File;
use std::io::{self, BufReader, BufWriter};

use super::ciphertext::Ciphertext;
use super::params::Params;
use super::polynomial::Polynomial;
use super::private_key::PrivateKey;
use super::public_key::PublicKey;

pub fn gen_seed() -> RAND {
    let mut raw: [u8; 100] = [0; 100];
    let mut rng = RAND::new();
    rng.clean();
    OsRng.fill_bytes(&mut raw);
    rng.seed(100, &raw);
    rng
}

pub fn hash_to_big(input: &[u8], order: &big::BIG) -> big::BIG {
    let mut hasher = SHA3::new(SHAKE256);
    hasher.process_array(input);
    let mut output = [0u8; big::MODBYTES];
    hasher.shake(&mut output, big::MODBYTES);
    let mut v = big::BIG::frombytes(&output);
    v.rmod(order);
    v
}

pub fn string_to_bytes(input: &str) -> Vec<u8> {
    input.as_bytes().to_vec()
}

pub fn big_to_bytes(b: &big::BIG) -> Vec<u8> {
    let mut bytes = vec![0u8; big::MODBYTES];
    b.tobytes(&mut bytes);
    bytes
}

pub fn bytes_to_big(b: &[u8]) -> big::BIG {
    big::BIG::frombytes(b)
}

pub fn ecp_to_bytes(p: &ecp::ECP) -> Vec<u8> {
    let mut bytes = vec![0u8; 2 * big::MODBYTES + 1];
    p.tobytes(&mut bytes, false);
    bytes
}

pub fn bytes_to_ecp(b: &[u8]) -> ecp::ECP {
    ecp::ECP::frombytes(b)
}

pub fn big_to_hex(b: &big::BIG) -> String {
    big_to_bytes(&b)
        .iter()
        .map(|byte| format!("{:02x}", byte))
        .collect()
}

pub fn ecp_to_hex(p: &ecp::ECP) -> String {
    let bytes = ecp_to_bytes(&p);
    let x_bytes = &bytes[1..big::MODBYTES + 1];
    let y_bytes = &bytes[big::MODBYTES + 1..];

    let x_hex: String = x_bytes.iter().map(|byte| format!("{:02x}", byte)).collect();
    let y_hex: String = y_bytes.iter().map(|byte| format!("{:02x}", byte)).collect();

    format!("({}, {})", x_hex, y_hex)
}

pub fn export_params(params: &Params, file_path: &str) -> io::Result<()> {
    let data = json!({
```

src-tauri/src/kr_peks/utils.rs (continued)

```
        "k": encode(params.get_k().to_ne_bytes()),
        "order": encode(big_to_bytes(params.get_order())),
        "g1": encode(ecp_to_bytes(params.get_g1())),
        "g2": encode(ecp_to_bytes(params.get_g2()))
    });

    let mut writer = BufWriter::new(File::create(file_path)?);
    serde_json::to_writer(&mut writer, &data)?;

    Ok(())
}

pub fn import_params(params: &mut Params, file_path: &str) -> io::Result<()> {
    let reader = BufReader::new(File::open(file_path)?);
    let data: Value = serde_json::from_reader(reader)?;

    let k = usize::from_ne_bytes(
        decode(
            data["k"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64")
        .try_into()
        .expect("Failed to convert to [u8; 8]"),
    );

    let order = bytes_to_big(
        &decode(
            data["order"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64"),
    );

    let g1 = bytes_to_ecp(
        &decode(
            data["g1"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64"),
    );

    let g2 = bytes_to_ecp(
        &decode(
            data["g2"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64"),
    );

    params.set_params(k, order, g1, g2);

    Ok(())
}

pub fn poly_to_string(p: &Polynomial) -> Value {
    let coeff_str: Vec<String> = p
        .get_coeff()
        .iter()
        .map(|c| encode(&big_to_bytes(c)))
        .collect();

    let data = json!({
        "degree": encode(p.get_degree().to_ne_bytes()),
        "order": encode(big_to_bytes(p.get_order())),
        "coeff": coeff_str.join("\n")
    });

    data
}

pub fn string_to_poly(s: &Value) -> Polynomial {
    let degree = usize::from_ne_bytes(
        decode(
            s["degree"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64")
        .try_into()
        .expect("Failed to convert to [u8; 8]"),
```

src-tauri/src/kr_peks/utils.rs (continued)

```
    );

    let order = bytes_to_big(
        &decode(
            s["order"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64"),
    );

    let coeff_str = s["coeff"]
        .as_str()
        .expect("Failed to get a valid string from JSON");
    let mut coeff = Vec::new();
    for line in coeff_str.lines() {
        coeff.push(bytes_to_big(
            &decode(line).expect("Failed to decode base64"),
        ));
    }

    let mut p = Polynomial::new();
    p.set_polynomial(degree, coeff, order);
    p
}

pub fn export_sk(sk: &PrivateKey, file_path: &str) -> io::Result<()> {
    let data = json!({
        "p1": poly_to_string(sk.get_p1()),
        "p2": poly_to_string(sk.get_p2())
    });

    let mut writer = BufWriter::new(File::create(file_path)?);
    serde_json::to_writer(&mut writer, &data)?;

    Ok(())
}

pub fn import_sk(sk: &mut PrivateKey, file_path: &str) -> io::Result<()> {
    let reader = BufReader::new(File::open(file_path)?);
    let data: Value = serde_json::from_reader(reader)?;

    let p1 = string_to_poly(&data["p1"]);
    let p2 = string_to_poly(&data["p2"]);

    sk.set_private_key(p1, p2);

    Ok(())
}

pub fn export_pk(pk: &PublicKey, file_path: &str) -> io::Result<()> {
    if pk.is_valid() != 0 {
        return Err(io::Error::new(
            io::ErrorKind::InvalidData,
            "Failed to export public key: Public key is invalid!",
        ));
    }

    let base64_strings: Vec<String> = pk
        .get_Dt()
        .iter()
        .map(|ecp| encode(&ecp_to_bytes(ecp)))
        .collect();

    let data = json!({
        "Dt": base64_strings.join("\n")
    });

    let mut writer = BufWriter::new(File::create(file_path)?);
    serde_json::to_writer(&mut writer, &data)?;

    Ok(())
}

pub fn import_pk(pk: &mut PublicKey, file_path: &str) -> io::Result<()> {
    let reader = BufReader::new(File::open(file_path)?);
    let data: Value = serde_json::from_reader(reader)?;

    let Dt_str = data["Dt"]
        .as_str()
        .expect("Failed to get a valid string from JSON");

    let mut Dt = Vec::new();
    for line in Dt_str.lines() {
        Dt.push(bytes_to_ecp(
```

src-tauri/src/kr_peks/utils.rs (continued)

```
            &decode(line).expect("Failed to decode base64"),
        ));
    }

    pk.set_public_key(Dt);

    Ok(())
}

pub fn export_c(c: &Ciphertext, file_path: &str) -> io::Result<()> {
    let data = json!({
        "u1": encode(ecp_to_bytes(c.get_u1())),
        "u2": encode(ecp_to_bytes(c.get_u2())),
        "sw": encode(ecp_to_bytes(c.get_sw()))
    });

    let mut writer = BufWriter::new(File::create(file_path)?);
    serde_json::to_writer(&mut writer, &data)?;

    Ok(())
}

pub fn import_c(c: &mut Ciphertext, file_path: &str) -> io::Result<()> {
    let reader = BufReader::new(File::open(file_path)?);
    let data: Value = serde_json::from_reader(reader)?;

    let u1 = bytes_to_ecp(
        &decode(
            data["u1"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64"),
    );
    let u2 = bytes_to_ecp(
        &decode(
            data["u2"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64"),
    );
    let sw = bytes_to_ecp(
        &decode(
            data["sw"]
                .as_str()
                .expect("Failed to get a valid string from JSON"),
        )
        .expect("Failed to decode base64"),
    );

    c.set_ciphertext(u1, u2, sw);

    Ok(())
}
```

src-tauri/src/lib.rs

```
// Learn more about Tauri commands at https://tauri.app/develop/calling-rust/
pub mod kr_ibe;
pub mod kr_ibi;
pub mod kr_paeks;
pub mod kr_peks;
pub mod system;

#[tauri::command]
fn greet(name: &str) -> String {
    format!("Hello, {}! You've been greeted from Rust!", name)
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_opener::init())
        .invoke_handler(tauri::generate_handler![greet])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

src-tauri/src/main.rs

```
// Prevents additional console window on Windows in release, DO NOT REMOVE!!
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

mod kr_ibe;
mod kr_ibi;
mod kr_paeks;
mod kr_peks;

use kr_ibe::api::*;
use kr_ibi::api::*;
use kr_paeks::api::*;
use kr_peks::api::*;

mod system;

use system::{auth, download, search, setup, upload, user};

#[tauri::command]
fn setup_all(k: usize) {
    setup::setup_all(k);
}

#[tauri::command]
fn register(id: String) -> Result<String, String> {
    user::register_user(&id)
}

#[tauri::command]
fn upload_file(
    sender: String,
    receiver: String,
    msg: String,
    keyword: String,
    scheme: String,
    payload_type: String,
    file_name: Option<String>,
    mime_type: Option<String>,
    content_base64: Option<String>,
) -> Result<(), String> {
    upload::upload(
        &sender,
        &receiver,
        &msg,
        &keyword,
        &scheme,
        &payload_type,
        file_name,
        mime_type,
        content_base64,
    )
}

#[tauri::command]
fn search_keyword(user: String, keyword: String, scheme: String) -> Result<Vec<usize>, String> {
    search::search(&user, &keyword, &scheme)
}

#[tauri::command]
fn download_file(user: String, index: usize) -> Result<system::state::SharedPayload, String> {
    download::download(&user, index)
}

#[tauri::command]
fn login_start(id: String) -> Result<(String, String), String> {
    auth::login_start(&id)
}

#[tauri::command]
fn login_respond(id: String) -> Result<(String, String), String> {
    auth::login_respond(&id)
}

#[tauri::command]
fn login_verify(id: String, s1: String, s2: String) -> Result<bool, String> {
    auth::login_verify(&id, &s1, &s2)
}

fn main() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![
```

src-tauri/src/main.rs (continued)

```
            // KR-IBE
            kr_ibe_setup,
            kr_ibe_extract,
            kr_ibe_encrypt,
            kr_ibe_decrypt,
            // KR-IBI
            kr_ibi_setup,
            kr_ibi_extract,
            kr_ibi_sign,
            kr_ibi_verify,
            // KR-PEKS
            kr_peks_setup,
            kr_peks_keygen,
            kr_peks_encrypt,
            kr_peks_trapdoor,
            kr_peks_test,
            // KR-PAEKS
            kr_paeks_setup,
            kr_paeks_keygen,
            kr_paeks_encrypt,
            kr_paeks_trapdoor,
            kr_paeks_test,
            // System
            setup_all,
            register,
            upload_file,
            search_keyword,
            download_file,
            login_start,
            login_respond,
            login_verify
        ])
        .run(tauri::generate_context!())
        .expect("error while running tauri app");
}
```

src-tauri/src/system/auth.rs

```
use crate::kr_ibi::main as kribi_core;
use crate::kr_peks::{self, utils::*};
use crate::system::state::LoginSession;
use crate::system::state::APP_STATE;
use crate::system::utils::id_to_bytes;

pub fn login_start(id: &str) -> Result<(String, String), String> {
    let mut state = APP_STATE.lock().unwrap();

    if !state.users.contains_key(id) {
        return Err("User is not registered. Please register first.".to_string());
    }

    if state.ibi_params.is_none() {
        panic!("IBI params not initialised. Call setup_all() first.");
    }

    let params = state.ibi_params.as_ref().unwrap();

    let mut rng = kribi_core::gen_seed();

    let (g_r, r) = kribi_core::commit(params, &mut rng);
    let (c1, c2) = kribi_core::challenge(params, &mut rng);

    println!("Login start for ID: {}", id);

    state
        .login_sessions
        .insert(id.to_string(), LoginSession { g_r, c1, c2, r });

    Ok((kribi_core::big_to_hex(&c1), kribi_core::big_to_hex(&c2)))
}

pub fn login_respond(id: &str) -> Result<(String, String), String> {
    let mut state = APP_STATE.lock().unwrap();

    if !state.users.contains_key(id) {
        return Err("User is not registered. Please register first.".to_string());
    }

    let params = state.ibi_params.as_ref().unwrap();
    let session = state.login_sessions.get(id).unwrap();

    let mut id_bytes = id_to_bytes(id);

    if id_bytes.len() < 32 {
        id_bytes.resize(32, 0);
    }

    let (f1, f2) = kribi_core::extract(params, &id_bytes);

    let (s1, s2) = kribi_core::respond(
        &session.r,
        &(session.c1, session.c2),
        &(f1, f2),
        params.get_order(),
    );

    Ok((kribi_core::big_to_hex(&s1), kribi_core::big_to_hex(&s2)))
}

pub fn login_verify(id: &str, s1_hex: &str, s2_hex: &str) -> Result<bool, String> {
    let mut state = APP_STATE.lock().unwrap();

    if !state.users.contains_key(id) {
        return Err("User is not registered. Please register first.".to_string());
    }

    let params = state.ibi_params.as_ref().unwrap();
    let session = state.login_sessions.get(id).unwrap();

    let s1 = kribi_core::hex_to_big(s1_hex);
    let s2 = kribi_core::hex_to_big(s2_hex);

    let mut id_bytes = id_to_bytes(id);

    let valid = kribi_core::verify(
        params,
        &session.g_r,
        &(s1, s2),
```

src-tauri/src/system/auth.rs (continued)

```
        &(session.c1, session.c2),
        &id_bytes,
    );

    if valid {
        state.active_sessions.insert(id.to_string(), true);
    }

    Ok(valid)
}
```

src-tauri/src/system/benchmark.rs

```
use std::collections::hash_map::DefaultHasher;
use std::fmt::{self, Display};
use std::fs::{self, File};
use std::hash::{Hash, Hasher};
use std::io::{BufRead, BufReader, BufWriter, Write};
use std::panic::{catch_unwind, AssertUnwindSafe};
use std::path::{Path, PathBuf};
use std::thread;
use std::time::Instant;

use rayon::prelude::*;
use rayon::ThreadPoolBuilder;

use crate::kr_ibe::{
    ciphertext::Ciphertext as IbeCiphertext, main as kribe_core, params::Params as IbeParams,
    plaintext::Plaintext, private_key::PrivateKey as IbePrivateKey,
};
use crate::kr_ibi::{main as kribi_core, params::Params as IbiParams};
use crate::kr_paeks::{
    ciphertext::Ciphertext as PaeksCiphertext, main as krpaeks_core, params::Params as PaeksParams,
    private_key::PrivateKey as PaeksPrivateKey, public_key::PublicKey as PaeksPublicKey,
};
use crate::kr_peks::{
    ciphertext::Ciphertext as PeksCiphertext, main as krpeks_core, params::Params as PeksParams,
    private_key::PrivateKey as PeksPrivateKey, public_key::PublicKey as PeksPublicKey,
};
use crate::system::utils::keyword_hash;
use mcore::ed25519::{big, ecp, rom};
use mcore::sha3::{SHA3, SHAKE256};

const DEFAULT_K: usize = 20;
const WRONG_KEYWORD: &str = "__wrong_keyword__";

type BenchmarkResult<T> = Result<T, Box<dyn std::error::Error + Send + Sync>>;

#[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum BenchmarkScheme {
    Peks,
    Paeks,
}

impl BenchmarkScheme {
    fn all() -> [Self; 2] {
        [Self::Peks, Self::Paeks]
    }

    fn opposite(self) -> Self {
        match self {
            Self::Peks => Self::Paeks,
            Self::Paeks => Self::Peks,
        }
    }
}

impl Display for BenchmarkScheme {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::Peks => write!(f, "KR-PEKS"),
            Self::Paeks => write!(f, "KR-PAEKS"),
        }
    }
}

#[derive(Clone)]
pub struct BenchmarkConfig {
    pub dataset_sizes: Vec<usize>,
    pub authorised_user_counts: Vec<usize>,
    pub runs_per_setting: usize,
    pub security_threshold: usize,
    pub schemes: Vec<BenchmarkScheme>,
    pub detected_cpu_threads: usize,
    pub benchmark_workers: usize,
    pub debug_raw: bool,
}

impl Default for BenchmarkConfig {
    fn default() -> Self {
        let detected_cpu_threads = detected_cpu_threads();
        let benchmark_workers = default_worker_count(detected_cpu_threads);
```

src-tauri/src/system/benchmark.rs (continued)

```
        Self {
            dataset_sizes: vec![100, 500, 1000, 5000, 10000],
            authorised_user_counts: vec![1, 5, 10, 20],
            runs_per_setting: 100,
            security_threshold: DEFAULT_K,
            schemes: BenchmarkScheme::all().to_vec(),
            detected_cpu_threads,
            benchmark_workers,
            debug_raw: false,
        }
    }
}

impl BenchmarkConfig {
    pub fn from_args<I>(args: I) -> Result<Self, String>
    where
        I: IntoIterator<Item = String>,
    {
        let mut config = Self::default();
        let mut args = args.into_iter();

        while let Some(arg) = args.next() {
            match arg.as_str() {
                "--threads" => {
                    let value = args.next().ok_or("--threads requires a positive integer")?;
                    config.benchmark_workers = value
                        .parse::<usize>()
                        .map_err(|_| "--threads requires a positive integer".to_string())?;
                    if config.benchmark_workers == 0 {
                        return Err("--threads must be at least 1".to_string());
                    }
                }
                "--debug-raw" => {
                    config.debug_raw = true;
                }
                "--help" | "-h" => {
                    return Err("Usage: [--threads N] [--debug-raw]".to_string());
                }
                other => {
                    return Err(format!("unknown argument '{other}'"));
                }
            }
        }

        Ok(config)
    }
}

fn detected_cpu_threads() -> usize {
    thread::available_parallelism()
        .map(|threads| threads.get())
        .unwrap_or(1)
}

fn default_worker_count(detected_cpu_threads: usize) -> usize {
    detected_cpu_threads.saturating_sub(2).max(1)
}

#[derive(Clone, Debug)]
pub struct EmailRecord {
    pub sender: String,
    pub receiver: String,
    pub subject: String,
    pub body: String,
    pub keyword: String,
}

#[derive(Clone, Debug)]
pub struct BenchmarkRawResult {
    pub scheme: BenchmarkScheme,
    pub dataset_size: usize,
    pub authorised_users: usize,
    pub run: usize,
    pub setup_ms: f64,
    pub registration_ms: f64,
    pub login_ms: f64,
    pub ibe_encrypt_ms: f64,
    pub index_generation_ms: f64,
    pub search_ms: f64,
    pub ibe_decrypt_ms: f64,
    pub total_upload_ms: f64,
    pub total_retrieval_ms: f64,
    pub payload_ciphertext_size_bytes: usize,
    pub search_index_size_bytes: usize,
    pub successful_searches: usize,
```

src-tauri/src/system/benchmark.rs (continued)

```
    pub successful_decryptions: usize,
    pub wrong_keyword_rejected: bool,
    pub raw_kr_paeks_wrong_keyword_rejected: bool,
    pub wrong_scheme_rejected: bool,
    pub unauthorised_decryption_failed: bool,
    pub authenticated_login_passed: bool,
    pub unauthenticated_rejected: bool,
    pub debug: BenchmarkDebugCheck,
}

#[derive(Clone, Debug)]
pub struct BenchmarkDebugCheck {
    pub sender_id: String,
    pub receiver_id: String,
    pub run: usize,
    pub dataset_size: usize,
    pub thread_id: String,
    pub ibe_params_hash: String,
    pub peks_params_hash: String,
    pub paeks_params_hash: String,
    pub ciphertext_and_decrypt_use_same_params: bool,
    pub search_index_and_trapdoor_use_same_params: bool,
    pub authorised_v_id_matched: bool,
    pub unauthorised_v_id_matched: bool,
    pub stored_keyword: String,
    pub correct_search_keyword: String,
    pub wrong_search_keyword: String,
    pub stored_keyword_hash: String,
    pub correct_keyword_hash: String,
    pub wrong_keyword_hash: String,
    pub correct_scheme: BenchmarkScheme,
    pub wrong_scheme: BenchmarkScheme,
    pub raw_kr_paeks_wrong_keyword_test_result: bool,
    pub correct_keyword_results: usize,
    pub wrong_keyword_results: usize,
    pub wrong_scheme_results: usize,
}

#[derive(Clone, Debug)]
pub struct BenchmarkSummaryResult {
    pub scheme: BenchmarkScheme,
    pub dataset_size: usize,
    pub authorised_users: usize,
    pub runs: usize,
    pub successful_runs: usize,
    pub authenticated_login_passed: usize,
    pub unauthenticated_rejected: usize,
    pub correct_keyword_search_passed: usize,
    pub wrong_keyword_rejected: usize,
    pub raw_kr_paeks_wrong_keyword_rejected: usize,
    pub wrong_scheme_rejected: usize,
    pub authorised_decryption_passed: usize,
    pub unauthorised_decryption_failed: usize,
    pub setup_ms: f64,
    pub registration_ms: f64,
    pub login_ms: f64,
    pub ibe_encrypt_ms: f64,
    pub index_generation_ms: f64,
    pub search_ms: f64,
    pub ibe_decrypt_ms: f64,
    pub total_upload_ms: f64,
    pub total_retrieval_ms: f64,
    pub payload_ciphertext_size_bytes: f64,
    pub search_index_size_bytes: f64,
    pub successful_searches: f64,
    pub successful_decryptions: f64,
}

enum SearchIndex {
    Peks(PeksCiphertext),
    Paeks(PaeksCiphertext),
}

struct StoredBenchmarkRecord {
    ct: IbeCiphertext,
    search_scheme: BenchmarkScheme,
    sender: String,
    owner: String,
    search_index: SearchIndex,
    keyword_hash: String,
}

struct BenchmarkState {
    ibe_params: IbeParams,
    ibi_params: IbiParams,
```

src-tauri/src/system/benchmark.rs (continued)

```
    peks_params: PeksParams,
    peks_pk: PeksPublicKey,
    peks_sk: PeksPrivateKey,
    paeks_params: PaeksParams,
    sender_paeks_pk: PaeksPublicKey,
    sender_paeks_sk: PaeksPrivateKey,
    receiver_paeks_pk: PaeksPublicKey,
    receiver_paeks_sk: PaeksPrivateKey,
    receiver_ibe_sk: IbePrivateKey,
}

struct DecryptTrace {
    scheme: BenchmarkScheme,
    dataset_size: usize,
    authorised_users: usize,
    run: usize,
    sender_id: String,
    receiver_id: String,
    attempted_user_id: String,
    thread_id: String,
    params_hash: String,
    ciphertext_params_hash: String,
    decrypt_params_hash: String,
    same_params: bool,
    authorised_attempt: bool,
}

#[derive(Clone, Copy)]
struct BenchmarkJob {
    scheme: BenchmarkScheme,
    dataset_size: usize,
    authorised_users: usize,
    run: usize,
}

pub fn run_enron_benchmark(config: BenchmarkConfig) -> BenchmarkResult<()> {
    let dataset_path = find_dataset_path()?;
    let max_records = config.dataset_sizes.iter().copied().max().unwrap_or(0);
    let records = load_enron_records(&dataset_path, max_records)?;

    if records.len() < max_records {
        return Err(format!(
            "Only {} valid email records were loaded, but {} are required.",
            records.len(),
            max_records
        )
        .into());
    }

    print_header(&dataset_path, &config);

    let jobs = build_jobs(&config);
    let pool = ThreadPoolBuilder::new()
        .num_threads(config.benchmark_workers)
        .build()?;

    let raw_results_result: BenchmarkResult<Vec<BenchmarkRawResult>> = pool.install(|| {
        jobs.par_iter()
            .map(|job| {
                run_one(
                    job.scheme,
                    job.dataset_size,
                    job.authorised_users,
                    job.run,
                    &records[..job.dataset_size],
                    &config,
                )
            })
            .collect()
    });

    let mut raw_results = raw_results_result?;
    raw_results.sort_by_key(|result| {
        (
            scheme_order(result.scheme),
            result.dataset_size,
            result.authorised_users,
            result.run,
        )
    });

    let summaries = summarise_all(&raw_results);
    for summary in &summaries {
        print_setting_report(summary, config.debug_raw);
        if config.debug_raw {
```

src-tauri/src/system/benchmark.rs (continued)

```
            if let Some(debug_result) = raw_results.iter().find(|result| {
                result.scheme == summary.scheme
                    && result.dataset_size == summary.dataset_size
                    && result.authorised_users == summary.authorised_users
                    && result.run == 1
            }) {
                print_debug_check(debug_result);
            }
        }
    }

    fs::create_dir_all("benchmark_results")?;
    write_raw_csv("benchmark_results/enron_raw_results.csv", &raw_results)?;
    write_summary_csv("benchmark_results/enron_summary_results.csv", &summaries)?;
    write_comparison_csv(
        "benchmark_results/enron_peks_vs_paeks_comparison.csv",
        &summaries,
    )?;

    print_summary_tables(&summaries);
    Ok(())
}

fn build_jobs(config: &BenchmarkConfig) -> Vec<BenchmarkJob> {
    let mut jobs = Vec::new();
    for scheme in config.schemes.iter().copied() {
        for dataset_size in &config.dataset_sizes {
            for authorised_users in &config.authorised_user_counts {
                for run in 1..=config.runs_per_setting {
                    jobs.push(BenchmarkJob {
                        scheme,
                        dataset_size: *dataset_size,
                        authorised_users: *authorised_users,
                        run,
                    });
                }
            }
        }
    }

    jobs
}

fn scheme_order(scheme: BenchmarkScheme) -> usize {
    match scheme {
        BenchmarkScheme::Peks => 0,
        BenchmarkScheme::Paeks => 1,
    }
}

fn run_one(
    scheme: BenchmarkScheme,
    dataset_size: usize,
    authorised_users: usize,
    run: usize,
    records: &[EmailRecord],
    config: &BenchmarkConfig,
) -> BenchmarkResult<BenchmarkRawResult> {
    let start_setup = Instant::now();
    let mut state = setup_state(config.security_threshold)?;
    let setup_ms = elapsed_ms(start_setup);

    let sender = format!("sender_{run}@benchmark.local");
    let receiver = format!("receiver_0_{run}@benchmark.local");
    let unauthorised = format!("unauthorised_{run}@benchmark.local");
    let thread_id = format!("{:?}", thread::current().id());
    let ibe_params_hash = params_hash_from_text(&state.ibe_params.format_full());
    let peks_params_hash = params_hash_from_text(&state.peks_params.format_full());
    let paeks_params_hash = params_hash_from_text(&state.paeks_params.format_full());

    let start_registration = Instant::now();
    let mut receiver_keys = Vec::with_capacity(authorised_users);
    for index in 0..authorised_users {
        let id = format!("receiver_{index}_{run}@benchmark.local");
        receiver_keys.push(extract_ibe_key(&state.ibe_params, &id));
    }
    state.receiver_ibe_sk = receiver_keys
        .first()
        .cloned()
        .ok_or("At least one authorised user is required")?;
    let unauthorised_ibe_sk = extract_ibe_key(&state.ibe_params, &unauthorised);
    let registration_ms = elapsed_ms(start_registration);

    let start_login = Instant::now();
    let authenticated_login_passed = authenticate_identity(&state.ibi_params, &receiver);
```

src-tauri/src/system/benchmark.rs (continued)

```
    let unauthenticated_rejected = !is_authenticated(false);
    let login_ms = elapsed_ms(start_login);

    let selected = &records[(run - 1) % dataset_size];
    let selected_index = (run - 1) % dataset_size;
    let payload = selected.body.as_bytes().to_vec();
    let receiver_bytes = receiver.as_bytes().to_vec();

    let start_encrypt = Instant::now();
    let mut ibe_ct = IbeCiphertext::new();
    kribe_core::encryption(&state.ibe_params, &mut ibe_ct, &receiver_bytes, &payload);
    let ibe_encrypt_ms = elapsed_ms(start_encrypt);

    let start_index = Instant::now();
    let stored_records = build_stored_records(
        scheme,
        records,
        &state,
        &sender,
        &receiver,
        selected_index,
        &ibe_ct,
    );
    let index_generation_ms = elapsed_ms(start_index);
    let search_index_size_bytes = stored_records
        .first()
        .map(|record| search_index_size(&record.search_index))
        .unwrap_or_default();

    let start_search = Instant::now();
    let correct_results = app_search(
        &receiver,
        scheme,
        &selected.keyword,
        &stored_records,
        &state,
    );
    let successful_searches = correct_results.len();
    let search_ms = elapsed_ms(start_search);

    let wrong_keyword_results =
        app_search(&receiver, scheme, WRONG_KEYWORD, &stored_records, &state).len();
    let wrong_scheme_results = app_search(
        &receiver,
        scheme.opposite(),
        &selected.keyword,
        &stored_records,
        &state,
    )
    .len();
    let raw_kr_paeks_wrong_keyword_results = if config.debug_raw && scheme == BenchmarkScheme::Paeks
    {
        raw_search_indexes(scheme, WRONG_KEYWORD, &stored_records, &state)
    } else {
        0
    };
    let wrong_keyword_rejected = wrong_keyword_results == 0;
    let raw_kr_paeks_wrong_keyword_rejected = !config.debug_raw
        || scheme != BenchmarkScheme::Paeks
        || raw_kr_paeks_wrong_keyword_results == 0;
    let wrong_scheme_rejected = wrong_scheme_results == 0;

    let start_decrypt = Instant::now();
    let ciphertext_and_decrypt_use_same_params = true;
    let authorised_v_id_matched = ibe_v_id_matches(
        &state.ibe_params,
        &state.receiver_ibe_sk,
        &stored_records[selected_index].ct,
    );
    let authorised_trace = DecryptTrace {
        scheme,
        dataset_size,
        authorised_users,
        run,
        sender_id: sender.clone(),
        receiver_id: receiver.clone(),
        attempted_user_id: receiver.clone(),
        thread_id: thread_id.clone(),
        params_hash: ibe_params_hash.clone(),
        ciphertext_params_hash: ibe_params_hash.clone(),
        decrypt_params_hash: ibe_params_hash.clone(),
        same_params: ciphertext_and_decrypt_use_same_params,
        authorised_attempt: true,
    };
    let authorised_plaintext = if correct_results.contains(&selected_index) {
```

src-tauri/src/system/benchmark.rs (continued)

```
        decrypt_payload_checked(
            &state.ibe_params,
            &state.receiver_ibe_sk,
            &stored_records[selected_index].ct,
            &authorised_trace,
        )
    } else {
        None
    };
    let ibe_decrypt_ms = elapsed_ms(start_decrypt);

    let successful_decryptions =
        usize::from(authorised_plaintext.as_deref() == Some(&selected.body));

    let unauthorised_v_id_matched =
        ibe_v_id_matches(&state.ibe_params, &unauthorised_ibe_sk, &ibe_ct);
    let unauthorised_trace = DecryptTrace {
        scheme,
        dataset_size,
        authorised_users,
        run,
        sender_id: sender.clone(),
        receiver_id: receiver.clone(),
        attempted_user_id: unauthorised.clone(),
        thread_id: thread_id.clone(),
        params_hash: ibe_params_hash.clone(),
        ciphertext_params_hash: ibe_params_hash.clone(),
        decrypt_params_hash: ibe_params_hash.clone(),
        same_params: ciphertext_and_decrypt_use_same_params,
        authorised_attempt: false,
    };
    let unauthorised_plaintext = decrypt_payload_checked(
        &state.ibe_params,
        &unauthorised_ibe_sk,
        &ibe_ct,
        &unauthorised_trace,
    );
    let unauthorised_decryption_failed = unauthorised_plaintext.as_deref() != Some(&selected.body);

    Ok(BenchmarkRawResult {
        scheme,
        dataset_size,
        authorised_users,
        run,
        setup_ms,
        registration_ms,
        login_ms,
        ibe_encrypt_ms,
        index_generation_ms,
        search_ms,
        ibe_decrypt_ms,
        total_upload_ms: ibe_encrypt_ms + index_generation_ms,
        total_retrieval_ms: login_ms + search_ms + ibe_decrypt_ms,
        payload_ciphertext_size_bytes: ibe_ciphertext_size(&ibe_ct),
        search_index_size_bytes,
        successful_searches,
        successful_decryptions,
        wrong_keyword_rejected,
        raw_kr_paeks_wrong_keyword_rejected,
        wrong_scheme_rejected,
        unauthorised_decryption_failed,
        authenticated_login_passed,
        unauthenticated_rejected,
        debug: BenchmarkDebugCheck {
            sender_id: sender,
            receiver_id: receiver,
            run,
            dataset_size,
            thread_id,
            ibe_params_hash,
            peks_params_hash,
            paeks_params_hash,
            ciphertext_and_decrypt_use_same_params,
            search_index_and_trapdoor_use_same_params: true,
            authorised_v_id_matched,
            unauthorised_v_id_matched,
            stored_keyword: selected.keyword.clone(),
            correct_search_keyword: selected.keyword.clone(),
            wrong_search_keyword: WRONG_KEYWORD.to_string(),
            stored_keyword_hash: stored_records[selected_index].keyword_hash.clone(),
            correct_keyword_hash: keyword_hash(&selected.keyword),
            wrong_keyword_hash: keyword_hash(WRONG_KEYWORD),
            correct_scheme: scheme,
            wrong_scheme: scheme.opposite(),
            raw_kr_paeks_wrong_keyword_test_result: raw_kr_paeks_wrong_keyword_results > 0,
```

src-tauri/src/system/benchmark.rs (continued)

```
            correct_keyword_results: successful_searches,
            wrong_keyword_results,
            wrong_scheme_results,
        },
    })
}

fn setup_state(k: usize) -> BenchmarkResult<BenchmarkState> {
    let mut ibe_params = IbeParams::new();
    kribe_core::setup(&mut ibe_params, k);

    let mut ibi_params = IbiParams::new();
    ibi_setup_silent(&mut ibi_params, k);

    let mut peks_params = PeksParams::new();
    krpeks_core::setup(&mut peks_params, k);
    let mut peks_pk = PeksPublicKey::new();
    let mut peks_sk = PeksPrivateKey::new();
    krpeks_core::keygen(&peks_params, &mut peks_pk, &mut peks_sk);

    let mut paeks_params = PaeksParams::new();
    krpaeks_core::setup(&mut paeks_params, k);
    let (sender_paeks_pk, sender_paeks_sk) = paeks_keypair(&paeks_params);
    let (receiver_paeks_pk, receiver_paeks_sk) = paeks_keypair(&paeks_params);

    let receiver_ibe_sk = extract_ibe_key(&ibe_params, "receiver_0@benchmark.local");

    Ok(BenchmarkState {
        ibe_params,
        ibi_params,
        peks_params,
        peks_pk,
        peks_sk,
        paeks_params,
        sender_paeks_pk,
        sender_paeks_sk,
        receiver_paeks_pk,
        receiver_paeks_sk,
        receiver_ibe_sk,
    })
}

fn paeks_keypair(params: &PaeksParams) -> (PaeksPublicKey, PaeksPrivateKey) {
    let mut pk = PaeksPublicKey::new();
    let mut sk = PaeksPrivateKey::new();
    krpaeks_core::keygen(params, &mut pk, &mut sk);
    (pk, sk)
}

fn extract_ibe_key(params: &IbeParams, id: &str) -> IbePrivateKey {
    let mut sk = IbePrivateKey::new();
    kribe_core::extract(params, &mut sk, &id.as_bytes().to_vec());
    sk
}

fn authenticate_identity(params: &IbiParams, id: &str) -> bool {
    let id_bytes = id.as_bytes().to_vec();
    let (f1, f2) = ibi_extract_silent(params, &id_bytes);
    let mut rng = kribi_core::gen_seed();
    let (g_r, r) = kribi_core::commit(params, &mut rng);
    let challenge = kribi_core::challenge(params, &mut rng);
    let response = kribi_core::respond(&r, &challenge, &(f1, f2), params.get_order());
    ibi_verify_silent(params, &g_r, &response, &challenge, &id_bytes)
}

fn ibi_setup_silent(params: &mut IbiParams, k: usize) {
    let order = big::BIG::new_ints(&rom::CURVE_ORDER);
    let g = ecp::ECP::generator();
    let mut rng = kribi_core::gen_seed();
    let t = k + 1;

    let mut fx1 = vec![big::BIG::new(); t];
    let mut fx2 = vec![big::BIG::new(); t];

    for i in 0..t {
        fx1[i] = big::BIG::randomnum(&order, &mut rng);
        fx2[i] = big::BIG::randomnum(&order, &mut rng);
    }

    let mut dt1 = vec![ecp::ECP::new(); t];
    let mut dt2 = vec![ecp::ECP::new(); t];

    for i in 0..t {
        dt1[i] = g.mul(&fx1[i]);
        dt2[i] = g.mul(&fx2[i]);
```

src-tauri/src/system/benchmark.rs (continued)

```
    }

    params.set_params(k, order, g, dt1, dt2, fx1, fx2);
}

fn ibi_extract_silent(params: &IbiParams, id: &[u8]) -> (big::BIG, big::BIG) {
    let k = params.get_k() + 1;
    let order = params.get_order();
    let d1 = params.get_msk1();
    let d2 = params.get_msk2();

    let x = kribi_core::hash_to_big(id);
    let mut f1 = big::BIG::new_int(0);
    let mut f2 = big::BIG::new_int(0);

    for i in 0..k {
        let mut x_for_pow = x.clone();
        let exponent = big::BIG::new_int(i as isize);
        let xpowi = big::BIG::powmod(&mut x_for_pow, &exponent, order);

        let temp1 = big::BIG::modmul(&d1[i], &xpowi, order);
        f1.add(&temp1);
        f1.rmod(order);

        let temp2 = big::BIG::modmul(&d2[i], &xpowi, order);
        f2.add(&temp2);
        f2.rmod(order);
    }

    (f1, f2)
}

fn ibi_verify_silent(
    params: &IbiParams,
    g_r: &(ecp::ECP, ecp::ECP),
    response: &(big::BIG, big::BIG),
    challenge: &(big::BIG, big::BIG),
    id: &[u8],
) -> bool {
    let x = kribi_core::hash_to_big(id);

    let f_id_point1 = {
        let mut sum = ecp::ECP::new();
        for i in 0..params.get_k() + 1 {
            let mut x_for_pow = x.clone();
            let exponent = big::BIG::new_int(i as isize);
            let xpowi = big::BIG::powmod(&mut x_for_pow, &exponent, params.get_order());
            let temp = params.get_Dt1()[i].mul(&xpowi);
            sum.add(&temp);
        }
        sum
    };

    let f_id_point2 = {
        let mut sum = ecp::ECP::new();
        for i in 0..params.get_k() + 1 {
            let mut x_for_pow = x.clone();
            let exponent = big::BIG::new_int(i as isize);
            let xpowi = big::BIG::powmod(&mut x_for_pow, &exponent, params.get_order());
            let temp = params.get_Dt2()[i].mul(&xpowi);
            sum.add(&temp);
        }
        sum
    };

    let mut g_r_f_id_c1 = f_id_point1.mul(&challenge.0);
    g_r_f_id_c1.add(&g_r.0);

    let mut g_r_f_id_c2 = f_id_point2.mul(&challenge.1);
    g_r_f_id_c2.add(&g_r.1);

    let valid_1 = g_r_f_id_c1.equals(&ecp::ECP::generator().mul(&response.0));
    let valid_2 = g_r_f_id_c2.equals(&ecp::ECP::generator().mul(&response.1));

    valid_1 && valid_2
}

fn is_authenticated(active_session: bool) -> bool {
    active_session
}

fn build_stored_records(
    scheme: BenchmarkScheme,
    records: &[EmailRecord],
    state: &BenchmarkState,
```

src-tauri/src/system/benchmark.rs (continued)

```
    sender: &str,
    owner: &str,
    selected_index: usize,
    selected_ct: &IbeCiphertext,
) -> Vec<StoredBenchmarkRecord> {
    records
        .iter()
        .enumerate()
        .filter_map(|(index, record)| {
            let search_index = match scheme {
                BenchmarkScheme::Peks => krpeks_core::peks(
                    &state.peks_params,
                    &state.peks_pk,
                    &record.keyword.as_bytes().to_vec(),
                )
                .map(SearchIndex::Peks)?,
                BenchmarkScheme::Paeks => {
                    let keyword_big = krpaeks_core::hash_to_big(&record.keyword);
                    SearchIndex::Paeks(krpaeks_core::encrypt(
                        &state.paeks_params,
                        &state.receiver_paeks_pk,
                        &state.sender_paeks_sk,
                        &keyword_big,
                    ))
                }
            };

            Some(StoredBenchmarkRecord {
                // Search correctness does not need every payload body decrypted. The selected
                // record carries the real upload ciphertext used by the retrieval check.
                ct: if index == selected_index {
                    selected_ct.clone()
                } else {
                    IbeCiphertext::new()
                },
                search_scheme: scheme,
                sender: sender.to_string(),
                owner: owner.to_string(),
                search_index,
                keyword_hash: keyword_hash(&record.keyword),
            })
        })
        .collect()
}

fn app_search(
    user: &str,
    requested_scheme: BenchmarkScheme,
    keyword: &str,
    records: &[StoredBenchmarkRecord],
    state: &BenchmarkState,
) -> Vec<usize> {
    let search_hash = keyword_hash(keyword);
    let mut results = Vec::new();

    match requested_scheme {
        BenchmarkScheme::Peks => {
            let keyword_bytes = keyword.as_bytes().to_vec();
            let trapdoor =
                krpeks_core::trapdoor(&state.peks_params, &state.peks_sk, &keyword_bytes);

            for (index, record) in records.iter().enumerate() {
                if record.owner != user {
                    continue;
                }

                if record.search_scheme != requested_scheme {
                    continue;
                }

                if record.keyword_hash != search_hash {
                    continue;
                }

                if let SearchIndex::Peks(peks_index) = &record.search_index {
                    if krpeks_core::test(peks_index, &trapdoor) {
                        results.push(index);
                    }
                }
            }
        }
        BenchmarkScheme::Paeks => {
            let keyword_big = krpaeks_core::hash_to_big(keyword);

            for (index, record) in records.iter().enumerate() {
```

src-tauri/src/system/benchmark.rs (continued)

```
                if record.owner != user {
                    continue;
                }

                if record.search_scheme != requested_scheme {
                    continue;
                }

                if record.keyword_hash != search_hash {
                    continue;
                }

                if let SearchIndex::Paeks(paeks_index) = &record.search_index {
                    // All benchmark uploads in one run use the same sender keypair. The sender
                    // field is still stored so this mirrors the application record shape.
                    let _sender = &record.sender;
                    let trapdoor = krpaeks_core::trapdoor(
                        &state.paeks_params,
                        &state.sender_paeks_pk,
                        &state.receiver_paeks_sk,
                        &keyword_big,
                    );

                    if krpaeks_core::test(paeks_index, &trapdoor) {
                        results.push(index);
                    }
                }
            }
        }
    }

    results
}

fn raw_search_indexes(
    scheme: BenchmarkScheme,
    keyword: &str,
    records: &[StoredBenchmarkRecord],
    state: &BenchmarkState,
) -> usize {
    match scheme {
        BenchmarkScheme::Peks => {
            let keyword_bytes = keyword.as_bytes().to_vec();
            let trapdoor =
                krpeks_core::trapdoor(&state.peks_params, &state.peks_sk, &keyword_bytes);
            records
                .iter()
                .filter_map(|record| match &record.search_index {
                    SearchIndex::Peks(ct) => Some(ct),
                    SearchIndex::Paeks(_) => None,
                })
                .filter(|ct| krpeks_core::test(ct, &trapdoor))
                .count()
        }
        BenchmarkScheme::Paeks => {
            let keyword_big = krpaeks_core::hash_to_big(keyword);
            let trapdoor = krpaeks_core::trapdoor(
                &state.paeks_params,
                &state.sender_paeks_pk,
                &state.receiver_paeks_sk,
                &keyword_big,
            );
            records
                .iter()
                .filter_map(|record| match &record.search_index {
                    SearchIndex::Peks(_) => None,
                    SearchIndex::Paeks(ct) => Some(ct),
                })
                .filter(|ct| krpaeks_core::test(ct, &trapdoor))
                .count()
        }
    }
}

fn decrypt_payload(params: &IbeParams, sk: &IbePrivateKey, ct: &IbeCiphertext) -> Option<String> {
    let mut ciphertext = ct.clone();
    let mut plaintext = Plaintext::new();
    kribe_core::decryption(params, sk, &mut ciphertext, &mut plaintext);
    let text = plaintext.to_string();
    if text.is_empty() {
        None
    } else {
        Some(text)
    }
}
```

src-tauri/src/system/benchmark.rs (continued)

```
fn decrypt_payload_checked(
    params: &IbeParams,
    sk: &IbePrivateKey,
    ct: &IbeCiphertext,
    trace: &DecryptTrace,
) -> Option<String> {
    if !ibe_v_id_matches(params, sk, ct) {
        if trace.authorised_attempt {
            print_v_id_mismatch_trace(trace);
        }

        return None;
    }

    catch_unwind(AssertUnwindSafe(|| decrypt_payload(params, sk, ct)))
        .ok()
        .flatten()
}

fn ibe_v_id_matches(params: &IbeParams, sk: &IbePrivateKey, ct: &IbeCiphertext) -> bool {
    let order = params.get_order();

    let u1 = ct.get_u1();
    let u2 = ct.get_u2();
    let c = ct.get_c();

    let mut temp_alpha = u1.clone();
    temp_alpha.add(u2);
    temp_alpha.add(c);
    let alpha = hash_ecp_to_big(temp_alpha);

    let h1_id_alpha = big::BIG::modmul(sk.get_h1ID(), &alpha, order);
    let pow1 = big::BIG::modadd(sk.get_f1ID(), &h1_id_alpha, order);
    let mut temp_u1 = u1.clmul(&pow1, order);

    let h2_id_alpha = big::BIG::modmul(sk.get_h2ID(), &alpha, order);
    let pow2 = big::BIG::modadd(sk.get_f2ID(), &h2_id_alpha, order);
    let temp_u2 = u2.clmul(&pow2, order);

    temp_u1.add(&temp_u2);
    ct.get_v_id().equals(&temp_u1)
}

fn hash_ecp_to_big(input: ecp::ECP) -> big::BIG {
    let mut hasher = SHA3::new(SHAKE256);
    let mut bytes = vec![0; big::MODBYTES + 1];
    input.tobytes(&mut bytes, true);
    hasher.process_array(&bytes);
    let mut output = [0u8; big::MODBYTES];
    hasher.shake(&mut output, big::MODBYTES);
    big::BIG::frombytes(&output)
}

fn params_hash_from_text(text: &str) -> String {
    let mut hasher = DefaultHasher::new();
    text.hash(&mut hasher);
    format!("{:016x}", hasher.finish())
}

fn print_v_id_mismatch_trace(trace: &DecryptTrace) {
    eprintln!("KR-IBE authorised decrypt v_id mismatch trace:");
    eprintln!("  scheme: {}", trace.scheme);
    eprintln!("  sender ID: {}", trace.sender_id);
    eprintln!("  receiver ID: {}", trace.receiver_id);
    eprintln!("  attempted user ID: {}", trace.attempted_user_id);
    eprintln!("  benchmark run: {}", trace.run);
    eprintln!("  dataset size: {}", trace.dataset_size);
    eprintln!("  authorised users: {}", trace.authorised_users);
    eprintln!("  thread ID: {}", trace.thread_id);
    eprintln!("  params hash: {}", trace.params_hash);
    eprintln!("  ciphertext params hash: {}", trace.ciphertext_params_hash);
    eprintln!("  decrypt params hash: {}", trace.decrypt_params_hash);
    eprintln!("  ciphertext/decrypt same params: {}", trace.same_params);
}

fn ibe_ciphertext_size(ct: &IbeCiphertext) -> usize {
    ct.format_full().len()
}

fn search_index_size(index: &SearchIndex) -> usize {
    match index {
        SearchIndex::Peks(ct) => ct.format_full().len(),
        SearchIndex::Paeks(ct) => ct.format_full().len(),
    }
```

src-tauri/src/system/benchmark.rs (continued)

```
}

pub fn load_enron_records(path: &Path, limit: usize) -> BenchmarkResult<Vec<EmailRecord>> {
    let file = File::open(path)?;
    let reader = BufReader::new(file);
    let mut records = Vec::with_capacity(limit);

    for line in reader.lines() {
        let line = line?;
        if !line.starts_with("INSERT INTO") {
            continue;
        }

        for row in parse_insert_values(&line) {
            if let Some(record) = row_to_email_record(row) {
                records.push(record);
                if records.len() >= limit {
                    return Ok(records);
                }
            }
        }
    }

    Ok(records)
}

fn row_to_email_record(fields: Vec<Option<String>>) -> Option<EmailRecord> {
    if fields.len() < 6 {
        return None;
    }

    let sender = clean_field(fields.get(3)?.as_ref()?)?;
    let receiver = clean_field(fields.get(4)?.as_ref()?)?;
    let body = clean_field(fields.get(5)?.as_ref()?)?;
    let subject = extract_subject(&body).unwrap_or_default();
    let keyword = if subject.is_empty() {
        derive_keyword(&body)?
    } else {
        subject
            .split_whitespace()
            .find(|word| word.chars().any(|ch| ch.is_alphabetic()))
            .unwrap_or(&subject)
            .to_string()
    };

    Some(EmailRecord {
        sender,
        receiver,
        subject,
        body,
        keyword: normalise_keyword(&keyword)?,
    })
}

fn parse_insert_values(line: &str) -> Vec<Vec<Option<String>>> {
    let mut rows = Vec::new();
    let mut current_row = Vec::new();
    let mut current_field = String::new();
    let mut in_string = false;
    let mut escaped = false;
    let mut in_row = false;
    let mut field_was_quoted = false;

    for ch in line.chars().skip_while(|ch| *ch != '(') {
        if in_string {
            if escaped {
                current_field.push(match ch {
                    'n' => '\n',
                    'r' => '\r',
                    't' => '\t',
                    '0' => '\0',
                    other => other,
                });
                escaped = false;
            } else if ch == '\\' {
                escaped = true;
            } else if ch == '\'' {
                in_string = false;
            } else {
                current_field.push(ch);
            }
            continue;
        }

        match ch {
```

src-tauri/src/system/benchmark.rs (continued)

```
            '(' if !in_row => {
                in_row = true;
                current_row.clear();
                current_field.clear();
                field_was_quoted = false;
            }
            '\'' if in_row => {
                in_string = true;
                field_was_quoted = true;
            }
            ',' if in_row => {
                push_sql_field(&mut current_row, &mut current_field, field_was_quoted);
                field_was_quoted = false;
            }
            ')' if in_row => {
                push_sql_field(&mut current_row, &mut current_field, field_was_quoted);
                rows.push(current_row.clone());
                current_row.clear();
                field_was_quoted = false;
                in_row = false;
            }
            _ if in_row => current_field.push(ch),
            _ => {}
        }
    }

    rows
}

fn push_sql_field(row: &mut Vec<Option<String>>, field: &mut String, was_quoted: bool) {
    let value = field.trim();
    if !was_quoted && value.eq_ignore_ascii_case("NULL") {
        row.push(None);
    } else {
        row.push(Some(field.clone()));
    }
    field.clear();
}

fn clean_field(value: &str) -> Option<String> {
    let cleaned = value.trim().replace('\0', "");
    if cleaned.is_empty() {
        None
    } else {
        Some(cleaned)
    }
}

fn extract_subject(body: &str) -> Option<String> {
    body.lines()
        .find_map(|line| line.trim().strip_prefix("Subject:"))
        .and_then(clean_field)
}

fn derive_keyword(body: &str) -> Option<String> {
    body.split(|ch: char| !ch.is_alphabetic())
        .find(|word| word.len() >= 4)
        .and_then(normalise_keyword)
}

fn normalise_keyword(keyword: &str) -> Option<String> {
    let cleaned = keyword
        .chars()
        .filter(|ch| ch.is_alphanumeric())
        .collect::<String>()
        .to_lowercase();
    if cleaned.len() >= 4 {
        Some(cleaned)
    } else {
        None
    }
}

fn find_dataset_path() -> BenchmarkResult<PathBuf> {
    for candidate in [
        PathBuf::from("dataset/EnronMailDB.sql"),
        PathBuf::from("../dataset/EnronMailDB.sql"),
        PathBuf::from("/dataset/EnronMailDB.sql"),
    ] {
        if candidate.exists() {
            return Ok(candidate);
        }
    }
    Err("EnronMailDB.sql was not found in dataset/, ../dataset/, or /dataset/".into())
}
```

src-tauri/src/system/benchmark.rs (continued)

```
fn summarise_all(raw_results: &[BenchmarkRawResult]) -> Vec<BenchmarkSummaryResult> {
    let mut summaries = Vec::new();
    for scheme in selected_schemes(raw_results) {
        let mut groups: Vec<(usize, usize)> = raw_results
            .iter()
            .filter(|result| result.scheme == scheme)
            .map(|result| (result.dataset_size, result.authorised_users))
            .collect();
        groups.sort_unstable();
        groups.dedup();

        for (dataset_size, authorised_users) in groups {
            let group: Vec<_> = raw_results
                .iter()
                .filter(|result| {
                    result.scheme == scheme
                        && result.dataset_size == dataset_size
                        && result.authorised_users == authorised_users
                })
                .cloned()
                .collect();
            summaries.push(summarise(&group));
        }
    }
    summaries
}

fn selected_schemes(raw_results: &[BenchmarkRawResult]) -> Vec<BenchmarkScheme> {
    let mut schemes: Vec<_> = raw_results.iter().map(|result| result.scheme).collect();
    schemes.sort_by_key(|scheme| match scheme {
        BenchmarkScheme::Peks => 0,
        BenchmarkScheme::Paeks => 1,
    });
    schemes.dedup();
    schemes
}

fn selected_schemes_from_summaries(summaries: &[BenchmarkSummaryResult]) -> Vec<BenchmarkScheme> {
    let mut schemes: Vec<_> = summaries.iter().map(|summary| summary.scheme).collect();
    schemes.sort_by_key(|scheme| match scheme {
        BenchmarkScheme::Peks => 0,
        BenchmarkScheme::Paeks => 1,
    });
    schemes.dedup();
    schemes
}

fn has_scheme(summaries: &[BenchmarkSummaryResult], scheme: BenchmarkScheme) -> bool {
    summaries.iter().any(|summary| summary.scheme == scheme)
}

fn summarise(results: &[BenchmarkRawResult]) -> BenchmarkSummaryResult {
    let runs = results.len().max(1);
    let first = results
        .first()
        .expect("summary requires at least one result");
    let successful_runs = results
        .iter()
        .filter(|r| {
            r.authenticated_login_passed
                && r.unauthenticated_rejected
                && r.successful_searches > 0
                && r.successful_decryptions > 0
                && r.wrong_keyword_rejected
                && r.wrong_scheme_rejected
                && r.unauthorised_decryption_failed
        })
        .count();

    BenchmarkSummaryResult {
        scheme: first.scheme,
        dataset_size: first.dataset_size,
        authorised_users: first.authorised_users,
        runs: results.len(),
        successful_runs,
        authenticated_login_passed: count_bool(results, |r| r.authenticated_login_passed),
        unauthenticated_rejected: count_bool(results, |r| r.unauthenticated_rejected),
        correct_keyword_search_passed: results.iter().filter(|r| r.successful_searches > 0).count(),
        wrong_keyword_rejected: count_bool(results, |r| r.wrong_keyword_rejected),
        raw_kr_paeks_wrong_keyword_rejected: count_bool(results, |r| {
            r.raw_kr_paeks_wrong_keyword_rejected
        }),
        wrong_scheme_rejected: count_bool(results, |r| r.wrong_scheme_rejected),
        authorised_decryption_passed: results
```

src-tauri/src/system/benchmark.rs (continued)

```
            .iter()
            .filter(|r| r.successful_decryptions > 0)
            .count(),
        unauthorised_decryption_failed: count_bool(results, |r| r.unauthorised_decryption_failed),
        setup_ms: avg(results, |r| r.setup_ms, runs),
        registration_ms: avg(results, |r| r.registration_ms, runs),
        login_ms: avg(results, |r| r.login_ms, runs),
        ibe_encrypt_ms: avg(results, |r| r.ibe_encrypt_ms, runs),
        index_generation_ms: avg(results, |r| r.index_generation_ms, runs),
        search_ms: avg(results, |r| r.search_ms, runs),
        ibe_decrypt_ms: avg(results, |r| r.ibe_decrypt_ms, runs),
        total_upload_ms: avg(results, |r| r.total_upload_ms, runs),
        total_retrieval_ms: avg(results, |r| r.total_retrieval_ms, runs),
        payload_ciphertext_size_bytes: avg_usize(
            results,
            |r| r.payload_ciphertext_size_bytes,
            runs,
        ),
        search_index_size_bytes: avg_usize(results, |r| r.search_index_size_bytes, runs),
        successful_searches: avg_usize(results, |r| r.successful_searches, runs),
        successful_decryptions: avg_usize(results, |r| r.successful_decryptions, runs),
    }
}

fn count_bool<F>(results: &[BenchmarkRawResult], predicate: F) -> usize
where
    F: Fn(&BenchmarkRawResult) -> bool,
{
    results.iter().filter(|result| predicate(result)).count()
}

fn avg<F>(results: &[BenchmarkRawResult], field: F, runs: usize) -> f64
where
    F: Fn(&BenchmarkRawResult) -> f64,
{
    results.iter().map(field).sum::<f64>() / runs as f64
}

fn avg_usize<F>(results: &[BenchmarkRawResult], field: F, runs: usize) -> f64
where
    F: Fn(&BenchmarkRawResult) -> usize,
{
    results.iter().map(field).sum::<usize>() as f64 / runs as f64
}

fn elapsed_ms(start: Instant) -> f64 {
    start.elapsed().as_secs_f64() * 1000.0
}

fn print_header(dataset_path: &Path, config: &BenchmarkConfig) {
    println!("================================================");
    println!("KR Data Sharing Evaluation over EnronMail Dataset");
    println!("================================================");
    println!();
    print_row("Dataset source:", &dataset_path.display().to_string());
    print_row(
        "Detected CPU threads:",
        &config.detected_cpu_threads.to_string(),
    );
    print_row(
        "Using benchmark workers:",
        &config.benchmark_workers.to_string(),
    );
    print_row("Runs per setting:", &config.runs_per_setting.to_string());
    print_row("Dataset sizes:", &format!("{:?}", config.dataset_sizes));
    print_row(
        "Authorised identity counts:",
        &format!("{:?}", config.authorised_user_counts),
    );
    print_row(
        "Search schemes:",
        &format!(
            "[{}]",
            config
                .schemes
                .iter()
                .map(ToString::to_string)
                .collect::<Vec<_>>()
                .join(", ")
        ),
    );
}

fn print_setting_report(summary: &BenchmarkSummaryResult, debug_raw: bool) {
    println!();
```

src-tauri/src/system/benchmark.rs (continued)

```
    println!("--------------------------------------------------");
    print_row("Scheme:", &summary.scheme.to_string());
    print_row("Dataset size:", &summary.dataset_size.to_string());
    print_row(
        "Authorised identities:",
        &summary.authorised_users.to_string(),
    );
    println!("--------------------------------------------------");
    print_count("Successful runs:", summary.successful_runs, summary.runs);
    print_count(
        "Authenticated login passed:",
        summary.authenticated_login_passed,
        summary.runs,
    );
    print_count(
        "Unauthenticated user rejected:",
        summary.unauthenticated_rejected,
        summary.runs,
    );
    print_count(
        "Correct keyword search passed:",
        summary.correct_keyword_search_passed,
        summary.runs,
    );
    if debug_raw && summary.scheme == BenchmarkScheme::Paeks {
        print_count(
            "Raw KR-PAEKS wrong keyword rejected:",
            summary.raw_kr_paeks_wrong_keyword_rejected,
            summary.runs,
        );
    }
    print_count(
        "Wrong keyword rejected:",
        summary.wrong_keyword_rejected,
        summary.runs,
    );
    print_count(
        "Wrong scheme rejected:",
        summary.wrong_scheme_rejected,
        summary.runs,
    );
    print_count(
        "Authorised decryption passed:",
        summary.authorised_decryption_passed,
        summary.runs,
    );
    print_count(
        "Unauthorised decryption failed:",
        summary.unauthorised_decryption_failed,
        summary.runs,
    );
    println!();
    println!("Average Performance:");
    print_ms("Setup time:", summary.setup_ms);
    print_ms("Registration/KeyGen time:", summary.registration_ms);
    print_ms("Login/Auth time:", summary.login_ms);
    print_ms("KR-IBE encryption time:", summary.ibe_encrypt_ms);
    print_ms("Search index generation time:", summary.index_generation_ms);
    print_ms("Search/Trapdoor/Test time:", summary.search_ms);
    print_ms("KR-IBE decryption time:", summary.ibe_decrypt_ms);
    print_ms("Total upload time:", summary.total_upload_ms);
    print_ms("Total retrieval time:", summary.total_retrieval_ms);
    println!();
    println!("Average Size:");
    print_row(
        "Payload ciphertext size:",
        &format!("{:.0} bytes", summary.payload_ciphertext_size_bytes),
    );
    print_row(
        "Search index size:",
        &format!("{:.0} bytes", summary.search_index_size_bytes),
    );
}

fn print_debug_check(result: &BenchmarkRawResult) {
    println!();
    println!("Debug Check:");
    print_row("Sender ID:", &result.debug.sender_id);
    print_row("Receiver ID:", &result.debug.receiver_id);
    print_row("Benchmark run:", &result.debug.run.to_string());
    print_row("Dataset size:", &result.debug.dataset_size.to_string());
    print_row("Thread ID:", &result.debug.thread_id);
    print_row("KR-IBE params hash:", &result.debug.ibe_params_hash);
    print_row("KR-PEKS params hash:", &result.debug.peks_params_hash);
    print_row("KR-PAEKS params hash:", &result.debug.paeks_params_hash);
```

src-tauri/src/system/benchmark.rs (continued)

```
    print_row(
        "Ciphertext/decrypt same params:",
        &result
            .debug
            .ciphertext_and_decrypt_use_same_params
            .to_string(),
    );
    print_row(
        "Index/trapdoor same params:",
        &result
            .debug
            .search_index_and_trapdoor_use_same_params
            .to_string(),
    );
    print_row(
        "Authorised v_id matched:",
        &result.debug.authorised_v_id_matched.to_string(),
    );
    print_row(
        "Unauthorised v_id matched:",
        &result.debug.unauthorised_v_id_matched.to_string(),
    );
    print_row("Stored keyword:", &result.debug.stored_keyword);
    print_row(
        "Correct search keyword:",
        &result.debug.correct_search_keyword,
    );
    print_row("Wrong search keyword:", &result.debug.wrong_search_keyword);
    print_row("Stored keyword_hash:", &result.debug.stored_keyword_hash);
    print_row("Correct keyword_hash:", &result.debug.correct_keyword_hash);
    print_row("Wrong keyword_hash:", &result.debug.wrong_keyword_hash);
    print_row("Correct scheme:", &result.debug.correct_scheme.to_string());
    print_row("Wrong scheme:", &result.debug.wrong_scheme.to_string());
    if result.scheme == BenchmarkScheme::Paeks {
        print_row(
            "Raw KR-PAEKS test result:",
            &result
                .debug
                .raw_kr_paeks_wrong_keyword_test_result
                .to_string(),
        );
    }
    print_row(
        "Final correct app results:",
        &result.debug.correct_keyword_results.to_string(),
    );
    print_row(
        "Final wrong keyword app results:",
        &result.debug.wrong_keyword_results.to_string(),
    );
    print_row(
        "Final wrong scheme app results:",
        &result.debug.wrong_scheme_results.to_string(),
    );
    if result.scheme == BenchmarkScheme::Paeks
        && result.debug.raw_kr_paeks_wrong_keyword_test_result
        && result.debug.wrong_keyword_results == 0
    {
        print_row(
            "PAEKS wrong-keyword note:",
            "Raw test matched, but application keyword_hash rejected it.",
        );
    }
}

fn print_summary_tables(summaries: &[BenchmarkSummaryResult]) {
    for scheme in selected_schemes_from_summaries(summaries) {
        println!();
        println!("Summary Table: {scheme}");
        println!("dataset_size,authorised_users,setup_ms,registration_ms,login_ms,encrypt_ms,index_ms,search_ms,decrypt_ms,upload
_ms,retrieval_ms");
        for summary in summaries.iter().filter(|summary| summary.scheme == scheme) {
            println!(
                "{},{},{:.3},{:.3},{:.3},{:.3},{:.3},{:.3},{:.3},{:.3},{:.3}",
                summary.dataset_size,
                summary.authorised_users,
                summary.setup_ms,
                summary.registration_ms,
                summary.login_ms,
                summary.ibe_encrypt_ms,
                summary.index_generation_ms,
                summary.search_ms,
                summary.ibe_decrypt_ms,
                summary.total_upload_ms,
                summary.total_retrieval_ms
```

src-tauri/src/system/benchmark.rs (continued)

```
            );
        }
    }

    println!();
    if !has_scheme(summaries, BenchmarkScheme::Peks)
        || !has_scheme(summaries, BenchmarkScheme::Paeks)
    {
        return;
    }

    println!("Comparison Table: KR-PEKS vs KR-PAEKS");
    println!("dataset_size,authorised_users,peks_search_ms,paeks_search_ms,peks_index_ms,paeks_index_ms,peks_upload_ms,paeks_uplo
ad_ms");
    for peks in summaries
        .iter()
        .filter(|summary| summary.scheme == BenchmarkScheme::Peks)
    {
        if let Some(paeks) = summaries.iter().find(|summary| {
            summary.scheme == BenchmarkScheme::Paeks
                && summary.dataset_size == peks.dataset_size
                && summary.authorised_users == peks.authorised_users
        }) {
            println!(
                "{},{},{:.3},{:.3},{:.3},{:.3},{:.3},{:.3}",
                peks.dataset_size,
                peks.authorised_users,
                peks.search_ms,
                paeks.search_ms,
                peks.index_generation_ms,
                paeks.index_generation_ms,
                peks.total_upload_ms,
                paeks.total_upload_ms
            );
        }
    }
}

fn print_row(label: &str, value: &str) {
    println!("{label:<35}{value}");
}

fn print_ms(label: &str, value: f64) {
    print_row(label, &format!("{value:.3} ms"));
}

fn print_count(label: &str, value: usize, total: usize) {
    print_row(label, &format!("{value} / {total}"));
}

fn write_raw_csv(path: &str, results: &[BenchmarkRawResult]) -> BenchmarkResult<()> {
    let mut writer = csv_writer(path)?;
    writeln!(
        writer,
        "scheme,dataset_size,authorised_users,run,setup_ms,registration_ms,login_ms,ibe_encrypt_ms,index_generation_ms,search_ms,
ibe_decrypt_ms,total_upload_ms,total_retrieval_ms,payload_ciphertext_size_bytes,search_index_size_bytes,successful_searches,succe
ssful_decryptions,wrong_keyword_rejected,wrong_scheme_rejected,unauthorised_decryption_failed"
    )?;
    for result in results {
        writeln!(
            writer,
            "{},{},{},{},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{},{},{},{},{},{},{}",
            result.scheme,
            result.dataset_size,
            result.authorised_users,
            result.run,
            result.setup_ms,
            result.registration_ms,
            result.login_ms,
            result.ibe_encrypt_ms,
            result.index_generation_ms,
            result.search_ms,
            result.ibe_decrypt_ms,
            result.total_upload_ms,
            result.total_retrieval_ms,
            result.payload_ciphertext_size_bytes,
            result.search_index_size_bytes,
            result.successful_searches,
            result.successful_decryptions,
            result.wrong_keyword_rejected,
            result.wrong_scheme_rejected,
            result.unauthorised_decryption_failed
        )?;
    }
    Ok(())
```

src-tauri/src/system/benchmark.rs (continued)

```
}

fn write_summary_csv(path: &str, summaries: &[BenchmarkSummaryResult]) -> BenchmarkResult<()> {
    let mut writer = csv_writer(path)?;
    writeln!(
        writer,
        "scheme,dataset_size,authorised_users,runs,successful_runs,setup_ms,registration_ms,login_ms,ibe_encrypt_ms,index_generat
ion_ms,search_ms,ibe_decrypt_ms,total_upload_ms,total_retrieval_ms,payload_ciphertext_size_bytes,search_index_size_bytes,successf
ul_searches,successful_decryptions,wrong_keyword_rejected,wrong_scheme_rejected,unauthorised_decryption_failed"
    )?;
    for summary in summaries {
        writeln!(
            writer,
            "{},{},{},{},{},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.2},{:.2},{:.2},{:.2},{},{},{}",
            summary.scheme,
            summary.dataset_size,
            summary.authorised_users,
            summary.runs,
            summary.successful_runs,
            summary.setup_ms,
            summary.registration_ms,
            summary.login_ms,
            summary.ibe_encrypt_ms,
            summary.index_generation_ms,
            summary.search_ms,
            summary.ibe_decrypt_ms,
            summary.total_upload_ms,
            summary.total_retrieval_ms,
            summary.payload_ciphertext_size_bytes,
            summary.search_index_size_bytes,
            summary.successful_searches,
            summary.successful_decryptions,
            summary.wrong_keyword_rejected,
            summary.wrong_scheme_rejected,
            summary.unauthorised_decryption_failed
        )?;
    }
    Ok(())
}

fn write_comparison_csv(path: &str, summaries: &[BenchmarkSummaryResult]) -> BenchmarkResult<()> {
    let mut writer = csv_writer(path)?;
    writeln!(
        writer,
        "dataset_size,authorised_users,peks_search_ms,paeks_search_ms,search_delta_ms,peks_index_generation_ms,paeks_index_genera
tion_ms,index_delta_ms,peks_total_upload_ms,paeks_total_upload_ms,upload_delta_ms,peks_total_retrieval_ms,paeks_total_retrieval_m
s,retrieval_delta_ms"
    )?;
    for peks in summaries
        .iter()
        .filter(|summary| summary.scheme == BenchmarkScheme::Peks)
    {
        if let Some(paeks) = summaries.iter().find(|summary| {
            summary.scheme == BenchmarkScheme::Paeks
                && summary.dataset_size == peks.dataset_size
                && summary.authorised_users == peks.authorised_users
        }) {
            writeln!(
                writer,
                "{},{},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6},{:.6}",
                peks.dataset_size,
                peks.authorised_users,
                peks.search_ms,
                paeks.search_ms,
                paeks.search_ms - peks.search_ms,
                peks.index_generation_ms,
                paeks.index_generation_ms,
                paeks.index_generation_ms - peks.index_generation_ms,
                peks.total_upload_ms,
                paeks.total_upload_ms,
                paeks.total_upload_ms - peks.total_upload_ms,
                peks.total_retrieval_ms,
                paeks.total_retrieval_ms,
                paeks.total_retrieval_ms - peks.total_retrieval_ms
            )?;
        }
    }
    Ok(())
}

fn csv_writer(path: &str) -> BenchmarkResult<BufWriter<File>> {
    Ok(BufWriter::new(File::create(path)?))
}
```

src-tauri/src/system/download.rs

```
use crate::kr_ibe::{main as kribe_core, plaintext::Plaintext};
use crate::system::state::{SharedPayload, APP_STATE};

pub fn download(user: &str, index: usize) -> Result<SharedPayload, String> {
    let state = APP_STATE.lock().map_err(|_| "State lock failed")?;

    if !state.active_sessions.get(user).unwrap_or(&false) {
        return Err("User is not authenticated.".to_string());
    }

    let params = state
        .ibe_params
        .as_ref()
        .ok_or("IBE params not initialised.")?
        .clone();

    let sk = state
        .users
        .get(user)
        .ok_or("User private key not found.")?
        .clone();

    let data = state
        .database
        .get(index)
        .ok_or("Invalid ciphertext index.")?;

    if data.owner != user {
        return Err("Access denied. This ciphertext does not belong to this user.".to_string());
    }

    let mut ct = data.ct.clone();

    drop(state);

    let mut pt = Plaintext::new();

    kribe_core::decryption(&params, &sk, &mut ct, &mut pt);

    let plaintext = pt.to_string();
    match serde_json::from_str::<SharedPayload>(&plaintext) {
        Ok(payload) => Ok(payload),
        Err(_) => Ok(SharedPayload {
            payload_type: "text".to_string(),
            content: plaintext,
            file_name: None,
            mime_type: None,
            content_base64: None,
        }),
    }
}
```

src-tauri/src/system/mod.rs

```
pub mod auth;
pub mod benchmark;
pub mod download;
pub mod search;
pub mod setup;
pub mod state;
pub mod upload;
pub mod user;
pub mod utils;
```

src-tauri/src/system/search.rs

```
use crate::system::state::{SearchIndex, SearchScheme, APP_STATE};
use crate::system::utils::keyword_hash;

use crate::kr_paeks::main as krpaeks_core;
use crate::kr_peks::main as krpeks_core;

pub fn search(user: &str, keyword: &str, scheme: &str) -> Result<Vec<usize>, String> {
    let state = APP_STATE.lock().map_err(|_| "State lock failed")?;

    if !state.active_sessions.get(user).unwrap_or(&false) {
        return Err("User is not authenticated.".to_string());
    }

    let selected_scheme = match scheme.to_lowercase().as_str() {
        "peks" => SearchScheme::Peks,
        "paeks" => SearchScheme::Paeks,
        _ => return Err("Invalid search scheme. Use 'peks' or 'paeks'.".to_string()),
    };

    let search_hash = keyword_hash(keyword);
    let mut results = Vec::new();

    match selected_scheme {
        SearchScheme::Peks => {
            let peks_params = state
                .peks_params
                .as_ref()
                .ok_or("PEKS params not initialised.")?;

            let peks_sk = state.peks_sk.as_ref().ok_or("PEKS private key missing.")?;

            let keyword_bytes = keyword.as_bytes().to_vec();

            let trapdoor = krpeks_core::trapdoor(peks_params, peks_sk, &keyword_bytes);

            for (index, data) in state.database.iter().enumerate() {
                if data.owner != user {
                    continue;
                }

                if data.keyword_hash != search_hash {
                    continue;
                }

                if let SearchIndex::Peks(peks_index) = &data.search_index {
                    let matched = krpeks_core::test(peks_index, &trapdoor);

                    println!(
                        "PEKS search debug => keyword: {}, index: {}, owner: {}, sender: {}, matched: {}",
                        keyword,
                        index,
                        data.owner,
                        data.sender,
                        matched
                    );

                    if matched {
                        results.push(index);
                    }
                }
            }
        }

        SearchScheme::Paeks => {
            let paeks_params = state
                .paeks_params
                .as_ref()
                .ok_or("PAEKS params not initialised.")?;

            let receiver_paeks = state
                .paeks_users
                .get(user)
                .ok_or("Receiver PAEKS keypair missing.")?;

            let keyword_big = krpaeks_core::hash_to_big(keyword);

            for (index, data) in state.database.iter().enumerate() {
                if data.owner != user {
                    continue;
                }
```

src-tauri/src/system/search.rs (continued)

```
                if data.keyword_hash != search_hash {
                    continue;
                }

                let sender_paeks = match state.paeks_users.get(&data.sender) {
                    Some(keys) => keys,
                    None => continue,
                };

                if let SearchIndex::Paeks(paeks_index) = &data.search_index {
                    let trapdoor = krpaeks_core::trapdoor(
                        paeks_params,
                        &sender_paeks.pk,
                        &receiver_paeks.sk,
                        &keyword_big,
                    );

                    let matched = krpaeks_core::test(paeks_index, &trapdoor);

                    println!(
                        "PAEKS search debug => keyword: {}, index: {}, owner: {}, sender: {}, matched: {}",
                        keyword,
                        index,
                        data.owner,
                        data.sender,
                        matched
                    );

                    if matched {
                        results.push(index);
                    }
                }
            }
        }
    }

    println!("Search result count: {}", results.len());

    Ok(results)
}
```

src-tauri/src/system/setup.rs

```
use crate::system::state::APP_STATE;

use crate::kr_ibe::main as kribe_core;
use crate::kr_ibi::main as kribi_core;
use crate::kr_paeks::main as krpaeks_core;
use crate::kr_peks::main as krpeks_core;

use crate::kr_ibe::params::Params as IbeParams;
use crate::kr_ibi::params::Params as IbiParams;

use crate::kr_peks::{
    params::Params as PeksParams, private_key::PrivateKey as PeksPrivateKey,
    public_key::PublicKey as PeksPublicKey,
};

use crate::kr_paeks::params::Params as PaeksParams;

pub fn setup_all(k: usize) {
    let mut state = APP_STATE.lock().unwrap();

    // ========================
    // KR-IBE Setup
    // ========================
    let mut ibe_params = IbeParams::new();
    kribe_core::setup(&mut ibe_params, k);
    state.ibe_params = Some(ibe_params);

    // ========================
    // KR-IBI Setup
    // ========================
    let mut ibi_params = IbiParams::new();
    kribi_core::setup(&mut ibi_params, k);
    state.ibi_params = Some(ibi_params);

    // ========================
    // KR-PEKS Setup
    // ========================
    let mut peks_params = PeksParams::new();
    krpeks_core::setup(&mut peks_params, k);

    let mut peks_pk = PeksPublicKey::new();
    let mut peks_sk = PeksPrivateKey::new();

    krpeks_core::keygen(&peks_params, &mut peks_pk, &mut peks_sk);

    state.peks_params = Some(peks_params);
    state.peks_pk = Some(peks_pk);
    state.peks_sk = Some(peks_sk);

    // ========================
    // KR-PAEKS Setup
    // ========================
    let mut paeks_params = PaeksParams::new();
    krpaeks_core::setup(&mut paeks_params, k);

    state.paeks_params = Some(paeks_params);

    println!("System setup completed.");
}
```

src-tauri/src/system/state.rs

```
use once_cell::sync::Lazy;
use serde::{Deserialize, Serialize};
use std::collections::HashMap;
use std::sync::Mutex;

use mcore::ed25519::big;
use mcore::ed25519::ecp;

use crate::kr_ibe::{
    ciphertext::Ciphertext as IbeCiphertext, params::Params as IbeParams,
    private_key::PrivateKey as IbePrivateKey,
};

use crate::kr_ibi::params::Params as IbiParams;

use crate::kr_peks::{
    ciphertext::Ciphertext as PeksCiphertext, params::Params as PeksParams,
    private_key::PrivateKey as PeksPrivateKey, public_key::PublicKey as PeksPublicKey,
};

use crate::kr_paeks::{
    ciphertext::Ciphertext as PaeksCiphertext, params::Params as PaeksParams,
    private_key::PrivateKey as PaeksPrivateKey, public_key::PublicKey as PaeksPublicKey,
};

pub struct LoginSession {
    pub g_r: (ecp::ECP, ecp::ECP),
    pub c1: big::BIG,
    pub c2: big::BIG,
    pub r: (big::BIG, big::BIG),
}

#[derive(Clone)]
pub struct PaeksKeyPair {
    pub pk: PaeksPublicKey,
    pub sk: PaeksPrivateKey,
}

#[derive(Clone)]
pub enum SearchScheme {
    Peks,
    Paeks,
}

#[derive(Clone)]
pub enum SearchIndex {
    Peks(PeksCiphertext),
    Paeks(PaeksCiphertext),
}

#[derive(Clone)]
pub struct StoredData {
    pub ct: IbeCiphertext,
    pub search_index: SearchIndex,
    pub search_scheme: SearchScheme,
    pub sender: String,
    pub owner: String,
    pub keyword_hash: String,
}

#[derive(Clone, Deserialize, Serialize)]
#[serde(rename_all = "camelCase")]
pub struct SharedPayload {
    pub payload_type: String,
    pub content: String,
    pub file_name: Option<String>,
    pub mime_type: Option<String>,
    pub content_base64: Option<String>,
}

pub struct AppState {
    pub ibe_params: Option<IbeParams>,
    pub ibi_params: Option<IbiParams>,

    // KR-PEKS
    pub peks_params: Option<PeksParams>,
    pub peks_pk: Option<PeksPublicKey>,
    pub peks_sk: Option<PeksPrivateKey>,

    // KR-PAEKS
```

src-tauri/src/system/state.rs (continued)

```
    pub paeks_params: Option<PaeksParams>,
    pub paeks_users: HashMap<String, PaeksKeyPair>,

    // KR-IBE user private keys
    pub users: HashMap<String, IbePrivateKey>,

    pub database: Vec<StoredData>,

    pub login_sessions: HashMap<String, LoginSession>,
    pub active_sessions: HashMap<String, bool>,
}

pub static APP_STATE: Lazy<Mutex<AppState>> = Lazy::new(|| {
    Mutex::new(AppState {
        ibe_params: None,
        ibi_params: None,

        peks_params: None,
        peks_pk: None,
        peks_sk: None,

        paeks_params: None,
        paeks_users: HashMap::new(),

        users: HashMap::new(),

        database: Vec::new(),

        login_sessions: HashMap::new(),
        active_sessions: HashMap::new(),
    })
});
```

src-tauri/src/system/upload.rs

```
use crate::system::state::{SearchIndex, SearchScheme, SharedPayload, StoredData, APP_STATE};
use crate::system::utils::id_to_bytes;
use crate::system::utils::keyword_hash;

use crate::kr_ibe::{ciphertext::Ciphertext as IbeCiphertext, main as kribe_core};

use crate::kr_paeks::main as krpaeks_core;
use crate::kr_peks::main as krpeks_core;

pub fn upload(
    sender: &str,
    receiver: &str,
    msg: &str,
    keyword: &str,
    scheme: &str,
    payload_type: &str,
    file_name: Option<String>,
    mime_type: Option<String>,
    content_base64: Option<String>,
) -> Result<(), String> {
    let mut state = APP_STATE.lock().map_err(|_| "State lock failed")?;

    if !state.active_sessions.get(sender).unwrap_or(&false) {
        return Err("Sender is not authenticated.".to_string());
    }

    if !state.users.contains_key(receiver) {
        return Err("Receiver is not registered.".to_string());
    }

    let ibe_params = state
        .ibe_params
        .as_ref()
        .ok_or("IBE params not initialised.")?;

    let mut ibe_ct = IbeCiphertext::new();

    let receiver_bytes = id_to_bytes(receiver);

    let payload = match payload_type {
        "text" => SharedPayload {
            payload_type: "text".to_string(),
            content: msg.to_string(),
            file_name: None,
            mime_type: None,
            content_base64: None,
        },
        "file" | "image" => {
            let content_base64 = content_base64
                .filter(|content| !content.is_empty())
                .ok_or("File content is missing.")?;

            SharedPayload {
                payload_type: payload_type.to_string(),
                content: msg.to_string(),
                file_name,
                mime_type,
                content_base64: Some(content_base64),
            }
        }
        _ => return Err("Unsupported payload type.".to_string()),
    };

    let payload_json = serde_json::to_string(&payload)
        .map_err(|error| format!("Failed to serialise upload payload: {error}"))?;

    let msg_bytes = payload_json.as_bytes().to_vec();

    kribe_core::encryption(ibe_params, &mut ibe_ct, &receiver_bytes, &msg_bytes);

    let selected_scheme = match scheme.to_lowercase().as_str() {
        "peks" => SearchScheme::Peks,
        "paeks" => SearchScheme::Paeks,
        _ => return Err("Invalid search scheme. Use 'peks' or 'paeks'.".to_string()),
    };

    let search_index = match selected_scheme {
        SearchScheme::Peks => {
            let peks_params = state
                .peks_params
```

src-tauri/src/system/upload.rs (continued)

```
                .as_ref()
                .ok_or("PEKS params not initialised.")?;

            let peks_pk = state.peks_pk.as_ref().ok_or("PEKS public key missing.")?;

            let keyword_bytes = keyword.as_bytes().to_vec();

            let index = krpeks_core::peks(peks_params, peks_pk, &keyword_bytes)
                .ok_or("KR-PEKS encryption failed.")?;

            SearchIndex::Peks(index)
        }

        SearchScheme::Paeks => {
            let paeks_params = state
                .paeks_params
                .as_ref()
                .ok_or("PAEKS params not initialised.")?;

            let sender_paeks = state
                .paeks_users
                .get(sender)
                .ok_or("Sender PAEKS keypair missing.")?;

            let receiver_paeks = state
                .paeks_users
                .get(receiver)
                .ok_or("Receiver PAEKS keypair missing.")?;

            let keyword_big = krpaeks_core::hash_to_big(keyword);

            let index = krpaeks_core::encrypt(
                paeks_params,
                &receiver_paeks.pk,
                &sender_paeks.sk,
                &keyword_big,
            );

            SearchIndex::Paeks(index)
        }
    };

    state.database.push(StoredData {
        ct: ibe_ct,
        search_index,
        search_scheme: selected_scheme,
        sender: sender.to_string(),
        owner: receiver.to_string(),
        keyword_hash: keyword_hash(keyword),
    });

    Ok(())
}
```

src-tauri/src/system/user.rs

```
use crate::system::state::{PaeksKeyPair, APP_STATE};
use crate::system::utils::id_to_bytes;

use crate::kr_ibe::{main as kribe_core, private_key::PrivateKey as IbePrivateKey};

use crate::kr_paeks::{
    main as krpaeks_core, private_key::PrivateKey as PaeksPrivateKey,
    public_key::PublicKey as PaeksPublicKey,
};

pub fn register_user(id: &str) -> Result<String, String> {
    let mut state = APP_STATE.lock().map_err(|_| "State lock failed")?;

    if state.users.contains_key(id) {
        return Ok(format!("User {} already registered.", id));
    }

    let ibe_params = state
        .ibe_params
        .as_ref()
        .ok_or("IBE params not initialised. Call setup_all() first.")?;

    let paeks_params = state
        .paeks_params
        .as_ref()
        .ok_or("PAEKS params not initialised. Call setup_all() first.")?;

    let mut ibe_sk = IbePrivateKey::new();
    let id_bytes = id_to_bytes(id);

    kribe_core::extract(ibe_params, &mut ibe_sk, &id_bytes);

    let mut paeks_pk = PaeksPublicKey::new();
    let mut paeks_sk = PaeksPrivateKey::new();

    krpaeks_core::keygen(paeks_params, &mut paeks_pk, &mut paeks_sk);

    state.users.insert(id.to_string(), ibe_sk);
    state.paeks_users.insert(
        id.to_string(),
        PaeksKeyPair {
            pk: paeks_pk,
            sk: paeks_sk,
        },
    );

    println!("Register called for ID: {}", id);

    Ok(format!("User {} registered successfully.", id))
}
```

src-tauri/src/system/utils.rs

```
pub fn id_to_bytes(id: &str) -> Vec<u8> {
    let mut bytes = id.as_bytes().to_vec();

    if bytes.len() < 32 {
        bytes.resize(32, 0);
    }

    bytes
}

use sha2::{Digest, Sha256};

pub fn keyword_hash(keyword: &str) -> String {
    let normalised = keyword.trim().to_lowercase();
    let digest = Sha256::digest(normalised.as_bytes());
    hex::encode(digest)
}
```

src-tauri/tauri.conf.json

```
{
  "$schema": "https://schema.tauri.app/config/2",
  "productName": "k-resilient-suite-react",
  "version": "1.0.0",
  "identifier": "com.k-resilient-suite-react.app",
  "build": {
    "beforeDevCommand": "npm run dev",
    "devUrl": "http://localhost:1420",
    "beforeBuildCommand": "npm run build",
    "frontendDist": "../dist"
  },
  "app": {
    "windows": [
      {
        "title": "k-resilient-suite-react",
        "width": 1000,
        "height": 800
      }
    ],
    "security": {
      "csp": null
    }
  },
  "bundle": {
    "active": true,
    "targets": "all",
    "icon": [
      "icons/32x32.png",
      "icons/128x128.png",
      "icons/128x128@2x.png",
      "icons/icon.icns",
      "icons/icon.ico"
    ]
  }
}
```

src/App.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f5f7fb;
  color: #222;
}

.app-container {
  max-width: 900px;
  margin: 0 auto;
  padding: 24px;
}

.auth-card,
.section-card {
  background: white;
  padding: 20px;
  border-radius: 12px;
  margin-bottom: 20px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
}

.auth-card input,
.section-card input,
.section-card textarea {
  width: 100%;
  margin-top: 10px;
  margin-bottom: 14px;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 8px;
  box-sizing: border-box;
}

.section-card textarea {
  min-height: 100px;
  resize: vertical;
}

.mode-row {
  display: flex;
  gap: 10px;
  margin: 4px 0 14px;
}

button {
  padding: 10px 16px;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  background: #2563eb;
  color: white;
  margin-right: 10px;
}

button:hover {
  background: #1d4ed8;
}

.secondary-button {
  background: #e2e8f0;
  color: #1e293b;
}

.secondary-button:hover {
  background: #cbd5e1;
}

.active-mode {
  background: #0f172a;
}

button:disabled {
  background: #94a3b8;
  cursor: not-allowed;
}

.auth-buttons {
  display: flex;
  gap: 12px;
```

src/App.css (continued)

```
  margin-top: 10px;
}

.status-text {
  margin-top: 14px;
  color: #444;
}

.file-meta {
  margin: -4px 0 14px;
  color: #475569;
  font-size: 14px;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 24px;
}

.welcome-text {
  margin-right: 12px;
  font-weight: bold;
}

.results-block {
  margin-top: 20px;
}

.message-box {
  background: #f8fafc;
  border: 1px solid #dbeafe;
  padding: 12px;
  border-radius: 8px;
  margin-top: 10px;
}

.downloaded-file {
  display: flex;
  gap: 14px;
  align-items: flex-start;
  margin-top: 10px;
}

.downloaded-file img {
  width: 160px;
  max-height: 160px;
  object-fit: contain;
  border-radius: 8px;
  border: 1px solid #cbd5e1;
  background: white;
}

.downloaded-file a {
  color: #1d4ed8;
  font-weight: bold;
}
```

src/App.jsx

```
import { useEffect, useState } from "react";
import { invoke } from "@tauri-apps/api/core";
import AuthPage from "./components/AuthPage";
import Dashboard from "./components/Dashboard";
import "./App.css";

function App() {
  const [currentUser, setCurrentUser] = useState("");
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [ready, setReady] = useState(false);
  const [setupError, setSetupError] = useState("");

  useEffect(() => {
    async function init() {
      try {
        await invoke("setup_all", { k: 3 });
        setReady(true);
      } catch (err) {
        console.error(err);
        setSetupError(String(err));
      }
    }

    init();
  }, []);

  if (setupError) {
    return <p>Setup failed: {setupError}</p>;
  }

  if (!ready) {
    return <p>Initialising cryptographic system...</p>;
  }

  return (
    <div className="app-container">
      {!isLoggedIn ? (
        <AuthPage
          onLoginSuccess={(id) => {
            setCurrentUser(id);
            setIsLoggedIn(true);
          }}
        />
      ) : (
        <Dashboard
          user={currentUser}
          onLogout={() => {
            setCurrentUser("");
            setIsLoggedIn(false);
          }}
        />
      )}
    </div>
  );
}

export default App;
```

src/components/AuthPage.jsx

```
import { useState } from "react";
import { invoke } from "@tauri-apps/api/core";

export default function AuthPage({ onLoginSuccess }) {
  const [userId, setUserId] = useState("");
  const [status, setStatus] = useState("Please register or login.");
  const [isBusy, setIsBusy] = useState(false);

  const handleRegister = async () => {
    if (!userId.trim()) {
      setStatus("Please enter a user ID first.");
      return;
    }

    try {
      setIsBusy(true);
      setStatus("Registering new user...");

      await invoke("register", { id: userId });

      setStatus(`User "${userId}" registered successfully. You may now login.`);
    } catch (error) {
      console.error(error);
      setStatus(`Registration failed: ${error}`);
    } finally {
      setIsBusy(false);
    }
  };

  const handleLogin = async () => {
    if (!userId.trim()) {
      setStatus("Please enter a user ID first.");
      return;
    }

    try {
      setIsBusy(true);
      setStatus("Starting KR-IBI login...");

      // Step 1: server creates challenge
      const [c1, c2] = await invoke("login_start", { id: userId });

      setStatus("Challenge received. Generating KR-IBI response...");

      // Step 2: client/backend-side response generation
      // In your current prototype, backend stores session state,
      // so only ID is needed here.
      const [s1, s2] = await invoke("login_respond", { id: userId });

      setStatus("Verifying login...");

      // Step 3: verification
      const verified = await invoke("login_verify", {
        id: userId,
        s1,
        s2,
      });

      if (verified) {
        setStatus(`Login successful. Welcome, ${userId}.`);
        onLoginSuccess(userId);
      } else {
        setStatus("Login failed. Invalid KR-IBI proof.");
      }
    } catch (error) {
      console.error(error);
      setStatus(`Login error: ${error}`);
    } finally {
      setIsBusy(false);
    }
  };

  return (
    <div className="auth-card">
      <h1>Secure Data Sharing Platform</h1>
      <h2>KR-IBI Authentication</h2>

      <input
        type="text"
        placeholder="Enter user ID"
```

src/components/AuthPage.jsx (continued)

```
        value={userId}
        onChange={(e) => setUserId(e.target.value)}
        disabled={isBusy}
      />

      <div className="auth-buttons">
        <button onClick={handleRegister} disabled={isBusy}>
          Register New User
        </button>

        <button onClick={handleLogin} disabled={isBusy}>
          Login
        </button>
      </div>

      <p className="status-text">{status}</p>
    </div>
  );
}
```

src/components/Dashboard.jsx

```
import Upload from "./Upload";
import Search from "./Search";

export default function Dashboard({ user, onLogout }) {
  return (
    <div className="dashboard-container">
      <div className="dashboard-header">
        <h1>Dashboard</h1>
        <div>
          <span className="welcome-text">Logged in as: {user}</span>
          <button onClick={onLogout}>Logout</button>
        </div>
      </div>

      <Upload user={user} />
      <Search user={user} />
    </div>
  );
}
```

src/components/Search.jsx

```
import { useState } from "react";
import { invoke } from "@tauri-apps/api/core";

export default function Search({ user }) {
  const [keyword, setKeyword] = useState("");
  const [scheme, setScheme] = useState("peks");
  const [results, setResults] = useState([]);
  const [downloadedMessages, setDownloadedMessages] = useState([]);
  const [status, setStatus] = useState("");
  const [hasSearched, setHasSearched] = useState(false);

  const handleSearch = async () => {
    if (!keyword.trim()) {
      setStatus("Please enter a keyword.");
      setResults([]);
      setDownloadedMessages([]);
      setHasSearched(false);
      return;
    }

    try {
      setResults([]);
      setDownloadedMessages([]);
      setHasSearched(true);
      setStatus(`Searching encrypted data using ${scheme.toUpperCase()}...`);

      const indexes = await invoke("search_keyword", {
        user,
        keyword,
        scheme,
      });

      setResults(indexes);

      if (indexes.length === 0) {
        setStatus("No matching ciphertext found.");
      } else {
        setStatus(`Found ${indexes.length} result(s).`);
      }
    } catch (error) {
      console.error(error);
      setResults([]);
      setDownloadedMessages([]);
      setStatus(`Search failed: ${error}`);
    }
  };

  const handleDownload = async (index) => {
    try {
      const payload = await invoke("download_file", {
        user,
        index,
      });

      setDownloadedMessages((prev) => [
        ...prev,
        { index, payload },
      ]);
    } catch (error) {
      console.error(error);
      setStatus(`Download failed: ${error}`);
    }
  };

  const buildDataUrl = (payload) => {
    if (!payload?.contentBase64) {
      return "";
    }

    return `data:${payload.mimeType || "application/octet-stream"};base64,${payload.contentBase64}`;
  };

  return (
    <div className="section-card">
      <h2>Search over Encrypted Data</h2>

      <select value={scheme} onChange={(e) => setScheme(e.target.value)}>
        <option value="peks">KR-PEKS</option>
        <option value="paeks">KR-PAEKS</option>
      </select>
```

src/components/Search.jsx (continued)

```
      <input
        type="text"
        placeholder="Keyword"
        value={keyword}
        onChange={(e) => setKeyword(e.target.value)}
      />

      <button onClick={handleSearch}>Search</button>

      <p className="status-text">{status}</p>

      <div className="results-block">
        <h3>Search Results</h3>

        {!hasSearched ? (
          <p>No search performed yet.</p>
        ) : results.length === 0 ? (
          <p>No matching ciphertext found.</p>
        ) : (
          <ul>
            {results.map((index) => (
              <li key={index}>
                Ciphertext #{index}
                <button onClick={() => handleDownload(index)}>
                  Download
                </button>
              </li>
            ))}
          </ul>
        )}
      </div>

      <div className="results-block">
        <h3>Downloaded Messages</h3>

        {downloadedMessages.length === 0 ? (
          <p>No downloaded messages yet.</p>
        ) : (
          downloadedMessages.map((item, i) => (
            <div key={`${item.index}-${i}`} className="message-box">
              <strong>From result #{item.index}</strong>
              {item.payload.payloadType === "text" ? (
                <p>{item.payload.content}</p>
              ) : (
                <div className="downloaded-file">
                  {item.payload.payloadType === "image" && (
                    <img
                      src={buildDataUrl(item.payload)}
                      alt={item.payload.fileName || "Downloaded image"}
                    />
                  )}
                  <div>
                    <p>{item.payload.content || "Encrypted file restored."}</p>
                    <a
                      href={buildDataUrl(item.payload)}
                      download={item.payload.fileName || "downloaded-file"}
                    >
                      Download {item.payload.fileName || "file"}
                    </a>
                  </div>
                </div>
              )}
            </div>
          ))
        )}
      </div>
    </div>
  );
}
```

src/components/Upload.jsx

```
import { useState } from "react";
import { invoke } from "@tauri-apps/api/core";

export default function Upload({ user }) {
  const [receiver, setReceiver] = useState("");
  const [message, setMessage] = useState("");
  const [keyword, setKeyword] = useState("");
  const [scheme, setScheme] = useState("peks");
  const [payloadType, setPayloadType] = useState("text");
  const [selectedFile, setSelectedFile] = useState(null);
  const [status, setStatus] = useState("");

  const readFileAsBase64 = (file) =>
    new Promise((resolve, reject) => {
      const reader = new FileReader();

      reader.onload = () => {
        const result = String(reader.result || "");
        resolve(result.includes(",") ? result.split(",")[1] : result);
      };

      reader.onerror = () => reject(reader.error);
      reader.readAsDataURL(file);
    });

  const handleUpload = async () => {
    if (!receiver.trim() || !keyword.trim()) {
      setStatus("Please fill in receiver and keyword.");
      return;
    }

    if (payloadType === "text" && !message.trim()) {
      setStatus("Please enter a message.");
      return;
    }

    if (payloadType !== "text" && !selectedFile) {
      setStatus("Please choose a file.");
      return;
    }

    try {
      setStatus(`Encrypting and uploading using ${scheme.toUpperCase()}...`);

      const contentBase64 =
        payloadType === "text" ? null : await readFileAsBase64(selectedFile);

      const effectivePayloadType =
        payloadType === "file" && selectedFile?.type.startsWith("image/")
          ? "image"
          : payloadType;

      await invoke("upload_file", {
        sender: user,
        receiver,
        msg: message,
        keyword,
        scheme,
        payloadType: effectivePayloadType,
        fileName: selectedFile?.name ?? null,
        mimeType: selectedFile?.type || null,
        contentBase64,
      });

      setStatus("Upload successful.");
      setReceiver("");
      setMessage("");
      setKeyword("");
      setSelectedFile(null);
    } catch (error) {
      console.error(error);
      setStatus(`Upload failed: ${error}`);
    }
  };

  return (
    <div className="section-card">
      <h2>Upload Secure Data</h2>

      <input
```

src/components/Upload.jsx (continued)

```
        type="text"
        placeholder="Receiver ID"
        value={receiver}
        onChange={(e) => setReceiver(e.target.value)}
      />

      <label>Search Scheme</label>
      <select value={scheme} onChange={(e) => setScheme(e.target.value)}>
        <option value="peks">KR-PEKS</option>
        <option value="paeks">KR-PAEKS</option>
      </select>

      <div className="mode-row">
        <button
          type="button"
          className={payloadType === "text" ? "active-mode" : "secondary-button"}
          onClick={() => {
            setPayloadType("text");
            setSelectedFile(null);
          }}
        >
          Text
        </button>

        <button
          type="button"
          className={payloadType === "file" ? "active-mode" : "secondary-button"}
          onClick={() => setPayloadType("file")}
        >
          File / Image
        </button>
      </div>

      {payloadType === "text" ? (
        <textarea
          placeholder="Message"
          value={message}
          onChange={(e) => setMessage(e.target.value)}
        />
      ) : (
        <>
          <input
            type="file"
            onChange={(e) => setSelectedFile(e.target.files?.[0] ?? null)}
          />

          <textarea
            placeholder="Optional note"
            value={message}
            onChange={(e) => setMessage(e.target.value)}
          />

          {selectedFile && (
            <p className="file-meta">
              Selected: {selectedFile.name} ({Math.ceil(selectedFile.size / 1024)} KB)
            </p>
          )}
        </>
      )}

      <input
        type="text"
        placeholder="Keyword"
        value={keyword}
        onChange={(e) => setKeyword(e.target.value)}
      />

      <button onClick={handleUpload}>Upload</button>

      <p className="status-text">{status}</p>
    </div>
  );
}
```

src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

const host = process.env.TAURI_DEV_HOST;

// https://vitejs.dev/config/
export default defineConfig(async () => ({
  plugins: [react()],

  // Vite options tailored for Tauri development and only applied in `tauri dev` or `tauri build`
  //
  // 1. prevent vite from obscuring rust errors
  clearScreen: false,
  // 2. tauri expects a fixed port, fail if that port is not available
  server: {
    port: 1420,
    strictPort: true,
    host: host || false,
    hmr: host
      ? {
          protocol: "ws",
          host,
          port: 1421,
        }
      : undefined,
    watch: {
      // 3. tell vite to ignore watching `src-tauri`
      ignored: ["**/src-tauri/**"],
    },
  },
}));
```